

SQL PROJECTS QUIRIES

```
-- SELECT * FROM table --
```

```
-- Query 1 --
```

```
SELECT * FROM Dirty_Sales_Dataset;
```

```
-- A2.1.2 Selecting specific columns only --
```

```
-- Query 1 --
```

```
SELECT Order_ID, Customer_Name, Sales FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT Order_ID, Order_Date, Product_Name, Quantity, Sales FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT Order_ID, Unit_Price, Discount, Sales, Cost, Profit FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --
```

```
SELECT Sales, Order_ID, Customer_Name FROM Dirty_Sales_Dataset;
```

```
-- Query 5 --
```

```
SELECT Customer_Name, Product_Name, Sales, Profit FROM Dirty_Sales_Dataset;
```

```
-- A2.1.3 Order of clauses – SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT --
```

```
-- Query 1 --
```

```
SELECT Category, COUNT(*) AS total_orders  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Category  
HAVING COUNT(*) > 100  
ORDER BY total_orders DESC  
LIMIT 5;
```

```
-- Query 2 --
```

```
SELECT Region, COUNT(*) AS order_count  
FROM Dirty_Sales_Dataset  
WHERE Quantity > 0  
GROUP BY Region  
ORDER BY order_count DESC;
```

```
-- Why SELECT * is bad in production --
```

```
-- Query 1 --
```

```
SELECT * FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT Customer_Name, Sales FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT Customer_Name, Product_Name, Sales, Order_Date
FROM Dirty_Sales_Dataset;
```

-- A2.2.1 Column alias using AS keyword --

-- Query 1 --

```
SELECT Customer_Name AS Customer, Sales AS Total_Sales
FROM Dirty_Sales_Dataset;
```

-- Query 2 --

```
SELECT Order_ID, Sales, Cost, (Sales - Cost) AS Calculated_Profit
FROM Dirty_Sales_Dataset;
```

-- Query 3 --

```
SELECT
    Order_ID AS OrderNumber,
    Customer_Name AS Customer,
    Product_Name AS Product,
    Sales AS Revenue
FROM Dirty_Sales_Dataset;
```

-- Query 4 --

```
SELECT Sales AS "Total Sales Amount" FROM Dirty_Sales_Dataset;
```

-- Query 5 --

```
SELECT Sales AS Total_Sales FROM Dirty_Sales_Dataset WHERE Sales > 1000;
```

-- Query 6 --

```
SELECT
    Order_ID AS Order_Number,
    Customer_Name AS Customer,
    Product_Name AS Product,
    Quantity,
    Unit_Price,
    Sales AS Total_Sales
FROM Dirty_Sales_Dataset
LIMIT 20;
```

-- A2.2.2 Creating calculated columns (e.g., price * quantity AS revenue) --

-- Query 1 --

```
SELECT
    Order_ID,
    Unit_Price,
    Quantity,
    (Unit_Price * Quantity) AS Revenue
FROM Dirty_Sales_Dataset;
```

-- Query 2 --

```
SELECT
    Order_ID,
    Sales,
    Cost,
    ((Sales - Cost) / Cost) * 100 AS Profit_Margin_Percent
FROM Dirty_Sales_Dataset;
```

-- Query 3 --

```
SELECT
    Order_ID,
    Unit_Price,
    Discount,
    (Unit_Price - (Unit_Price * Discount)) AS Final_Price
FROM Dirty_Sales_Dataset;
```

-- Query 4 --

```
SELECT
    Order_ID,
    Sales,
    Cost,
    (Sales - Cost) AS Profit,
    Shipping_Cost,
    (Sales - Cost - Shipping_Cost) AS Net_Profit
FROM Dirty_Sales_Dataset;
```

-- Query 5 --

```
SELECT Order_ID, (Unit_Price * Quantity) AS Revenue FROM Dirty_Sales_Dataset;
```

-- Query 6 --

```
SELECT
    Order_ID,
    Customer_Name,
    Unit_Price,
    Quantity,
    (Unit_Price * Quantity) AS Revenue
FROM Dirty_Sales_Dataset
LIMIT 20;
```

-- A2.2.3 CONCAT – combining string columns --

-- Query 1 --

```
SELECT
    Order_ID,
    Customer_Name,
    CONCAT(Order_ID, ' - ', Customer_Name) AS Order_Customer_Info
FROM Dirty_Sales_Dataset;
```

-- Query 2 --

```
SELECT
    City,
    State,
    Country,
    CONCAT(City, ', ', State, ', ', Country) AS Full_Location
FROM Dirty_Sales_Dataset;
```

-- Query 3 --

```
SELECT
    CONCAT(Category, ' > ', Sub_Category, ' > ', Product_Name) AS Product_Path
FROM Dirty_Sales_Dataset;
```

-- Query 4 --

```

SELECT
    Customer_Name,
    City,
    State,
    CONCAT(Customer_Name, ' from ', City, ', ', State) AS Customer_Info
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 5 --
SELECT
    Customer_Name || ' from ' || City || ', ' || State AS Customer_Info
FROM Dirty_Sales_Dataset
LIMIT 20;

-- A2.2.4 COALESCE – replacing NULLs with default values --

-- Query 1 --
SELECT
    Order_ID,
    Customer_Rating,
    COALESCE(Customer_Rating, 'Not Rated') AS Rating_Status
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT
    Order_ID,
    Sales,
    COALESCE(Sales, 0) AS Sales_Fixed
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT
    Order_ID,
    COALESCE(Customer_Name, Customer_ID, 'Unknown Customer') AS Customer_Info
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT Unit_Price - (Unit_Price * Discount) AS Final_Price FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Unit_Price - (Unit_Price * COALESCE(Discount, 0)) AS Final_Price
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT COUNT(*) FROM Dirty_Sales_Dataset WHERE Customer_Rating IS NULL;

-- Query 7 --
SELECT COUNT(*) FROM Dirty_Sales_Dataset WHERE Sales IS NULL;

-- Query 8 --
SELECT
    Order_ID,
    Customer_Name,
    Customer_Rating,
    COALESCE(Customer_Rating, 'Not Rated') AS Final_Rating

```

```
FROM Dirty_Sales_Dataset
LIMIT 30;
```

```
-- A2.3.1 SELECT DISTINCT on single column --
```

```
-- Query 1 --
```

```
SELECT DISTINCT Region FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT DISTINCT Category FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT DISTINCT Country FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --
```

```
SELECT DISTINCT Customer_Rating FROM Dirty_Sales_Dataset;
```

```
-- Query 5 --
```

```
SELECT DISTINCT City FROM Dirty_Sales_Dataset;
```

```
-- Query 6 --
```

```
SELECT COUNT(DISTINCT Region) FROM Dirty_Sales_Dataset;
```

```
-- Query 7 --
```

```
SELECT DISTINCT City FROM Dirty_Sales_Dataset;
```

```
-- A2.3.2 SELECT DISTINCT on multiple columns --
```

```
-- Query 1 --
```

```
SELECT DISTINCT Region, Category FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT DISTINCT City, State FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT DISTINCT Segment, Payment_Method FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --
```

```
SELECT DISTINCT Customer_ID, Customer_Name FROM Dirty_Sales_Dataset;
```

```
-- Query 5 --
```

```
SELECT DISTINCT Region, Category FROM Dirty_Sales_Dataset;
```

```
-- Query 6 --
```

```
SELECT DISTINCT Customer_ID, Customer_Name FROM Dirty_Sales_Dataset
ORDER BY Customer_ID;
```

```
-- A2.3.3 COUNT(DISTINCT col) for unique value counting --
```

```
-- Query 1 --
```

```
SELECT COUNT(DISTINCT Region) FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT COUNT(DISTINCT Customer_ID) FROM Dirty_Sales_Dataset;
```

```

-- Query 3 --
SELECT COUNT(DISTINCT Product_ID) FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT COUNT(*) AS Total_Rows, COUNT(DISTINCT Order_ID) AS Unique_Orders
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT COUNT(*) AS Total, COUNT(DISTINCT Customer_ID) AS Unique_Customers
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT
    COUNT(*) AS Total_Rows,
    COUNT(DISTINCT Order_ID) AS Unique_OrderIDs,
    COUNT(DISTINCT Customer_ID) AS Unique_Customers,
    COUNT(DISTINCT Product_ID) AS Unique_Products
FROM Dirty_Sales_Dataset;

-- A2.3.4 DISTINCT vs GROUP BY – performance difference --

-- Query 1 --
SELECT DISTINCT Region FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT Region FROM Dirty_Sales_Dataset GROUP BY Region;

-- Query 3 --
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 4 --
SELECT Category, COUNT(*) AS Total_Orders, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Category;

-- Query 5 --
SELECT DISTINCT Category FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Category, COUNT(*) AS Total_Orders, SUM(Sales) AS Total_Revenue
FROM Dirty_Sales_Dataset
GROUP BY Category;

-- Query 7 --
SELECT DISTINCT Payment_Method FROM Dirty_Sales_Dataset;

-- Query 8 --
SELECT Payment_Method, COUNT(*) AS Times_Used
FROM Dirty_Sales_Dataset
GROUP BY Payment_Method;

-- Filtering with WHERE – A2.4.1 Comparison operators – =, !=, >, <, >=, <= --

```

```

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset WHERE Region = 'North';

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset WHERE Region != 'North';

-- Query 3 --
SELECT * FROM Dirty_Sales_Dataset WHERE Region <> 'North';

-- Query 4 --
SELECT * FROM Dirty_Sales_Dataset WHERE Sales > 5000;

-- Query 5 --
SELECT * FROM Dirty_Sales_Dataset WHERE Sales < 0;

-- Query 6 --
SELECT * FROM Dirty_Sales_Dataset WHERE Quantity >= 10;

-- Query 7 --
SELECT * FROM Dirty_Sales_Dataset WHERE Quantity <= 0;

-- Query 8 --
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset WHERE Sales < 0;

-- Query 9 --
SELECT Order_ID, Quantity FROM Dirty_Sales_Dataset WHERE Quantity <= 0;

-- Query 10 --
SELECT Order_ID, Discount FROM Dirty_Sales_Dataset WHERE Discount < 0 OR Discount > 1;

-- Query 11 --
SELECT Order_ID, Quantity, Sales, Discount
FROM Dirty_Sales_Dataset
WHERE Quantity <= 0 OR Sales < 0 OR Discount < 0;

-- Filtering with WHERE – A2.4.2 AND, OR, NOT – combining conditions --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region = 'North' AND Category = 'Electronics';

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales < 0 AND Quantity < 0;

-- Query 3 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region = 'North' OR Region = 'South';

-- Query 4 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales < 0 OR Quantity < 0 OR Discount < 0;

-- Query 5 --

```

```

SELECT * FROM Dirty_Sales_Dataset
WHERE NOT Region = 'North';

-- Query 6 --
SELECT * FROM Dirty_Sales_Dataset
WHERE NOT Region IN ('North', 'South');

-- Query 7 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales < 0 AND Quantity < 0;

-- Query 8 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales < 0 OR Quantity < 0;

-- Query 9 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region = 'North' AND (Category = 'Electronics' OR Category = 'Furniture');

-- Query 10 --
SELECT Order_ID, Region, Category, Sales, Quantity, Discount
FROM Dirty_Sales_Dataset
WHERE (Sales < 0 OR Quantity <= 0 OR Discount < 0 OR Discount > 1)
AND Region = 'North';

-- Query 11 --
SELECT Order_ID, Region, Sales, Quantity, Discount
FROM Dirty_Sales_Dataset
WHERE Sales < 0 OR Quantity <= 0 OR Discount < 0 OR Discount > 1;

-- A2.4.3 IN – matching list of values --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region = 'North' OR Region = 'South' OR Region = 'East';

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region IN ('North', 'South', 'East');

-- Query 3 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region IN ('North', 'South');

-- Query 4 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Category IN ('Electronics', 'Furniture', 'Clothing');

-- Query 5 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Payment_Method IN ('Credit Card', 'UPI');

-- Query 6 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Order_Status IN ('Cancelled', 'Returned');

```

```

-- Query 7 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region NOT IN ('North', 'South');

-- Query 8 --
SELECT Order_ID, Region, Category, Sales
FROM Dirty_Sales_Dataset
WHERE Region IN ('North', 'South', 'East');

-- Query 9 --
SELECT Order_ID, Region, Category, Sales
FROM Dirty_Sales_Dataset
WHERE Region NOT IN ('North', 'South', 'East');

-- A2.4.4 BETWEEN – range filtering (inclusive on both ends) --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales >= 1000 AND Sales <= 5000;

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales BETWEEN 1000 AND 5000;

-- Query 3 --
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset
WHERE Sales BETWEEN 1000 AND 5000;

-- Query 4 --
SELECT Order_ID, Order_Date FROM Dirty_Sales_Dataset
WHERE Order_Date BETWEEN '2024-01-01' AND '2024-12-31';

-- Query 5 --
SELECT Order_ID, Quantity FROM Dirty_Sales_Dataset
WHERE Quantity BETWEEN 1 AND 10;

-- Query 6 --
SELECT Order_ID, Discount FROM Dirty_Sales_Dataset
WHERE Discount BETWEEN 0 AND 1;

-- Query 7 --
SELECT Order_ID, Discount FROM Dirty_Sales_Dataset
WHERE Discount NOT BETWEEN 0 AND 1;

-- A2.4.5 LIKE – pattern matching with % and _ --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Name LIKE 'Neha%';

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Name LIKE '%Singh';

-- Query 3 --
SELECT * FROM Dirty_Sales_Dataset WHERE City LIKE '%mumbai%';

```

```

-- Query 4 --
SELECT * FROM Dirty_Sales_Dataset WHERE Product_ID LIKE 'P0092_';

-- Query 5 --
SELECT * FROM Dirty_Sales_Dataset WHERE Order_ID LIKE 'ORD_____';

-- Query 6 --
SELECT DISTINCT City FROM Dirty_Sales_Dataset WHERE City LIKE '%umbai%';

-- Query 7 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Name LIKE 'Neha%';

-- Query 8 --
SELECT * FROM Dirty_Sales_Dataset WHERE Product_Name LIKE '%Phone%';

-- Query 9 --
SELECT Postal_Code FROM Dirty_Sales_Dataset WHERE Postal_Code LIKE '%[a-z]%;

-- Query 10 --
SELECT * FROM Dirty_Sales_Dataset WHERE City NOT LIKE '%mumbai%';

-- Query 11 --
SELECT DISTINCT City FROM Dirty_Sales_Dataset WHERE City LIKE '%mumbai%';

-- A2.4.6 IS NULL and IS NOT NULL – NULL filtering --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Rating = NULL;

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Rating IS NULL;

-- Query 3 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Rating IS NOT NULL;

-- Query 4 --
SELECT
    COUNT(*) AS Total_Rows,
    COUNT(*) - COUNT(Customer_Rating) AS Null_Ratings,
    COUNT(*) - COUNT(Sales) AS Null_Sales,
    COUNT(*) - COUNT(Discount) AS Null_Discount
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Customer_Rating IS NULL AND Region = 'North';

-- Query 6 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales IS NULL OR Quantity IS NULL OR Discount IS NULL;

-- Query 7 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Rating = '';

```

```

-- Query 8 --
SELECT * FROM Dirty_Sales_Dataset WHERE TRIM(Customer_Rating) = '';

-- Query 9 --
SELECT * FROM Dirty_Sales_Dataset WHERE Customer_Rating IS NULL OR
TRIM(Customer_Rating) = '';

-- Query 10 --
SELECT
    COUNT(*) AS Total_Rows,
    COUNT(*) - COUNT(Customer_Rating) AS Missing_Rating,
    COUNT(*) - COUNT(Sales) AS Missing_Sales,
    COUNT(*) - COUNT(Discount) AS Missing_Discount,
    COUNT(*) - COUNT(Profit) AS Missing_Profit
FROM Dirty_Sales_Dataset;

-- A2.4.7 NOT IN with NULLs – the hidden trap --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Region NOT IN ('North', 'South', NULL);

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Customer_ID NOT IN (
    SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE Order_Status = 'Cancelled'
);

-- Query 3 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Customer_ID NOT IN (
    SELECT Customer_ID FROM Dirty_Sales_Dataset
    WHERE Order_Status = 'Cancelled' AND Customer_ID IS NOT NULL
);

-- Query 4 --
SELECT * FROM Dirty_Sales_Dataset d1
WHERE NOT EXISTS (
    SELECT 1 FROM Dirty_Sales_Dataset d2
    WHERE d2.Customer_ID = d1.Customer_ID AND d2.Order_Status = 'Cancelled'
);

-- Query 5 --
SELECT COUNT(*) FROM Dirty_Sales_Dataset WHERE Customer_ID IS NULL;

-- Query 6 --
SELECT COUNT(*) - COUNT(Region) AS Null_Region,
    COUNT(*) - COUNT(Customer_ID) AS Null_CustomerID
FROM Dirty_Sales_Dataset;

-- B1.1.1 COUNT(*) – count all rows including NULLs --

-- Query 1 --
SELECT COUNT(*) FROM Dirty_Sales_Dataset;

```

```
-- Query 2 --  
SELECT COUNT(*) FROM Dirty_Sales_Dataset WHERE Region = 'North';
```

```
-- Query 3 --  
SELECT COUNT(*) AS Total_Orders FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --  
SELECT COUNT(*) AS Total_Rows FROM Dirty_Sales_Dataset;
```

```
-- Query 5 --  
SELECT COUNT(*) AS Cancelled_Orders  
FROM Dirty_Sales_Dataset  
WHERE Order_Status = 'Cancelled';
```

-- B1.1.2 COUNT(column) – count non-NULL values only --

```
-- Query 1 --  
SELECT COUNT(Customer_Rating) FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --  
SELECT  
    COUNT(*) AS Total_Rows,  
    COUNT(Customer_Rating) AS Rows_With_Rating  
FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --  
SELECT  
    COUNT(*) AS Total_Rows,  
    COUNT(Customer_Rating) AS Non_Null_Ratings,  
    COUNT(*) - COUNT(Customer_Rating) AS Missing_Ratings  
FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --  
SELECT  
    COUNT(*) AS Total_Rows,  
    COUNT(*) - COUNT(Customer_Rating) AS Missing_Rating,  
    COUNT(*) - COUNT(Sales) AS Missing_Sales,  
    COUNT(*) - COUNT(Cost) AS Missing_Cost,  
    COUNT(*) - COUNT(Profit) AS Missing_Profit,  
    COUNT(*) - COUNT(Unit_Price) AS Missing_Unit_Price,  
    COUNT(*) - COUNT(Discount) AS Missing_Discount  
FROM Dirty_Sales_Dataset;
```

```
-- Query 5 --  
SELECT  
    COUNT(*) AS Total_Rows,  
    COUNT(Customer_Rating) AS Non_Null_Rating,  
    COUNT(*) - COUNT(Customer_Rating) AS Missing_Rating,  
    COUNT(Sales) AS Non_Null_Sales,  
    COUNT(*) - COUNT(Sales) AS Missing_Sales  
FROM Dirty_Sales_Dataset;
```

-- B1.1.3 COUNT(DISTINCT column) – count unique values --

```
-- Query 1 --
```

```
SELECT COUNT(DISTINCT Region) FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT
    COUNT(*) AS Total_Rows,
    COUNT(Customer_ID) AS Non_Null_CustomerIDs,
    COUNT(DISTINCT Customer_ID) AS Unique_Customers
FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT
    COUNT(*) AS Total_Rows,
    COUNT(DISTINCT Order_ID) AS Unique_Orders,
    COUNT(DISTINCT Customer_ID) AS Unique_Customers,
    COUNT(DISTINCT Product_ID) AS Unique_Products,
    COUNT(DISTINCT Region) AS Unique_Regions,
    COUNT(DISTINCT Category) AS Unique_Categories
FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --
```

```
SELECT
    COUNT(*) AS Total_Rows,
    COUNT(DISTINCT Order_ID) AS Unique_Orders,
    COUNT(DISTINCT Customer_ID) AS Unique_Customers,
    COUNT(DISTINCT Product_ID) AS Unique_Products
FROM Dirty_Sales_Dataset;
```

```
-- B1.1.4 Difference between COUNT(*) and COUNT(1) --
```

```
-- Query 1 --
```

```
SELECT COUNT(*) FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT COUNT(1) FROM Dirty_Sales_Dataset;
```

```
-- B1.2.1 SUM() – total of a numeric column --
```

```
-- Query 1 --
```

```
SELECT SUM(Sales) FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
```

```
SELECT SUM(Profit) FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --
```

```
SELECT SUM(Sales) AS North_Sales
FROM Dirty_Sales_Dataset
WHERE Region = 'North';
```

```
-- Query 5 --
```

```
SELECT
    SUM(Sales) AS Total_Sales,
    SUM(Cost) AS Total_Cost,
```

```

    SUM(Profit) AS Total_Profit
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT SUM(Sales) FROM Dirty_Sales_Dataset;

-- Query 7 --
SELECT SUM(Sales) AS Total_With_Errors FROM Dirty_Sales_Dataset;

-- Query 8 --
SELECT SUM(Sales) AS Total_Valid_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0;

-- Query 9 --
SELECT
    SUM(Sales) AS Total_Sales,
    SUM(Profit) AS Total_Profit,
    SUM(Cost) AS Total_Cost
FROM Dirty_Sales_Dataset;

-- Query 10 --
SELECT
    SUM(Sales) AS Total_With_Negatives,
    SUM(CASE WHEN Sales > 0 THEN Sales ELSE 0 END) AS Total_Only_Positive
FROM Dirty_Sales_Dataset;

-- B1.2.2 AVG() – average (ignores NULLs automatically) --

-- Query 1 --
SELECT AVG(Sales) FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT AVG(Discount) FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT AVG(Sales) AS Average_Sales FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT AVG(Sales) AS Avg_North_Sales
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

-- Query 5 --
SELECT
    AVG(Sales) AS Avg_Sales,
    AVG(Profit) AS Avg_Profit,
    AVG(Quantity) AS Avg_Quantity
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT AVG(Sales) FROM Dirty_Sales_Dataset;

-- Query 7 --
SELECT AVG(COALESCE(Sales, 0)) FROM Dirty_Sales_Dataset;

```

```

-- Query 8 --
SELECT
    AVG(Sales) AS Avg_Sales,
    AVG(Profit) AS Avg_Profit,
    AVG(Discount) AS Avg_Discount,
    COUNT(*) AS Total_Rows,
    COUNT(Sales) AS Non_Null_Sales_Count
FROM Dirty_Sales_Dataset;

-- B1.2.3 MIN() and MAX() – on numbers, dates, and strings --

-- Query 1 --
SELECT MIN(Sales) AS Lowest_Sales, MAX(Sales) AS Highest_Sales
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT MIN(Quantity) AS Min_Qty, MAX(Quantity) AS Max_Qty
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT MIN(Order_Date) AS Earliest_Order, MAX(Order_Date) AS Latest_Order
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT MIN(Customer_Name) AS First_Alphabetically, MAX(Customer_Name) AS
Last_Alphabetically
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT MIN(City) AS First_City, MAX(City) AS Last_City
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT
    MIN(Sales) AS Min_Sales,
    MAX(Sales) AS Max_Sales,
    MIN(Discount) AS Min_Discount,
    MAX(Discount) AS Max_Discount,
    MIN(Quantity) AS Min_Quantity,
    MAX(Quantity) AS Max_Quantity
FROM Dirty_Sales_Dataset;

-- Query 7 --
SELECT
    MIN(Order_Date) AS Earliest_Order,
    MAX(Order_Date) AS Latest_Order,
    MIN(Sales) AS Min_Sales,
    MAX(Sales) AS Max_Sales,
    MIN(Discount) AS Min_Discount,
    MAX(Discount) AS Max_Discount
FROM Dirty_Sales_Dataset;

-- B1.2.4 SUM with CASE WHEN for conditional sum (very powerful pattern) --

```

```

-- Query 1 --
SELECT
    SUM(CASE WHEN Region = 'North' THEN Sales ELSE 0 END) AS North_Sales,
    SUM(CASE WHEN Region = 'South' THEN Sales ELSE 0 END) AS South_Sales,
    SUM(CASE WHEN Region = 'East' THEN Sales ELSE 0 END) AS East_Sales,
    SUM(CASE WHEN Region = 'West' THEN Sales ELSE 0 END) AS West_Sales
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT SUM(CASE WHEN Sales > 0 THEN Sales ELSE 0 END) AS Valid_Sales_Total
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT
    SUM(CASE WHEN Order_Status = 'Cancelled' THEN 1 ELSE 0 END) AS Cancelled_Count,
    SUM(CASE WHEN Order_Status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Count
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT
    SUM(CASE WHEN Profit > 0 THEN Profit ELSE 0 END) AS Total_Profit_Orders,
    SUM(CASE WHEN Profit < 0 THEN Profit ELSE 0 END) AS Total_Loss_Orders
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT
    SUM(CASE WHEN Region = 'North' THEN Sales ELSE 0 END) AS North_Sales,
    SUM(CASE WHEN Region = 'South' THEN Sales ELSE 0 END) AS South_Sales,
    SUM(CASE WHEN Region = 'East' THEN Sales ELSE 0 END) AS East_Sales,
    SUM(CASE WHEN Region = 'West' THEN Sales ELSE 0 END) AS West_Sales,
    SUM(Sales) AS Grand_Total
FROM Dirty_Sales_Dataset;

-- B1.2.5 AVG vs SUM/COUNT – why they can give different results with NULLs --

-- Query 1 --
SELECT SUM(Sales) FROM table;

-- Query 2 --
SELECT COUNT(*) FROM table;

-- Query 3 --
SELECT COUNT(Sales) FROM table;

-- Query 4 --
SELECT AVG(Sales) FROM table;

-- Query 5 --
SELECT
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Total_Rows,
    COUNT(Sales) AS Non_Null_Sales_Rows,
    AVG(Sales) AS Actual_Average,
    SUM(Sales) / COUNT(*) AS Wrong_Average_Using_CountStar,
    SUM(Sales) / COUNT(Sales) AS Correct_Manual_Average

```

```
FROM Dirty_Sales_Dataset;
```

```
-- Query 6 --
```

```
SELECT SUM(Sales) / COUNT(*) AS Avg_Sales FROM Dirty_Sales_Dataset;
```

```
-- Query 7 --
```

```
SELECT AVG(Sales) AS Avg_Sales FROM Dirty_Sales_Dataset;
```

```
-- Query 8 --
```

```
SELECT
    SUM(Discount) AS Total_Discount,
    COUNT(*) AS Total_Rows,
    COUNT(Discount) AS Non_Null_Discount_Rows,
    AVG(Discount) AS Correct_AVG,
    SUM(Discount) / COUNT(*) AS Manual_Wrong_Method,
    SUM(Discount) / COUNT(Discount) AS Manual_Correct_Method
FROM Dirty_Sales_Dataset;
```

```
-- Basic GROUP BY – B2.1.1 What GROUP BY actually does behind the scenes --
```

```
-- Query 1 --
```

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Region;
```

```
-- Query 2 --
```

```
SELECT Region, SUM(Sales) AS Total_Sales, COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 3 --
```

```
SELECT Region, SUM(Sales) FROM Dirty_Sales_Dataset GROUP BY Region;
```

```
-- Query 4 --
```

```
SELECT Region, Order_ID, SUM(Sales) FROM Dirty_Sales_Dataset GROUP BY Region;
```

```
-- Query 5 --
```

```
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category;
```

```
-- Query 6 --
```

```
SELECT Region, COUNT(*) AS Total_Orders, SUM(Sales) AS Total_Sales, AVG(Sales) AS
Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- B2.1.2 GROUP BY with COUNT – frequency table --
```

```
-- Query 1 --
```

```
SELECT Region, COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 2 --
```

```
SELECT Category, COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Category;
```

-- Query 3 --

```
SELECT Payment_Method, COUNT(*) AS Times_Used
FROM Dirty_Sales_Dataset
GROUP BY Payment_Method;
```

-- Query 4 --

```
SELECT Order_Status, COUNT(*) AS Status_Count
FROM Dirty_Sales_Dataset
GROUP BY Order_Status;
```

-- Query 5 --

```
SELECT City, COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY City
ORDER BY Order_Count DESC;
```

-- Query 6 --

```
SELECT City, COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY City
ORDER BY Frequency DESC;
```

-- B2.1.3 GROUP BY with SUM, AVG --

-- Query 1 --

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 2 --

```
SELECT Region, AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 3 --

```
SELECT
    Region,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales_Per_Order
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 4 --

```
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    SUM(Profit) AS Total_Profit,
    AVG(Profit) AS Avg_Profit_Per_Order
FROM Dirty_Sales_Dataset
GROUP BY Category;
```

-- Query 5 --

```
SELECT
    Salesperson,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales_Generated,
    AVG(Sales) AS Avg_Deal_Size
FROM Dirty_Sales_Dataset
GROUP BY Salesperson
ORDER BY Total_Sales_Generated DESC;
```

-- Query 6 --

```
SELECT
    Region,
    Category,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category
ORDER BY Region, Total_Sales DESC;
```

-- Query 7 --

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;
```

-- Query 8 --

```
SELECT
    Category,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales,
    SUM(Profit) AS Total_Profit
FROM Dirty_Sales_Dataset
GROUP BY Category
ORDER BY Total_Sales DESC;
```

-- B2.1.4 GROUP BY multiple columns --

-- Query 1 --

```
SELECT Region, SUM(Sales) FROM table GROUP BY Region;
```

-- Query 2 --

```
SELECT Region, Category, SUM(Sales) FROM table GROUP BY Region, Category;
```

-- Query 3 --

```
SELECT
    Region,
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Region, Category
ORDER BY Region, Total_Sales DESC;
```

-- Query 4 --

```
SELECT
    Segment,
    Payment_Method,
    COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Segment, Payment_Method
ORDER BY Segment, Frequency DESC;
```

-- Query 5 --

```
SELECT
    Region,
    Category,
    Ship_Mode,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category, Ship_Mode
ORDER BY Region, Category, Total_Sales DESC;
```

-- Query 6 --

```
SELECT
    strftime('%Y', Order_Date) AS Order_Year,
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Order_Year, Region
ORDER BY Order_Year, Total_Sales DESC;
```

-- Query 7 --

```
SELECT Region, Category, Order_ID, SUM(Sales) FROM table GROUP BY Region, Category;
```

-- Query 8 --

```
SELECT
    Region,
    Category,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category
ORDER BY Region, Total_Sales DESC;
```

-- B2.2.1 HAVING vs WHERE – the key difference --

-- Query 1 --

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;
```

-- Query 2 --

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
HAVING SUM(Sales) > 500000;
```

```

-- Query 3 --
SELECT Region, SUM(Sales)
FROM Dirty_Sales_Dataset
WHERE SUM(Sales) > 500000
GROUP BY Region;

-- Query 4 --
SELECT Region, SUM(Sales)
FROM Dirty_Sales_Dataset
GROUP BY Region
HAVING SUM(Sales) > 500000;

-- Query 5 --
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region
HAVING SUM(Sales) > 500000;

-- Query 6 --
SELECT Category, COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Category
HAVING COUNT(*) > 5000;

-- Query 7 --
SELECT City, AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY City
HAVING AVG(Sales) > 1000;

-- Query 8 --
SELECT Salesperson, SUM(Sales) AS Total_Sales, COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
WHERE Order_Status != 'Cancelled'
GROUP BY Salesperson
HAVING SUM(Sales) > 100000
ORDER BY Total_Sales DESC;

-- Query 9 --
SELECT Region, COUNT(*) AS Order_Count, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region
HAVING COUNT(*) > 1000
ORDER BY Total_Sales DESC;

-- B2.2.2 HAVING with COUNT – filter groups --

-- Query 1 --
SELECT City, COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY City
HAVING COUNT(*) > 1000;

```

-- Query 2 --

```
SELECT Customer_ID, COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
HAVING COUNT(*) > 5
ORDER BY Total_Orders DESC;
```

-- Query 3 --

```
SELECT Product_Name, COUNT(*) AS Times_Sold
FROM Dirty_Sales_Dataset
GROUP BY Product_Name
HAVING COUNT(*) < 10
ORDER BY Times_Sold ASC;
```

-- Query 4 --

```
SELECT Order_ID, COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Order_ID
HAVING COUNT(*) > 1;
```

-- Query 5 --

```
SELECT Order_ID, COUNT(*) AS Duplicate_Count
FROM Dirty_Sales_Dataset
GROUP BY Order_ID
HAVING COUNT(*) > 1
ORDER BY Duplicate_Count DESC;
```

-- Query 6 --

```
SELECT Customer_ID, COUNT(DISTINCT Product_ID) AS Unique_Products_Bought
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
HAVING COUNT(DISTINCT Product_ID) > 10;
```

-- Query 7 --

```
SELECT Order_ID, COUNT(*) AS Duplicate_Count
FROM Dirty_Sales_Dataset
GROUP BY Order_ID
HAVING COUNT(*) > 1
ORDER BY Duplicate_Count DESC;
```

-- B2.2.3 HAVING with AVG, SUM --

-- Query 1 --

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
HAVING SUM(Sales) > 1000000;
```

-- Query 2 --

```
SELECT Category, SUM(Profit) AS Total_Profit
FROM Dirty_Sales_Dataset
GROUP BY Category
HAVING SUM(Profit) < 0;
```

```

-- Query 3 --
SELECT Salesperson, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Salesperson
HAVING SUM(Sales) > 500000
ORDER BY Total_Sales DESC;

-- Query 4 --
SELECT City, AVG(Sales) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
GROUP BY City
HAVING AVG(Sales) > 2000;

-- Query 5 --
SELECT Product_Name, AVG(Discount) AS Avg_Discount
FROM Dirty_Sales_Dataset
GROUP BY Product_Name
HAVING AVG(Discount) > 0.20;

-- Query 6 --
SELECT Category, AVG(Profit_Margin) AS Avg_Margin
FROM Dirty_Sales_Dataset
GROUP BY Category
HAVING AVG(Profit_Margin) < 10;

-- Query 7 --
SELECT Region, SUM(Sales) AS Total_Sales, AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
HAVING SUM(Sales) > 500000 AND AVG(Sales) > 1500;

-- Query 8 --
SELECT
    Category,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales,
    SUM(Profit) AS Total_Profit
FROM Dirty_Sales_Dataset
WHERE Order_Status != 'Cancelled'
GROUP BY Category
HAVING SUM(Sales) > 200000
ORDER BY Total_Sales DESC;

-- Query 9 --
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    SUM(Profit) AS Total_Profit,
    AVG(Profit_Margin) AS Avg_Margin
FROM Dirty_Sales_Dataset
GROUP BY Category
HAVING SUM(Profit) < 0
ORDER BY Total_Profit ASC;

```

-- B2.2.4 Using both WHERE and HAVING together --

-- Query 1 --

```
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region
HAVING SUM(Sales) > 500000;
```

-- Query 2 --

```
SELECT Category, SUM(Sales) AS Total_Sales, COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
WHERE Order_Status != 'Cancelled'
GROUP BY Category
HAVING COUNT(*) > 1000
ORDER BY Total_Sales DESC;
```

-- Query 3 --

```
SELECT Customer_ID, SUM(Sales) AS Total_Spent
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Customer_ID
HAVING SUM(Sales) > 50000
ORDER BY Total_Spent DESC;
```

-- Query 4 --

```
SELECT Region, SUM(Profit) AS Total_Profit
FROM Dirty_Sales_Dataset
WHERE Order_Date >= '2024-01-01' AND Order_Date <= '2024-12-31'
GROUP BY Region
HAVING SUM(Profit) < 0;
```

-- Query 5 --

```
SELECT Region, SUM(Sales)
FROM Dirty_Sales_Dataset
WHERE SUM(Sales) > 500000
GROUP BY Region;
```

-- Query 6 --

```
SELECT Region, SUM(Sales)
FROM Dirty_Sales_Dataset
GROUP BY Region
HAVING Sales > 0;
```

-- Query 7 --

```
SELECT Region, SUM(Sales)
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region
HAVING SUM(Sales) > 500000;
```

-- Query 8 --

```
SELECT
    Category,
    COUNT(*) AS Total_Orders,
```

```

        SUM(Sales) AS Total_Sales,
        AVG(Sales) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0 AND Order_Status != 'Cancelled'
GROUP BY Category
HAVING SUM(Sales) > 100000
ORDER BY Total_Sales DESC;

-- B2.3.1 GROUPING SETS – multiple GROUP BY in one query --

-- Query 1 --
SELECT Region, SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset GROUP BY Region;

-- Query 2 --
SELECT Category, SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset GROUP BY
Category;

-- Query 3 --
SELECT SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY GROUPING SETS (
    (Region),
    (Category),
    ()
);

-- Query 5 --
SELECT Region, NULL AS Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region

UNION ALL

SELECT NULL AS Region, Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Category

UNION ALL

SELECT NULL AS Region, NULL AS Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region

UNION ALL

SELECT 'GRAND TOTAL' AS Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset;

```

-- B2.3.2 ROLLUP – subtotals and grand total automatically --

-- Query 1 --

```
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM table
GROUP BY ROLLUP (Region, Category);
```

-- Query 2 --

```
SELECT Region, Category, SUM(Sales) FROM Dirty_Sales_Dataset GROUP BY ROLLUP(Region,
Category);
```

-- Query 3 --

```
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category
```

UNION ALL

```
SELECT Region, 'ALL CATEGORIES' AS Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
```

UNION ALL

```
SELECT 'ALL REGIONS' AS Region, 'ALL CATEGORIES' AS Category, SUM(Sales) AS
Total_Sales
FROM Dirty_Sales_Dataset;
```

-- Query 4 --

```
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category
```

UNION ALL

```
SELECT Region, 'ALL CATEGORIES', SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
```

UNION ALL

```
SELECT 'ALL REGIONS', 'ALL CATEGORIES', SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
```

```
ORDER BY Region, Category;
```

-- B2.3.3 CUBE – all possible combinations of grouping --

-- Query 1 --

```
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM table
GROUP BY CUBE (Region, Category);
```

-- Query 2 --

```
SELECT Region, Category, SUM(Sales) FROM Dirty_Sales_Dataset GROUP BY CUBE(Region, Category);
```

```
-- Query 3 --
```

```
SELECT Region, Category, SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Region, Category
```

```
UNION ALL
```

```
SELECT Region, 'ALL CATEGORIES', SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Region
```

```
UNION ALL
```

```
SELECT 'ALL REGIONS', Category, SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Category
```

```
UNION ALL
```

```
SELECT 'ALL REGIONS', 'ALL CATEGORIES', SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset;
```

```
-- Query 4 --
```

```
SELECT Region, Category, SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Region, Category
```

```
UNION ALL
```

```
SELECT Region, 'ALL CATEGORIES', SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Region
```

```
UNION ALL
```

```
SELECT 'ALL REGIONS', Category, SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Category
```

```
UNION ALL
```

```
SELECT 'ALL REGIONS', 'ALL CATEGORIES', SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset
```

```
ORDER BY Region, Category;
```

```
-- B2.3.4 GROUPING() function – identify NULL from ROLLUP vs real NULL --
```

```
-- Query 1 --
```

```
SELECT  
    Region,  
    Category,
```

```

        GROUPING(Category) AS Is_Category_Subtotal,
        SUM(Sales) AS Total_Sales
FROM table
GROUP BY ROLLUP(Region, Category);

-- Query 2 --
SELECT Region, Category, SUM(Sales) AS Total_Sales, 0 AS Is_Subtotal
FROM Dirty_Sales_Dataset
GROUP BY Region, Category

UNION ALL

SELECT Region, 'ALL CATEGORIES', SUM(Sales) AS Total_Sales, 1 AS Is_Subtotal
FROM Dirty_Sales_Dataset
GROUP BY Region

UNION ALL

SELECT 'ALL REGIONS', 'ALL CATEGORIES', SUM(Sales) AS Total_Sales, 1 AS Is_Subtotal
FROM Dirty_Sales_Dataset

ORDER BY Region, Category;

-- Query 3 --
SELECT Region, Category, SUM(Sales) AS Total_Sales, 'Detail' AS Row_Type
FROM Dirty_Sales_Dataset
GROUP BY Region, Category

UNION ALL

SELECT Region, 'ALL CATEGORIES', SUM(Sales) AS Total_Sales, 'Subtotal' AS Row_Type
FROM Dirty_Sales_Dataset
GROUP BY Region

UNION ALL

SELECT 'ALL REGIONS', 'ALL CATEGORIES', SUM(Sales) AS Total_Sales, 'Grand Total' AS
Row_Type
FROM Dirty_Sales_Dataset

ORDER BY Region, Category;

-- B3.1.1 Simple CASE syntax --

-- Query 1 --
SELECT
    Order_ID,
    Sales,
    CASE
        WHEN Sales > 0 THEN 'Valid'
        ELSE 'Invalid'
    END AS Sales_Status
FROM Dirty_Sales_Dataset;

-- Query 2 --

```

```

SELECT
    Order_ID,
    Quantity,
    CASE
        WHEN Quantity > 10 THEN 'Bulk Order'
        WHEN Quantity > 5 THEN 'Medium Order'
        WHEN Quantity > 0 THEN 'Small Order'
        ELSE 'Invalid Quantity'
    END AS Order_Size
FROM Dirty_Sales_Dataset;

```

-- Query 3 --

```

SELECT
    Order_ID,
    Profit,
    CASE
        WHEN Profit > 0 THEN 'Profit'
        WHEN Profit < 0 THEN 'Loss'
        ELSE 'Break Even'
    END AS Profit_Status
FROM Dirty_Sales_Dataset;

```

-- Query 4 --

```

SELECT
    Order_ID,
    Sales,
    CASE
        WHEN Sales > 0 THEN 'Valid Sale'
        WHEN Sales = 0 THEN 'Zero Sale'
        ELSE 'Negative/Invalid'
    END AS Sales_Category
FROM Dirty_Sales_Dataset
LIMIT 30;

```

-- B3.1.2 Searched CASE syntax (with conditions) --

-- Query 1 --

```

SELECT
    Order_ID,
    Region,
    CASE Region
        WHEN 'North' THEN 'N'
        WHEN 'South' THEN 'S'
        WHEN 'East' THEN 'E'
        WHEN 'West' THEN 'W'
        ELSE 'Unknown'
    END AS Region_Code
FROM Dirty_Sales_Dataset;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Sales,
    CASE
        WHEN Sales > 10000 THEN 'High'

```

```

        WHEN Sales BETWEEN 1000 AND 10000 THEN 'Medium'
        WHEN Sales > 0 THEN 'Low'
        ELSE 'Invalid'
    END AS Sales_Tier
FROM Dirty_Sales_Dataset;

```

-- Query 3 --

```

SELECT
    Order_ID,
    Category,
    CASE Category
        WHEN 'Electronics' THEN 'ELEC'
        WHEN 'Furniture' THEN 'FURN'
        WHEN 'Clothing' THEN 'CLTH'
        ELSE 'OTHER'
    END AS Category_Code
FROM Dirty_Sales_Dataset;

```

-- Query 4 --

```

SELECT
    Order_ID,
    Sales,
    Discount,
    CASE
        WHEN Sales > 5000 AND Discount > 0.20 THEN 'High Value, High Discount'
        WHEN Sales > 5000 AND Discount <= 0.20 THEN 'High Value, Low Discount'
        WHEN Sales <= 5000 THEN 'Low Value Order'
        ELSE 'Check Data'
    END AS Order_Classification
FROM Dirty_Sales_Dataset;

```

-- Query 5 --

```

SELECT
    Order_ID,
    Sales,
    Discount,
    CASE
        WHEN Sales > 5000 AND Discount > 0.20 THEN 'High Value, High Discount'
        WHEN Sales > 5000 THEN 'High Value, Normal Discount'
        WHEN Sales > 0 THEN 'Normal Order'
        ELSE 'Invalid Sales'
    END AS Order_Classification
FROM Dirty_Sales_Dataset
LIMIT 30;

```

-- B3.1.3 CASE inside SELECT for custom categories --

-- Query 1 --

```

SELECT
    Order_ID,
    Sales,
    CASE
        WHEN Sales >= 10000 THEN 'Premium'
        WHEN Sales >= 5000 THEN 'Gold'
        WHEN Sales >= 1000 THEN 'Silver'

```

```

        WHEN Sales > 0 THEN 'Bronze'
        ELSE 'Invalid'
    END AS Order_Tier
FROM Dirty_Sales_Dataset;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Discount,
    CASE
        WHEN Discount = 0 THEN 'No Discount'
        WHEN Discount BETWEEN 0.01 AND 0.10 THEN 'Low Discount'
        WHEN Discount BETWEEN 0.11 AND 0.30 THEN 'Medium Discount'
        WHEN Discount > 0.30 THEN 'High Discount'
        ELSE 'Invalid Discount'
    END AS Discount_Level
FROM Dirty_Sales_Dataset;

```

-- Query 3 --

```

SELECT
    Order_ID,
    Profit_Margin,
    CASE
        WHEN Profit_Margin > 30 THEN 'Excellent'
        WHEN Profit_Margin BETWEEN 10 AND 30 THEN 'Good'
        WHEN Profit_Margin BETWEEN 0 AND 10 THEN 'Low Margin'
        WHEN Profit_Margin < 0 THEN 'Loss Making'
        ELSE 'Unknown'
    END AS Margin_Health
FROM Dirty_Sales_Dataset;

```

-- Query 4 --

```

SELECT
    Order_ID,
    Order_Date,
    CASE
        WHEN strftime('%m', Order_Date) IN ('01', '02', '03') THEN 'Q1'
        WHEN strftime('%m', Order_Date) IN ('04', '05', '06') THEN 'Q2'
        WHEN strftime('%m', Order_Date) IN ('07', '08', '09') THEN 'Q3'
        WHEN strftime('%m', Order_Date) IN ('10', '11', '12') THEN 'Q4'
        ELSE 'Unknown'
    END AS Quarter
FROM Dirty_Sales_Dataset;

```

-- Query 5 --

```

SELECT
    CASE
        WHEN Sales >= 10000 THEN 'Premium'
        WHEN Sales >= 5000 THEN 'Gold'
        WHEN Sales >= 1000 THEN 'Silver'
        WHEN Sales > 0 THEN 'Bronze'
        ELSE 'Invalid'
    END AS Order_Tier,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales

```

```

FROM Dirty_Sales_Dataset
GROUP BY
    CASE
        WHEN Sales >= 10000 THEN 'Premium'
        WHEN Sales >= 5000 THEN 'Gold'
        WHEN Sales >= 1000 THEN 'Silver'
        WHEN Sales > 0 THEN 'Bronze'
        ELSE 'Invalid'
    END
ORDER BY Total_Sales DESC;

```

-- Query 6 --

```

SELECT
    CASE
        WHEN Sales >= 10000 THEN 'Premium'
        WHEN Sales >= 5000 THEN 'Gold'
        WHEN Sales >= 1000 THEN 'Silver'
        WHEN Sales > 0 THEN 'Bronze'
        ELSE 'Invalid'
    END AS Order_Tier,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY
    CASE
        WHEN Sales >= 10000 THEN 'Premium'
        WHEN Sales >= 5000 THEN 'Gold'
        WHEN Sales >= 1000 THEN 'Silver'
        WHEN Sales > 0 THEN 'Bronze'
        ELSE 'Invalid'
    END
ORDER BY Total_Sales DESC;

```

-- B3.1.4 ELSE clause – default value --

-- Query 1 --

```

SELECT
    Order_ID,
    Region,
    CASE
        WHEN Region = 'North' THEN 'N'
        WHEN Region = 'South' THEN 'S'
    END AS Region_Code
FROM Dirty_Sales_Dataset;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Region,
    CASE
        WHEN Region = 'North' THEN 'N'
        WHEN Region = 'South' THEN 'S'
        ELSE 'Other'
    END AS Region_Code

```

```
FROM Dirty_Sales_Dataset;
```

```
-- Query 3 --
```

```
SELECT
    Order_ID,
    Region,
    CASE
        WHEN Region = 'North' THEN 'North'
        WHEN Region = 'South' THEN 'South'
        WHEN Region = 'East' THEN 'East'
        WHEN Region = 'West' THEN 'West'
        ELSE 'UNEXPECTED VALUE'
    END AS Region_Check
FROM Dirty_Sales_Dataset
WHERE Region NOT IN ('North', 'South', 'East', 'West') OR Region IS NULL;
```

```
-- Query 4 --
```

```
SELECT
    Order_ID,
    Customer_Rating,
    CASE
        WHEN Customer_Rating IS NULL THEN 'Not Rated'
        WHEN Customer_Rating = '5' THEN 'Excellent'
        WHEN Customer_Rating = '4' THEN 'Good'
        WHEN Customer_Rating = '3' THEN 'Average'
        ELSE 'Poor/Invalid'
    END AS Rating_Label
FROM Dirty_Sales_Dataset;
```

```
-- Query 5 --
```

```
SELECT
    Order_ID,
    Order_Status,
    CASE
        WHEN Order_Status = 'Delivered' THEN 'Delivered'
        WHEN Order_Status = 'Cancelled' THEN 'Cancelled'
        WHEN Order_Status = 'Pending' THEN 'Pending'
        WHEN Order_Status = 'Returned' THEN 'Returned'
        ELSE 'UNEXPECTED: ' || Order_Status
    END AS Status_Check
FROM Dirty_Sales_Dataset
GROUP BY Status_Check;
```

```
-- B3.2.1 CASE for creating bins/buckets --
```

```
-- Query 1 --
```

```
SELECT
    Order_ID,
    Sales,
    CASE
        WHEN Sales < 0 THEN '0. Invalid (Negative)'
        WHEN Sales BETWEEN 0 AND 999 THEN '1. 0-999'
        WHEN Sales BETWEEN 1000 AND 4999 THEN '2. 1000-4999'
        WHEN Sales BETWEEN 5000 AND 9999 THEN '3. 5000-9999'
        WHEN Sales >= 10000 THEN '4. 10000+'
    END AS Sales_Bin
```

```

        ELSE '5. Unknown'
    END AS Sales_Bin
FROM Dirty_Sales_Dataset;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Quantity,
    CASE
        WHEN Quantity <= 0 THEN 'Invalid'
        WHEN Quantity BETWEEN 1 AND 5 THEN 'Small (1-5)'
        WHEN Quantity BETWEEN 6 AND 15 THEN 'Medium (6-15)'
        WHEN Quantity > 15 THEN 'Large (15+)'
        ELSE 'Unknown'
    END AS Quantity_Bin
FROM Dirty_Sales_Dataset;

```

-- Query 3 --

```

SELECT
    Order_ID,
    Shipping_Cost,
    CASE
        WHEN Shipping_Cost < 50 THEN 'Low Shipping'
        WHEN Shipping_Cost BETWEEN 50 AND 200 THEN 'Medium Shipping'
        WHEN Shipping_Cost > 200 THEN 'High Shipping'
        ELSE 'Unknown'
    END AS Shipping_Bin
FROM Dirty_Sales_Dataset;

```

-- Query 4 --

```

SELECT
    CASE
        WHEN Sales < 0 THEN '0. Invalid'
        WHEN Sales BETWEEN 0 AND 999 THEN '1. 0-999'
        WHEN Sales BETWEEN 1000 AND 4999 THEN '2. 1000-4999'
        WHEN Sales BETWEEN 5000 AND 9999 THEN '3. 5000-9999'
        WHEN Sales >= 10000 THEN '4. 10000+'
    END AS Sales_Bin,
    COUNT(*) AS Number_of_Orders
FROM Dirty_Sales_Dataset
GROUP BY Sales_Bin
ORDER BY Sales_Bin;

```

-- Query 5 --

```

SELECT
    CASE
        WHEN Sales < 0 THEN '0. Invalid (Negative)'
        WHEN Sales BETWEEN 0 AND 999 THEN '1. 0-999'
        WHEN Sales BETWEEN 1000 AND 4999 THEN '2. 1000-4999'
        WHEN Sales BETWEEN 5000 AND 9999 THEN '3. 5000-9999'
        WHEN Sales >= 10000 THEN '4. 10000+'
    END AS Sales_Bin,
    COUNT(*) AS Number_of_Orders,
    SUM(Sales) AS Total_Sales_In_Bin
FROM Dirty_Sales_Dataset

```

```
GROUP BY Sales_Bin
ORDER BY Sales_Bin;
```

```
-- B3.2.2 CASE inside SUM for conditional aggregation --
```

```
-- Query 1 --
```

```
SELECT
    SUM(CASE WHEN Order_Status = 'Delivered' THEN Sales ELSE 0 END) AS
Delivered_Sales,
    SUM(CASE WHEN Order_Status = 'Cancelled' THEN Sales ELSE 0 END) AS
Cancelled_Sales,
    SUM(CASE WHEN Order_Status = 'Returned' THEN Sales ELSE 0 END) AS Returned_Sales
FROM Dirty_Sales_Dataset;
```

```
-- B3.2.3 CASE inside COUNT for conditional counting --
```

```
-- Query 1 --
```

```
SELECT
    COUNT(CASE WHEN Order_Status = 'Cancelled' THEN 1 END) AS Cancelled_Count,
    COUNT(CASE WHEN Order_Status = 'Delivered' THEN 1 END) AS Delivered_Count
FROM Dirty_Sales_Dataset;
```

```
-- B3.2.4 CASE inside ORDER BY for custom sort logic --
```

```
-- Query 1 --
```

```
SELECT * FROM Dirty_Sales_Dataset ORDER BY Order_Status;
```

```
-- Query 2 --
```

```
SELECT Order_ID, Order_Status, Sales
FROM Dirty_Sales_Dataset
ORDER BY
    CASE
        WHEN Order_Status = 'Pending' THEN 1
        WHEN Order_Status = 'Delivered' THEN 2
        WHEN Order_Status = 'Returned' THEN 3
        WHEN Order_Status = 'Cancelled' THEN 4
        ELSE 5
    END;
```

```
-- Query 3 --
```

```
SELECT Order_ID, Region, Sales
FROM Dirty_Sales_Dataset
ORDER BY
    CASE
        WHEN Region = 'North' THEN 1
        WHEN Region = 'East' THEN 2
        WHEN Region = 'South' THEN 3
        WHEN Region = 'West' THEN 4
        ELSE 5
    END;
```

```
-- Query 4 --
```

```
SELECT Order_ID, Order_Status, Sales
FROM Dirty_Sales_Dataset
ORDER BY
```

```

CASE
    WHEN Order_Status = 'Pending' THEN 1
    WHEN Order_Status = 'Delivered' THEN 2
    ELSE 3
END,
Sales DESC;

-- Query 5 --
SELECT Order_ID, Customer_Rating
FROM Dirty_Sales_Dataset
ORDER BY
    CASE WHEN Customer_Rating IS NULL THEN 1 ELSE 0 END,
    Customer_Rating DESC;

-- Query 6 --
SELECT Order_ID, Order_Status, Sales
FROM Dirty_Sales_Dataset
ORDER BY
    CASE
        WHEN Order_Status = 'Pending' THEN 1
        WHEN Order_Status = 'Delivered' THEN 2
        WHEN Order_Status = 'Returned' THEN 3
        WHEN Order_Status = 'Cancelled' THEN 4
        ELSE 5
    END
LIMIT 30;

-- B3.2.5 Pivot table manually using CASE + GROUP BY --

-- Query 1 --
SELECT Region, Category, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Category;

-- Query 2 --
SELECT
    Region,
    SUM(CASE WHEN Category = 'Electronics' THEN Sales ELSE 0 END) AS Electronics,
    SUM(CASE WHEN Category = 'Furniture' THEN Sales ELSE 0 END) AS Furniture,
    SUM(CASE WHEN Category = 'Clothing' THEN Sales ELSE 0 END) AS Clothing
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 3 --
SELECT
    Region,
    SUM(CASE WHEN Payment_Method = 'Credit Card' THEN Sales ELSE 0 END) AS
Credit_Card_Sales,
    SUM(CASE WHEN Payment_Method = 'UPI' THEN Sales ELSE 0 END) AS UPI_Sales,
    SUM(CASE WHEN Payment_Method = 'Cash' THEN Sales ELSE 0 END) AS Cash_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 4 --
SELECT

```

```

    Category,
    COUNT(CASE WHEN Order_Status = 'Delivered' THEN 1 END) AS Delivered_Count,
    COUNT(CASE WHEN Order_Status = 'Cancelled' THEN 1 END) AS Cancelled_Count,
    COUNT(CASE WHEN Order_Status = 'Returned' THEN 1 END) AS Returned_Count
FROM Dirty_Sales_Dataset
GROUP BY Category;

```

-- Query 5 --

```

SELECT
    Region,
    SUM(CASE WHEN Category = 'Electronics' THEN Sales ELSE 0 END) AS Electronics,
    SUM(CASE WHEN Category = 'Furniture' THEN Sales ELSE 0 END) AS Furniture
FROM Dirty_Sales_Dataset
GROUP BY Region

```

UNION ALL

```

SELECT
    'GRAND TOTAL',
    SUM(CASE WHEN Category = 'Electronics' THEN Sales ELSE 0 END),
    SUM(CASE WHEN Category = 'Furniture' THEN Sales ELSE 0 END)
FROM Dirty_Sales_Dataset;

```

-- Query 6 --

```

SELECT
    Region,
    SUM(CASE WHEN Category = 'Electronics' THEN Sales ELSE 0 END) AS Electronics,
    SUM(CASE WHEN Category = 'Furniture' THEN Sales ELSE 0 END) AS Furniture,
    SUM(CASE WHEN Category = 'Clothing' THEN Sales ELSE 0 END) AS Clothing,
    SUM(Sales) AS Total_All_Categories
FROM Dirty_Sales_Dataset
GROUP BY Region
ORDER BY Total_All_Categories DESC;

```

-- B4.1.1 UPPER(), LOWER() – case conversion --

-- Query 1 --

```

SELECT City, UPPER(City) AS City_Upper
FROM Dirty_Sales_Dataset;

```

-- Query 2 --

```

SELECT City, LOWER(City) AS City_Lower
FROM Dirty_Sales_Dataset;

```

-- Query 3 --

```

SELECT DISTINCT City FROM Dirty_Sales_Dataset;

```

-- Query 4 --

```

SELECT DISTINCT UPPER(City) AS City_Standardized FROM Dirty_Sales_Dataset;

```

-- Query 5 --

```

SELECT UPPER(City) AS City_Standardized, COUNT(*) AS Order_Count, SUM(Sales) AS
Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY UPPER(City)

```

```
ORDER BY Total_Sales DESC;
```

```
-- Query 6 --
```

```
SELECT * FROM Dirty_Sales_Dataset  
WHERE UPPER(City) = 'MUMBAI';
```

```
-- Query 7 --
```

```
SELECT DISTINCT City FROM Dirty_Sales_Dataset ORDER BY City;
```

```
-- Query 8 --
```

```
SELECT DISTINCT UPPER(City) AS City_Fixed FROM Dirty_Sales_Dataset ORDER BY  
City_Fixed;
```

```
-- B4.1.2 TRIM(), LTRIM(), RTRIM() – removing spaces --
```

```
-- Query 1 --
```

```
SELECT TRIM(column_name) FROM table_name;
```

```
-- Query 2 --
```

```
SELECT LTRIM(column_name) FROM ta;
```

```
-- Query 3 --
```

```
SELECT RTRIM(column_name) FROM Dirty_Sales_Dataset dsd ;
```

```
-- Query 4 --
```

```
SELECT Customer_Rating, LENGTH(Customer_Rating) AS Original_Length,  
LENGTH(TRIM(Customer_Rating)) AS Trimmed_Length  
FROM Dirty_Sales_Dataset  
WHERE TRIM(Customer_Rating) = '' OR Customer_Rating IS NULL;
```

```
-- Query 5 --
```

```
SELECT  
    Order_ID,  
    Customer_Rating,  
    COALESCE(NULLIF(TRIM(Customer_Rating), ''), 'Not Rated') AS Final_Rating  
FROM Dirty_Sales_Dataset;
```

```
-- Query 6 --
```

```
SELECT City, TRIM(City) AS City_Cleaned  
FROM Dirty_Sales_Dataset;
```

```
-- Query 7 --
```

```
SELECT City, UPPER(TRIM(City)) AS City_Fully_Cleaned  
FROM Dirty_Sales_Dataset;
```

```
-- Query 8 --
```

```
SELECT * FROM Dirty_Sales_Dataset  
WHERE TRIM(Region) = 'North';
```

```
-- Query 9 --
```

```
SELECT  
    Order_ID,  
    Customer_Rating,  
    COALESCE(NULLIF(TRIM(Customer_Rating), ''), 'Not Rated') AS Final_Rating  
FROM Dirty_Sales_Dataset
```

```

LIMIT 30;

-- B4.1.3 LENGTH() / LEN() – string length --

-- Query 1 --
SELECT LENGTH(column_name) FROM table_name;

-- Query 2 --
SELECT Customer_Name, LENGTH(Customer_Name) AS Name_Length
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT Customer_Name, LENGTH(Customer_Name) AS Name_Length
FROM Dirty_Sales_Dataset
WHERE LENGTH(Customer_Name) < 3 OR LENGTH(Customer_Name) > 50;

-- Query 4 --
SELECT Order_ID, LENGTH(Order_ID) AS ID_Length
FROM Dirty_Sales_Dataset
WHERE LENGTH(Order_ID) != 9;

-- Query 5 --
SELECT Postal_Code, LENGTH(Postal_Code) AS Code_Length
FROM Dirty_Sales_Dataset
GROUP BY LENGTH(Postal_Code);

-- Query 6 --
SELECT
    Customer_Rating,
    LENGTH(Customer_Rating) AS Original_Length,
    LENGTH(TRIM(Customer_Rating)) AS Trimmed_Length
FROM Dirty_Sales_Dataset
WHERE LENGTH(Customer_Rating) != LENGTH(TRIM(Customer_Rating));

-- Query 7 --
SELECT
    Order_ID,
    LENGTH(Order_ID) AS ID_Length
FROM Dirty_Sales_Dataset
GROUP BY LENGTH(Order_ID);

-- B4.1.4 REPLACE() – find and replace in strings --

-- Query 1 --
SELECT City, REPLACE(City, 'MUMBAI', 'Mumbai') AS City_Fixed
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT Unit_Price, REPLACE(Unit_Price, '$', '') AS Price_Cleaned
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT Sales, REPLACE(Sales, ',', '') AS Sales_Cleaned
FROM Dirty_Sales_Dataset;

```

```

-- Query 4 --
SELECT Unit_Price, REPLACE(REPLACE(Unit_Price, '$', ''), ',', '') AS
Price_Fully_Cleaned
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Product_Name, REPLACE(Product_Name, ' ', '_') AS Product_Name_Formatted
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT
    Unit_Price AS Original,
    REPLACE(REPLACE(Unit_Price, '$', ''), ',', '') AS Cleaned_Text,
    CAST(REPLACE(REPLACE(Unit_Price, '$', ''), ',', '') AS REAL) AS Cleaned_Number
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 7 --
SELECT
    Unit_Price AS Original_Price,
    REPLACE(REPLACE(Unit_Price, '$', ''), ',', '') AS Cleaned_Price
FROM Dirty_Sales_Dataset
LIMIT 20;

-- B4.2.1 SUBSTRING() / SUBSTR() – extract part of string --

-- Query 1 --
SELECT Order_ID, SUBSTR(Order_ID, 4) AS Order_Number
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT Order_ID, SUBSTR(Order_ID, 1, 3) AS Prefix
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT Product_ID, SUBSTR(Product_ID, 2) AS Product_Number
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT Order_Date, SUBSTR(Order_Date, 1, 4) AS Order_Year
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Customer_ID, SUBSTR(Customer_ID, 5) AS Customer_Number
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Order_ID, SUBSTR(Order_ID, 4, LENGTH(Order_ID) - 3) AS Order_Number
FROM Dirty_Sales_Dataset;

-- Query 7 --
SELECT
    Order_ID,
    SUBSTR(Order_ID, 1, 3) AS Prefix,
    SUBSTR(Order_ID, 4) AS Number_Part

```

```

FROM Dirty_Sales_Dataset
LIMIT 20;

-- B4.2.2 LEFT(), RIGHT() – first/last N characters --

-- Query 1 --
SELECT LEFT(Order_ID, 3) FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT SUBSTR(Order_ID, 1, 3) AS First_3_Chars
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT SUBSTR(Order_ID, -6) AS Last_6_Chars
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT Order_ID, SUBSTR(Order_ID, 1, 3) AS Prefix
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Order_ID, SUBSTR(Order_ID, -6) AS Last_6_Digits
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Order_Date, SUBSTR(Order_Date, 1, 4) AS Year_Extracted
FROM Dirty_Sales_Dataset;

-- Query 7 --
SELECT LEFT(Order_ID, 3) AS Prefix FROM table;

-- Query 8 --
SELECT RIGHT(Order_ID, 6) AS Number_Part FROM Dirty_Sales_Dataset dsd ;

-- Query 9 --
SELECT
    Order_ID,
    SUBSTR(Order_ID, 1, 3) AS Left_3_Chars,
    SUBSTR(Order_ID, -6) AS Right_6_Chars
FROM Dirty_Sales_Dataset
LIMIT 20;

-- B4.2.3 POSITION() / CHARINDEX() – find location of character --

-- Query 1 --
SELECT Customer_Name, INSTR(Customer_Name, ' ') AS Space_Position
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT Product_Name, INSTR(Product_Name, 'Pro') AS Position_of_Pro
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT Customer_Name,
    CASE

```

```

        WHEN INSTR(Customer_Name, ' ') = 0 THEN 'No Space Found'
        ELSE 'Space Found'
    END AS Space_Check
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT
    Customer_Name,
    SUBSTR(Customer_Name, 1, INSTR(Customer_Name, ' ') - 1) AS First_Name,
    SUBSTR(Customer_Name, INSTR(Customer_Name, ' ') + 1) AS Last_Name
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Email, SUBSTR(Email, INSTR(Email, '@') + 1) AS Domain
FROM table;

-- Query 6 --
SELECT
    Customer_Name,
    SUBSTR(Customer_Name, 1, INSTR(Customer_Name, ' ') - 1) AS First_Name,
    SUBSTR(Customer_Name, INSTR(Customer_Name, ' ') + 1) AS Last_Name
FROM Dirty_Sales_Dataset
LIMIT 20;

-- B4.2.4 SPLIT_PART() – split string by delimiter --

-- Query 1 --
SELECT SPLIT_PART(Customer_Name, ' ', 1) FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT
    SPLIT_PART(Customer_Name, ' ', 1) AS First_Name,
    SPLIT_PART(Customer_Name, ' ', 2) AS Last_Name
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT
    Customer_Name,
    SUBSTR(Customer_Name, 1, INSTR(Customer_Name, ' ') - 1) AS First_Name,
    SUBSTR(Customer_Name, INSTR(Customer_Name, ' ') + 1) AS Last_Name
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT
    Combined_Text,
    SUBSTR(Combined_Text, 1, INSTR(Combined_Text, ',') - 1) AS Part1,
    SUBSTR(Combined_Text, INSTR(Combined_Text, ',') + 1) AS Part2
FROM table;

-- Query 5 --
SELECT
    Customer_Name,
    SUBSTR(Customer_Name, 1, INSTR(Customer_Name, ' ') - 1) AS First_Name,
    SUBSTR(Customer_Name, INSTR(Customer_Name, ' ') + 1) AS Last_Name
FROM Dirty_Sales_Dataset

```

```

LIMIT 20;

-- B4.3.1 CONCAT() and CONCAT_WS() – join strings with separator --

-- Query 1 --
SELECT CONCAT(City, ', ', State) AS Location FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT CONCAT(City, ', ', State, ', ', Country) AS Full_Address
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT CONCAT_WS(', ', City, State, Country) AS Full_Address
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT CONCAT_WS(', ', City, State, Country) FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT City || ', ' || State || ', ' || Country AS Full_Address
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT
    Customer_Name,
    City,
    State,
    Country,
    Customer_Name || ' from ' || City || ', ' || State || ', ' || Country AS
Full_Info
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 7 --
SELECT City || ', ' || State || ', ' || Country AS Full_Location
FROM Dirty_Sales_Dataset
LIMIT 20;

-- B4.3.2 REGEXP_REPLACE() – clean data using regex --

-- Query 1 --
SELECT REGEXP_REPLACE(Unit_Price, '^[^0-9.]', '', 'g') AS Cleaned_Price
FROM table;

-- Query 2 --
SELECT REGEXP_REPLACE(Customer_Name, '\s+', ' ', 'g') AS Name_Cleaned
FROM table;

-- Query 3 --
SELECT REGEXP_REPLACE(Product_Name, '^[^a-zA-Z ]', '', 'g') AS Clean_Name
FROM table;

-- Query 4 --
SELECT REPLACE(REPLACE(REPLACE(Unit_Price, '$', ''), ',', ''), ' ', '') AS
Cleaned_Price

```

```

FROM Dirty_Sales_Dataset;

-- B4.3.3 REGEXP_LIKE() / ~ operator – filter using regex patterns --

-- Query 1 --
SELECT * FROM table WHERE REGEXP_LIKE(column_name, 'pattern');

-- Query 2 --
SELECT * FROM table WHERE column_name ~ 'pattern';

-- Query 3 --
SELECT * FROM table WHERE column_name ~ '^[0-9]+$';

-- Query 4 --
SELECT * FROM table WHERE Email ~ '^[a-zA-Z0-9]+@[a-zA-Z]+\.[a-zA-Z]+$';

-- Query 5 --
SELECT * FROM table WHERE Order_ID ~ '^ORD[0-9]{6}$';

-- Query 6 --
SELECT * FROM Dirty_Sales_Dataset WHERE Order_ID REGEXP '^ORD[0-9]{6}$';

-- Query 7 --
SELECT Order_ID
FROM Dirty_Sales_Dataset
WHERE Order_ID LIKE 'ORD%' AND LENGTH(Order_ID) = 9;

-- Query 8 --
SELECT Order_ID
FROM Dirty_Sales_Dataset
WHERE Order_ID GLOB 'ORD[0-9][0-9][0-9][0-9][0-9][0-9]';

-- Query 9 --
SELECT Customer_Name
FROM Dirty_Sales_Dataset
WHERE Customer_Name GLOB '*[0-9]*';

-- Query 10 --
SELECT Order_ID
FROM Dirty_Sales_Dataset
WHERE Order_ID NOT GLOB 'ORD[0-9][0-9][0-9][0-9][0-9][0-9]'
LIMIT 20;

-- B4.3.4 CAST() and TO_CHAR() – data type conversion tricks --

-- Query 1 --
SELECT
    Unit_Price,
    CAST(Unit_Price AS REAL) AS Unit_Price_Number
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT
    Order_ID,
    Unit_Price,

```

```

        Quantity,
        CAST(Unit_Price AS REAL) * Quantity AS Calculated_Revenue
FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT
    Unit_Price AS Original,
    CAST(REPLACE(REPLACE(Unit_Price, '$', ''), ',', '')) AS REAL) AS Cleaned_Number
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT CAST(Order_Date AS DATE) FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT strftime('%d-%m-%Y', Order_Date) AS Formatted_Date
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT
    Order_ID,
    Unit_Price,
    CAST(Unit_Price AS REAL) AS Unit_Price_Number,
    Quantity,
    CAST(Unit_Price AS REAL) * Quantity AS Calculated_Total
FROM Dirty_Sales_Dataset
LIMIT 20;

-- B5.1.1 CURRENT_DATE, CURRENT_TIMESTAMP, NOW() --

-- Query 1 --
SELECT CURRENT_DATE;

-- Query 2 --
SELECT CURRENT_TIMESTAMP;

-- Query 3 --
SELECT NOW();

-- Query 4 --
SELECT datetime('now');

-- Query 5 --
SELECT
    Order_ID,
    Order_Date,
    CURRENT_DATE AS Today,
    julianday(CURRENT_DATE) - julianday(Order_Date) AS Days_Since_Order
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 6 --
SELECT Order_ID, Order_Date
FROM Dirty_Sales_Dataset
WHERE Order_Date > CURRENT_DATE;

```

```

-- Query 7 --
INSERT INTO some_log_table (action, log_time) VALUES ('Data Cleaned',
CURRENT_TIMESTAMP);

-- Query 8 --
SELECT
    CURRENT_DATE AS Today_Date,
    CURRENT_TIMESTAMP AS Today_DateTime,
    datetime('now') AS Now_Alternative;

-- Query 9 --
SELECT Order_ID, Order_Date
FROM Dirty_Sales_Dataset
WHERE Order_Date > CURRENT_DATE
LIMIT 20;

-- B5.1.2 EXTRACT() – get year, month, day, hour from date --

-- Query 1 --
SELECT EXTRACT(YEAR FROM Order_Date) FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT EXTRACT(MONTH FROM Order_Date) FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT EXTRACT(DAY FROM Order_Date) FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT EXTRACT(HOUR FROM Order_Date) FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Order_Date, strftime('%Y', Order_Date) AS Order_Year
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Order_Date, strftime('%m', Order_Date) AS Order_Month
FROM Dirty_Sales_Dataset;

-- Query 7 --
SELECT
    Order_Date,
    strftime('%Y', Order_Date) AS Order_Year,
    strftime('%m', Order_Date) AS Order_Month,
    strftime('%d', Order_Date) AS Order_Day
FROM Dirty_Sales_Dataset;

-- Query 8 --
SELECT
    strftime('%Y', Order_Date) AS Order_Year,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Order_Year
ORDER BY Order_Year;

```

```

-- Query 9 --
SELECT
    strftime('%Y-%m', Order_Date) AS Order_Month,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Order_Month
ORDER BY Order_Month;

-- Query 10 --
SELECT Order_Date, strftime('%Y', Order_Date) AS Year_Extracted
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) IS NULL;

-- Query 11 --
SELECT
    strftime('%Y', Order_Date) AS Order_Year,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Order_Year
ORDER BY Order_Year;

-- Query 12 --
SELECT COUNT(*) AS Rows_With_Bad_Date_Format
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) IS NULL;

-- TO_DATE() – converting text to date --

-- Query 1 --
SELECT TO_DATE('15-04-2023', 'DD-MM-YYYY');

-- Query 2 --
SELECT TO_DATE('Apr 15, 2023', 'Mon DD, YYYY');

-- Query 3 --
SELECT date('2024-07-05') AS Converted_Date;

-- Query 4 --
SELECT date('Apr 15, 2023') AS Converted_Date;

-- Query 5 --
SELECT
    Order_Date,
    CASE
        WHEN Order_Date LIKE '____-__-__' THEN date(Order_Date)
        WHEN Order_Date LIKE '__/__/____' THEN date(SUBSTR(Order_Date,7,4) || '-' ||
SUBSTR(Order_Date,4,2) || '-' || SUBSTR(Order_Date,1,2))
        ELSE NULL
    END AS Standardized_Date
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 6 --
SELECT Order_Date, date(Order_Date) AS Converted_Date

```

```

FROM Dirty_Sales_Dataset
WHERE date(Order_Date) IS NULL
LIMIT 30;

-- TO_DATE() – converting text to date --

-- Query 1 --
SELECT TO_DATE('15-04-2023', 'DD-MM-YYYY');

-- Query 2 --
SELECT TO_DATE('Apr 15, 2023', 'Mon DD, YYYY');

-- Query 3 --
SELECT date('2024-07-05') AS Converted_Date;

-- Query 4 --
SELECT date('Apr 15, 2023') AS Converted_Date;

-- Query 5 --
SELECT
    Order_Date,
    CASE
        WHEN Order_Date LIKE '____-__-__' THEN date(Order_Date)
        WHEN Order_Date LIKE '__/__/____' THEN date(SUBSTR(Order_Date,7,4) || '-' ||
SUBSTR(Order_Date,4,2) || '-' || SUBSTR(Order_Date,1,2))
        ELSE NULL
    END AS Standardized_Date
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 6 --
SELECT Order_Date, date(Order_Date) AS Converted_Date
FROM Dirty_Sales_Dataset
WHERE date(Order_Date) IS NULL
LIMIT 30;

-- B5.2.1 DATE + INTERVAL – adding/subtracting time --

-- Query 1 --
SELECT Order_Date + INTERVAL '7 days' FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT Order_Date - INTERVAL '1 month' FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT
    Order_Date,
    date(Order_Date, '+7 days') AS Date_After_7_Days
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 4 --
SELECT
    Order_Date,
    date(Order_Date, '-1 months') AS One_Month_Before

```

```
FROM Dirty_Sales_Dataset
LIMIT 20;
```

```
-- Query 5 --
```

```
SELECT
    Order_Date,
    date(Order_Date, '+1 years') AS Warranty_Expiry
FROM Dirty_Sales_Dataset
LIMIT 20;
```

```
-- Query 6 --
```

```
SELECT * FROM Dirty_Sales_Dataset
WHERE Order_Date >= date(CURRENT_DATE, '-30 days');
```

```
-- Query 7 --
```

```
SELECT date(Order_Date, '+1 months', '+15 days') AS Future_Date
FROM Dirty_Sales_Dataset
LIMIT 20;
```

```
-- Query 8 --
```

```
SELECT
    Order_ID,
    Order_Date,
    Ship_Date,
    date(Order_Date, '+3 days') AS Expected_Ship_Date
FROM Dirty_Sales_Dataset
LIMIT 20;
```

```
-- Query 9 --
```

```
SELECT
    Order_ID,
    Order_Date,
    date(Order_Date, '+7 days') AS Week_Later,
    date(Order_Date, '+1 months') AS Month_Later
FROM Dirty_Sales_Dataset
LIMIT 20;
```

```
-- B5.2.2 DATEDIFF() – difference between two dates --
```

```
-- Query 1 --
```

```
SELECT DATEDIFF(date1, date2);
```

```
-- Query 2 --
```

```
SELECT julianday('2024-07-15') - julianday('2024-07-05') AS Days_Difference;
```

```
-- Query 3 --
```

```
SELECT
    Order_ID,
    Order_Date,
    Ship_Date,
    julianday(Ship_Date) - julianday(Order_Date) AS Shipping_Days
FROM Dirty_Sales_Dataset
LIMIT 20;
```

```
-- Query 4 --
```

```

SELECT
    Order_ID,
    Order_Date,
    julianday(CURRENT_DATE) - julianday(Order_Date) AS Days_Since_Order
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 5 --
SELECT
    Region,
    AVG(julianday(Ship_Date) - julianday(Order_Date)) AS Avg_Shipping_Days
FROM Dirty_Sales_Dataset
GROUP BY Region
ORDER BY Avg_Shipping_Days;

-- Query 6 --
SELECT Order_ID, Order_Date, Ship_Date,
    julianday(Ship_Date) - julianday(Order_Date) AS Shipping_Days
FROM Dirty_Sales_Dataset
WHERE julianday(Ship_Date) - julianday(Order_Date) > 30
ORDER BY Shipping_Days DESC;

-- Query 7 --
SELECT Order_ID, Order_Date, Ship_Date,
    julianday(Ship_Date) - julianday(Order_Date) AS Shipping_Days
FROM Dirty_Sales_Dataset
WHERE julianday(Ship_Date) - julianday(Order_Date) < 0;

-- Query 8 --
SELECT
    Order_ID,
    CAST(julianday(Ship_Date) - julianday(Order_Date) AS INTEGER) AS Shipping_Days
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 9 --
SELECT
    Order_ID,
    Order_Date,
    Ship_Date,
    CAST(julianday(Ship_Date) - julianday(Order_Date) AS INTEGER) AS Shipping_Days
FROM Dirty_Sales_Dataset
WHERE julianday(Ship_Date) - julianday(Order_Date) < 0
LIMIT 30;

-- B5.2.3 AGE() function --

-- Query 1 --
SELECT AGE(date1, date2);

-- Query 2 --
SELECT AGE('2024-07-15', '2020-03-10');

-- Query 3 --
SELECT AGE('2000-05-15');

```

```

-- Query 4 --
SELECT
    CAST(julianday(CURRENT_DATE) - julianday(Order_Date) AS INTEGER) AS Days_Old
FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT
    Order_Date,
    CAST((julianday(CURRENT_DATE) - julianday(Order_Date)) / 365 AS INTEGER) AS
Approx_Years,
    CAST(((julianday(CURRENT_DATE) - julianday(Order_Date)) % 365) / 30 AS INTEGER)
AS Approx_Months
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 6 --
SELECT
    Order_ID,
    Order_Date,
    CAST((julianday(CURRENT_DATE) - julianday(Order_Date)) / 365 AS INTEGER) AS
Order_Age_Years
FROM Dirty_Sales_Dataset
LIMIT 20;

-- B5.2.4 DATEADD() in SQL Server and MySQL equivalent --

-- Query 1 --
SELECT DATEADD(day, 7, Order_Date) FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT DATEADD(month, 1, Order_Date) FROM Dirty_Sales_Dataset;

-- Query 3 --
SELECT DATE_ADD(Order_Date, INTERVAL 7 DAY) FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT date(Order_Date, '+7 days') AS Week_Later
FROM Dirty_Sales_Dataset;

-- B5.3.1 DATE_TRUNC() – truncate to month, week, year --

-- Query 1 --
SELECT DATE_TRUNC('month', '2024-07-15');

-- Query 2 --
SELECT DATE_TRUNC('year', '2024-07-15');

-- Query 3 --
SELECT DATE_TRUNC('week', '2024-07-15');

-- Query 4 --
SELECT strftime('%Y-%m', Order_Date) || '-01' AS Month_Truncated
FROM Dirty_Sales_Dataset;

```

```

-- Query 5 --
SELECT strftime('%Y', Order_Date) || '-01-01' AS Year_Truncated
FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT
    strftime('%Y-%m', Order_Date) AS Sales_Month,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Sales_Month
ORDER BY Sales_Month;

-- Query 7 --
SELECT
    strftime('%Y', Order_Date) AS Sales_Year,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Sales_Year
ORDER BY Sales_Year;

-- Query 8 --
SELECT
    strftime('%Y-W%W', Order_Date) AS Sales_Week,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Sales_Week
ORDER BY Sales_Week
LIMIT 20;

-- Query 9 --
SELECT
    strftime('%Y-%m', Order_Date) AS Sales_Month,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Sales_Month
ORDER BY Sales_Month;

-- B5.3.2 Generating date series for gap analysis --

-- Query 1 --
SELECT generate_series('2024-01-01'::date, '2024-12-31'::date, '1 day'::interval) AS
calendar_date;

-- Query 2 --
WITH RECURSIVE date_series(calendar_date) AS (
    SELECT date('2024-01-01')
    UNION ALL
    SELECT date(calendar_date, '+1 day')
    FROM date_series
    WHERE calendar_date < date('2024-12-31')
)
SELECT calendar_date FROM date_series;

```

```

-- Query 3 --
WITH RECURSIVE date_series(calendar_date) AS (
    SELECT date('2023-01-01')
    UNION ALL
    SELECT date(calendar_date, '+1 day')
    FROM date_series
    WHERE calendar_date < date('2025-06-19')
)
SELECT ds.calendar_date
FROM date_series ds
LEFT JOIN Dirty_Sales_Dataset sales ON ds.calendar_date = sales.Order_Date
WHERE sales.Order_Date IS NULL;

-- B5.3.3 Timezone handling – AT TIME ZONE --

-- Query 1 --
SELECT '2024-07-15 10:00:00' AT TIME ZONE 'UTC' AT TIME ZONE 'Asia/Kolkata';

-- Query 2 --
SELECT datetime('2024-07-15 10:00:00', 'localtime');

-- Query 3 --
SELECT datetime('2024-07-15 10:00:00', 'utc');

-- B5.3.4 Fiscal year calculations using CASE + EXTRACT --

-- Query 1 --
SELECT
    Order_Date,
    CASE
        WHEN CAST(strftime('%m', Order_Date) AS INTEGER) >= 4
            THEN strftime('%Y', Order_Date) || '-' || (CAST(strftime('%Y',
Order_Date) AS INTEGER) + 1)
        ELSE (CAST(strftime('%Y', Order_Date) AS INTEGER) - 1) || '-' ||
strftime('%Y', Order_Date)
        END AS Fiscal_Year
FROM Dirty_Sales_Dataset;

-- Query 2 --
SELECT
    CASE
        WHEN CAST(strftime('%m', Order_Date) AS INTEGER) >= 4
            THEN strftime('%Y', Order_Date) || '-' || (CAST(strftime('%Y',
Order_Date) AS INTEGER) + 1)
        ELSE (CAST(strftime('%Y', Order_Date) AS INTEGER) - 1) || '-' ||
strftime('%Y', Order_Date)
        END AS Fiscal_Year,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Fiscal_Year
ORDER BY Fiscal_Year;

-- Query 3 --
SELECT

```

```

    Order_Date,
    CASE
        WHEN CAST(strftime('%m', Order_Date) AS INTEGER) >= 4
            THEN CAST(strftime('%Y', Order_Date) AS INTEGER)
        ELSE CAST(strftime('%Y', Order_Date) AS INTEGER) - 1
    END AS Fiscal_Year_Start
FROM Dirty_Sales_Dataset;

-- Query 4 --
SELECT
    CASE
        WHEN CAST(strftime('%m', Order_Date) AS INTEGER) >= 4
            THEN strftime('%Y', Order_Date) || '-' || (CAST(strftime('%Y',
Order_Date) AS INTEGER) + 1)
        ELSE (CAST(strftime('%Y', Order_Date) AS INTEGER) - 1) || '-' ||
strftime('%Y', Order_Date)
    END AS Fiscal_Year,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Fiscal_Year
ORDER BY Fiscal_Year;

```

-- C1.1.1 Primary Key – unique identifier for each row --

```

-- Query 1 --
CREATE TABLE Orders (
    Order_ID VARCHAR(10) PRIMARY KEY,
    Customer_Name VARCHAR(50),
    Sales DECIMAL(10,2)
);

```

```

-- Query 2 --
SELECT Order_ID, COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Order_ID
HAVING COUNT(*) > 1
ORDER BY Frequency DESC;

```

```

-- Query 3 --
SELECT Order_ID, COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Order_ID
HAVING COUNT(*) > 1
ORDER BY Frequency DESC
LIMIT 20;

```

-- C1.1.2 Foreign Key – connecting two tables --

```

-- Query 1 --
CREATE TABLE Customers (
    Customer_ID VARCHAR(10) PRIMARY KEY,
    Customer_Name VARCHAR(50)
);

```

```

CREATE TABLE Orders (
    Order_ID VARCHAR(10) PRIMARY KEY,
    Customer_ID VARCHAR(10),
    Sales DECIMAL(10,2),
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)
);

-- Query 2 --
INSERT INTO Orders (Order_ID, Customer_ID, Sales)
VALUES ('ORD999', 'CUST999', 5000);

-- C1.1.3 One-to-One, One-to-Many, Many-to-Many relationships --

-- Query 1 --
SELECT Customer_ID, COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
ORDER BY Total_Orders DESC
LIMIT 10;

-- Query 2 --
SELECT Customer_ID, COUNT(DISTINCT Product_ID) AS Unique_Products_Bought
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
ORDER BY Unique_Products_Bought DESC
LIMIT 10;

-- Query 3 --
SELECT Product_ID, COUNT(DISTINCT Customer_ID) AS Unique_Customers
FROM Dirty_Sales_Dataset
GROUP BY Product_ID
ORDER BY Unique_Customers DESC
LIMIT 10;

-- Query 4 --
SELECT Customer_ID, COUNT(DISTINCT Product_ID) AS Unique_Products_Bought
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
HAVING COUNT(DISTINCT Product_ID) > 5
ORDER BY Unique_Products_Bought DESC
LIMIT 10;

-- C1.2.1 Normalization – why data is split across tables --

-- Query 1 --
SELECT Customer_ID, COUNT(DISTINCT Customer_Name) AS Name_Variations
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
HAVING COUNT(DISTINCT Customer_Name) > 1;

-- Query 2 --
SELECT Product_ID, COUNT(DISTINCT Product_Name) AS Name_Variations
FROM Dirty_Sales_Dataset
GROUP BY Product_ID
HAVING COUNT(DISTINCT Product_Name) > 1;

```

-- C1.2.3 JOIN vs Subquery – when to use which --

-- Query 1 --

```
SELECT a.column1, b.column2
FROM TableA a
JOIN TableB b ON a.id = b.id;
```

-- Query 2 --

```
SELECT column1
FROM TableA
WHERE id IN (SELECT id FROM TableB WHERE condition);
```

-- Query 3 --

```
SELECT DISTINCT c.Customer_ID, c.Customer_Name
FROM Customers c
JOIN Orders o ON c.Customer_ID = o.Customer_ID;
```

-- Query 4 --

```
SELECT Customer_ID, Customer_Name
FROM Customers
WHERE Customer_ID IN (SELECT Customer_ID FROM Orders);
```

-- Query 5 --

```
SELECT c.Customer_Name, o.Order_Date, o.Sales
FROM Customers c
JOIN Orders o ON c.Customer_ID = o.Customer_ID;
```

-- Query 6 --

```
SELECT * FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE Sales > 10000);
```

-- Query 7 --

```
SELECT * FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT AVG(Sales) FROM Dirty_Sales_Dataset);
```

-- Query 8 --

```
SELECT Order_ID, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT AVG(Sales) FROM Dirty_Sales_Dataset);
```

-- Query 9 --

```
SELECT * FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (
    SELECT Customer_ID FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
    HAVING SUM(Sales) > 100000
);
```

-- C1.2.4 Cartesian product – what happens without JOIN condition --

-- Query 1 --

```
SELECT * FROM Customers, Products;
```

```

-- Query 2 --
SELECT * FROM Customers CROSS JOIN Products;

-- Query 3 --
SELECT c.Customer_Name, o.Sales
FROM Customers c
JOIN Orders o;

-- Query 4 --
SELECT c.Customer_Name, o.Sales
FROM Customers c, Orders o;

-- Query 5 --
SELECT s.Size, c.Color
FROM Sizes s
CROSS JOIN Colors c;

-- Query 6 --
SELECT c.Customer_Name, o.Sales
FROM Customers c
JOIN Orders o ON c.Customer_ID = o.Customer_ID;

-- Query 7 --
SELECT * FROM Sizes CROSS JOIN Colors;

-- C2.1.1 INNER JOIN syntax – only matching rows from both tables --

-- Query 1 --
SELECT DISTINCT Salesperson FROM Dirty_Sales_Dataset;

-- Query 2 --
CREATE TABLE Salesperson_Info (
    Salesperson TEXT PRIMARY KEY,
    Region_Assigned TEXT,
    Years_Experience INTEGER
);

INSERT INTO Salesperson_Info VALUES
('Ravi Kumar', 'North', 5),
('Anjali Mehta', 'South', 3),
('Suresh Rao', 'East', 8),
('Priya Nair', 'West', 2);

-- Query 3 --
SELECT
    d.Order_ID,
    d.Salesperson,
    d.Sales,
    s.Region_Assigned,
    s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;

-- Query 4 --
SELECT

```

```

    d.Order_ID,
    d.Salesperson,
    d.Region AS Order_Region,
    s.Region_Assigned AS Assigned_Region,
    CASE
        WHEN d.Region = s.Region_Assigned THEN 'Match'
        ELSE 'Mismatch'
    END AS Region_Check
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;

```

-- Query 5 --

```

CREATE TABLE Salesperson_Info (
    Salesperson TEXT PRIMARY KEY,
    Region_Assigned TEXT,
    Years_Experience INTEGER
);

```

```

INSERT INTO Salesperson_Info VALUES
('Actual_Name_1', 'North', 5),
('Actual_Name_2', 'South', 3);

```

-- Query 6 --

```

SELECT d.Order_ID, d.Salesperson, d.Sales, s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 20;

```

-- C2.1.2 JOIN with WHERE filter --

-- Query 1 --

```

SELECT
    d.Order_ID,
    d.Salesperson,
    d.Sales,
    s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE s.Years_Experience > 5;

```

-- Query 2 --

```

SELECT
    d.Order_ID,
    d.Salesperson,
    d.Sales,
    s.Region_Assigned
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE s.Region_Assigned = 'North' AND d.Sales > 5000;

```

-- Query 3 --

```

SELECT
    d.Order_ID,
    d.Customer_Name,
    d.Sales,

```

```

        s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE d.Sales > 0
      AND d.Order_Status != 'Cancelled'
      AND s.Years_Experience >= 3;

```

-- Query 4 --

```

SELECT d.Order_ID, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE d.Sales > 5000;

```

-- Query 5 --

```

SELECT
    d.Salesperson,
    s.Region_Assigned,
    s.Years_Experience,
    COUNT(*) AS Total_Orders,
    SUM(d.Sales) AS Total_Sales
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE d.Sales > 0 AND d.Order_Status != 'Cancelled'
GROUP BY d.Salesperson, s.Region_Assigned, s.Years_Experience
ORDER BY Total_Sales DESC;

```

-- Query 6 --

```

SELECT
    d.Order_ID,
    d.Salesperson,
    d.Sales,
    s.Region_Assigned,
    s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE d.Sales > 3000 AND s.Years_Experience > 3
ORDER BY d.Sales DESC
LIMIT 20;

```

-- C2.1.3 JOIN with GROUP BY and aggregate --

-- Query 1 --

```

SELECT
    s.Region_Assigned,
    SUM(d.Sales) AS Total_Sales,
    COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
GROUP BY s.Region_Assigned
ORDER BY Total_Sales DESC;

```

-- Query 2 --

```

SELECT
    s.Years_Experience,
    COUNT(*) AS Total_Orders,

```

```

        SUM(d.Sales) AS Total_Sales,
        AVG(d.Sales) AS Avg_Sales_Per_Order
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
GROUP BY s.Years_Experience
ORDER BY s.Years_Experience;

```

-- Query 3 --

```

SELECT
    d.Salesperson,
    s.Region_Assigned,
    s.Years_Experience,
    COUNT(*) AS Total_Orders,
    SUM(d.Sales) AS Total_Sales,
    AVG(d.Sales) AS Avg_Order_Value
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
GROUP BY d.Salesperson, s.Region_Assigned, s.Years_Experience
ORDER BY Total_Sales DESC;

```

-- Query 4 --

```

SELECT
    s.Region_Assigned,
    COUNT(*) AS Valid_Orders,
    SUM(d.Sales) AS Total_Sales
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE d.Sales > 0 AND d.Order_Status != 'Cancelled'
GROUP BY s.Region_Assigned
HAVING SUM(d.Sales) > 100000
ORDER BY Total_Sales DESC;

```

-- Query 5 --

```

SELECT
    s.Region_Assigned,
    COUNT(*) AS Total_Orders,
    SUM(d.Sales) AS Total_Sales,
    AVG(d.Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
GROUP BY s.Region_Assigned
ORDER BY Total_Sales DESC;

```

```

SELECT * FROM Salesperson_Info;

```

```

SELECT DISTINCT Salesperson FROM Dirty_Sales_Dataset;

```

```

DROP TABLE Salesperson_Info;

```

```

CREATE TABLE Salesperson_Info (
    Salesperson TEXT PRIMARY KEY,
    Region_Assigned TEXT,
    Years_Experience INTEGER
);

```

```

INSERT INTO Salesperson_Info VALUES
('Karan', 'North', 5),
('Meera', 'South', 3),
('Anjali', 'East', 8),
('Rohit', 'West', 2);

-- C2.1.4 Joining on multiple columns --

-- Query 1 --
JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson

-- Query 2 --
JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson AND d.Region =
s.Region_Assigned

-- Query 3 --
DROP TABLE Salesperson_Info;

CREATE TABLE Salesperson_Info (
    Salesperson TEXT,
    Region_Assigned TEXT,
    Years_Experience INTEGER,
    PRIMARY KEY (Salesperson, Region_Assigned)
);

INSERT INTO Salesperson_Info VALUES
('Karan', 'North', 5),
('Meera', 'South', 3),
('Anjali', 'East', 8),
('Rohit', 'West', 2);

-- Query 4 --
SELECT
    d.Order_ID,
    d.Salesperson,
    d.Region,
    d.Sales,
    s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s
    ON d.Salesperson = s.Salesperson
    AND d.Region = s.Region_Assigned
LIMIT 20;

-- Query 5 --
SELECT
    d.Salesperson,
    d.Region,
    COUNT(*) AS Total_Orders,
    SUM(d.Sales) AS Total_Sales
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s
    ON d.Salesperson = s.Salesperson
    AND d.Region = s.Region_Assigned
WHERE d.Sales > 0

```

```
GROUP BY d.Salesperson, d.Region
ORDER BY Total_Sales DESC;
```

```
-- C2.2.1 LEFT JOIN – all rows from left, matching from right --
```

```
-- Query 1 --
```

```
SELECT d.Salesperson, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 2 --
```

```
SELECT d.Salesperson, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 3 --
```

```
DELETE FROM Salesperson_Info WHERE Salesperson = 'Rohit';
```

```
-- Query 4 --
```

```
SELECT
    d.Order_ID,
    d.Salesperson,
    d.Sales,
    s.Region_Assigned
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 30;
```

```
-- Query 5 --
```

```
SELECT DISTINCT d.Salesperson
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE s.Salesperson IS NULL;
```

```
-- Query 6 --
```

```
SELECT
    d.Salesperson,
    d.Sales,
    s.Region_Assigned
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 30;
```

```
-- C2.2.2 Finding missing records using LEFT JOIN + IS NULL --
```

```
-- Query 1 --
```

```
SELECT a.*
FROM TableA a
LEFT JOIN TableB b ON a.key = b.key
WHERE b.key IS NULL;
```

```
-- Query 2 --
```

```
SELECT DISTINCT d.Salesperson
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
```

```
WHERE s.Salesperson IS NULL;
```

```
-- Query 3 --
```

```
SELECT c.Customer_ID, c.Customer_Name
FROM Customers c
LEFT JOIN Orders o ON c.Customer_ID = o.Customer_ID
WHERE o.Customer_ID IS NULL;
```

```
-- Query 4 --
```

```
SELECT p.Product_ID, p.Product_Name
FROM Products p
LEFT JOIN Orders o ON p.Product_ID = o.Product_ID
WHERE o.Product_ID IS NULL;
```

```
-- Query 5 --
```

```
SELECT DISTINCT d.Salesperson
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE s.Region_Assigned IS NULL;
```

```
-- Query 6 --
```

```
SELECT Customer_ID FROM Customers
WHERE Customer_ID NOT IN (SELECT Customer_ID FROM Orders);
```

```
-- C2.2.3 Difference between LEFT JOIN and INNER JOIN in output --
```

```
-- Query 1 --
```

```
SELECT d.Salesperson, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 2 --
```

```
SELECT d.Salesperson, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 3 --
```

```
SELECT COUNT(*) AS Inner_Join_Count
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 4 --
```

```
SELECT COUNT(*) AS Left_Join_Count
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- C2.2.4 LEFT JOIN with aggregate – counting with NULLs --
```

```
-- Query 1 --
```

```
SELECT
    s.Region_Assigned,
    COUNT(*) AS Count_Star,
    COUNT(d.Order_ID) AS Count_Column
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
```

```
GROUP BY s.Region_Assigned;
```

```
-- Query 2 --
```

```
SELECT
    s.Salesperson,
    COUNT(d.Order_ID) AS Total_Orders
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
GROUP BY s.Salesperson;
```

```
-- Query 3 --
```

```
SELECT
    s.Salesperson,
    COUNT(*) AS Total_Orders
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
GROUP BY s.Salesperson;
```

```
-- Query 4 --
```

```
SELECT
    s.Salesperson,
    COUNT(*) AS Wrong_Count_Using_Star,
    COUNT(d.Order_ID) AS Correct_Count
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
GROUP BY s.Salesperson;
```

```
-- Query 5 --
```

```
SELECT
    s.Salesperson,
    COALESCE(SUM(d.Sales), 0) AS Total_Sales
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
GROUP BY s.Salesperson;
```

```
-- Query 6 --
```

```
SELECT
    s.Salesperson,
    s.Region_Assigned,
    COUNT(*) AS Count_Using_Star,
    COUNT(d.Order_ID) AS Correct_Order_Count,
    COALESCE(SUM(d.Sales), 0) AS Total_Sales
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
GROUP BY s.Salesperson, s.Region_Assigned;
```

```
-- C2.3.1 RIGHT JOIN syntax and use case --
```

```
-- Query 1 --
```

```
SELECT d.Salesperson, s.Region_Assigned
FROM Dirty_Sales_Dataset d
RIGHT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 2 --
```

```
SELECT d.Salesperson, s.Region_Assigned
```

```
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson;
```

```
-- Query 3 --
```

```
SELECT
    s.Salesperson,
    s.Region_Assigned,
    d.Order_ID,
    d.Sales
FROM Dirty_Sales_Dataset d
RIGHT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 4 --
```

```
INSERT INTO Salesperson_Info VALUES ('New_Person', 'Central', 1);
```

```
-- Query 5 --
```

```
SELECT s.Salesperson, s.Region_Assigned, d.Order_ID, d.Sales
FROM Dirty_Sales_Dataset d
RIGHT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 20;
```

```
-- Query 6 --
```

```
SELECT s.Salesperson, s.Region_Assigned, d.Order_ID, d.Sales
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
LIMIT 20;
```

```
-- Query 7 --
```

```
SELECT
    s.Salesperson,
    s.Region_Assigned,
    d.Order_ID,
    d.Sales
FROM Dirty_Sales_Dataset d
RIGHT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 20;
```

```
-- C2.3.2 FULL OUTER JOIN – all rows from both tables --
```

```
-- Query 1 --
```

```
SELECT
    d.Salesperson AS Dataset_Salesperson,
    s.Salesperson AS Info_Salesperson,
    d.Order_ID,
    s.Region_Assigned
FROM Dirty_Sales_Dataset d
FULL OUTER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 30;
```

```
-- Query 2 --
```

```
SELECT d.Salesperson AS Dataset_SP, s.Salesperson AS Info_SP, d.Order_ID,
s.Region_Assigned
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
```

UNION

```
SELECT d.Salesperson AS Dataset_SP, s.Salesperson AS Info_SP, d.Order_ID,  
s.Region_Assigned  
FROM Dirty_Sales_Dataset d  
RIGHT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

-- Query 3 --

```
SELECT d.Salesperson AS Dataset_SP, s.Salesperson AS Info_SP, d.Order_ID,  
s.Region_Assigned  
FROM Dirty_Sales_Dataset d  
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
```

UNION

```
SELECT d.Salesperson AS Dataset_SP, s.Salesperson AS Info_SP, d.Order_ID,  
s.Region_Assigned  
FROM Salesperson_Info s  
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson;
```

-- Query 4 --

```
SELECT  
    COALESCE(d.Salesperson, s.Salesperson) AS Salesperson,  
    CASE WHEN d.Salesperson IS NULL THEN 'Missing in Sales Data'  
          WHEN s.Salesperson IS NULL THEN 'Missing in Info Table'  
          ELSE 'Matched'  
    END AS Status  
FROM Dirty_Sales_Dataset d  
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
```

UNION

```
SELECT  
    COALESCE(d.Salesperson, s.Salesperson) AS Salesperson,  
    CASE WHEN d.Salesperson IS NULL THEN 'Missing in Sales Data'  
          WHEN s.Salesperson IS NULL THEN 'Missing in Info Table'  
          ELSE 'Matched'  
    END AS Status  
FROM Salesperson_Info s  
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson;
```

-- Query 5 --

```
SELECT  
    d.Salesperson AS Dataset_SP,  
    s.Salesperson AS Info_SP,  
    d.Order_ID,  
    s.Region_Assigned  
FROM Dirty_Sales_Dataset d  
FULL OUTER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson  
LIMIT 30;
```

-- C2.3.4 Real use case – finding unmatched records in both tables --

-- Query 1 --

```
SELECT
```

```

        COALESCE(d.Salesperson, s.Salesperson) AS Salesperson,
        CASE
            WHEN s.Salesperson IS NULL THEN 'Missing in Salesperson_Info (HR Record Missing)'
            WHEN d.Salesperson IS NULL THEN 'Missing in Sales Data (No Sales Made)'
        END AS Issue_Type
    FROM Dirty_Sales_Dataset d
    LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
    WHERE s.Salesperson IS NULL

```

UNION

```

SELECT
    COALESCE(d.Salesperson, s.Salesperson) AS Salesperson,
    CASE
        WHEN s.Salesperson IS NULL THEN 'Missing in Salesperson_Info (HR Record Missing)'
        WHEN d.Salesperson IS NULL THEN 'Missing in Sales Data (No Sales Made)'
    END AS Issue_Type
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
WHERE d.Salesperson IS NULL;

```

-- Query 2 --

```

SELECT
    d.Salesperson AS Missing_HR_Record
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE s.Salesperson IS NULL

```

UNION

```

SELECT
    s.Salesperson AS No_Sales_Made
FROM Salesperson_Info s
LEFT JOIN Dirty_Sales_Dataset d ON s.Salesperson = d.Salesperson
WHERE d.Salesperson IS NULL;

```

-- C2.4.1 What is a Self JOIN --

-- Query 1 --

```

SELECT DISTINCT a.Order_ID AS Order_1, b.Order_ID AS Order_2, a.Sales
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Sales = b.Sales
    AND a.Order_ID < b.Order_ID;

```

-- Query 2 --

```

SELECT
    a.Order_ID AS Order_1,
    b.Order_ID AS Order_2,
    a.Customer_Name,
    a.Sales
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b

```

```

    ON a.Customer_Name = b.Customer_Name
    AND a.Order_ID <> b.Order_ID
ORDER BY a.Customer_Name;

```

-- Query 3 --

```

SELECT
    a.Salesperson AS Salesperson_1,
    b.Salesperson AS Salesperson_2,
    a.Region
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Region = b.Region
    AND a.Salesperson <> b.Salesperson
GROUP BY a.Salesperson, b.Salesperson, a.Region;

```

-- Query 4 --

```

SELECT
    a.Order_ID,
    a.Sales AS Sales_A,
    b.Order_ID AS Compared_With,
    b.Sales AS Sales_B
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Salesperson = b.Salesperson
    AND a.Order_ID <> b.Order_ID
    AND a.Sales > b.Sales
ORDER BY a.Salesperson, a.Sales DESC;

```

-- C2.4.2 Comparing rows within same table --

-- Query 1 --

```

SELECT
    a.Order_ID AS Order_1,
    b.Order_ID AS Order_2,
    a.Customer_ID,
    a.Order_Date
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND a.Order_Date = b.Order_Date
    AND a.Order_ID <> b.Order_ID;

```

-- Query 2 --

```

SELECT
    a.Order_ID,
    a.Product_ID,
    a.Sales AS Higher_Sales,
    b.Order_ID AS Compared_With,
    b.Sales AS Lower_Sales
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Product_ID = b.Product_ID
    AND a.Sales > b.Sales;

```

-- Query 3 --

```

SELECT
    a.Customer_ID,
    a.Order_ID AS First_Order,
    a.Order_Date AS First_Date,
    b.Order_ID AS Second_Order,
    b.Order_Date AS Second_Date
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND b.Order_Date > a.Order_Date;

```

-- Query 4 --

```

SELECT
    a.Customer_ID,
    a.Order_ID AS First_Order,
    a.Order_Date AS First_Date,
    MIN(b.Order_Date) AS Next_Order_Date
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND b.Order_Date > a.Order_Date
GROUP BY a.Customer_ID, a.Order_ID, a.Order_Date;

```

-- C2.4.3 Employee-Manager hierarchy using Self JOIN --

-- Query 1 --

```

CREATE TABLE Salesperson_Info (
    Salesperson TEXT PRIMARY KEY,
    Region_Assigned TEXT,
    Years_Experience INTEGER,
    Reports_To TEXT
);

INSERT INTO Salesperson_Info VALUES
('Ravi Kumar', 'North', 5, NULL),
('Anjali Mehta', 'South', 3, 'Ravi Kumar'),
('Suresh Rao', 'East', 8, 'Ravi Kumar'),
('Priya Nair', 'West', 2, 'Anjali Mehta');

```

-- Query 2 --

```

SELECT
    s.Salesperson AS Employee,
    m.Salesperson AS Manager
FROM Salesperson_Info s
LEFT JOIN Salesperson_Info m
    ON s.Reports_To = m.Salesperson;

```

-- Query 3 --

```

SELECT
    s.Salesperson AS Employee,
    m.Salesperson AS Manager,
    s.Region_Assigned,
    s.Years_Experience
FROM Salesperson_Info s
LEFT JOIN Salesperson_Info m

```

```
    ON s.Reports_To = m.Salesperson
ORDER BY m.Salesperson;
```

```
-- Query 4 --
```

```
SELECT
    m.Salesperson AS Manager,
    COUNT(s.Salesperson) AS Number_Of_Reports
FROM Salesperson_Info m
LEFT JOIN Salesperson_Info s
    ON s.Reports_To = m.Salesperson
GROUP BY m.Salesperson;
```

```
-- C2.4.4 Finding duplicates using Self JOIN --
```

```
-- Query 1 --
```

```
SELECT
    a.Order_ID AS Order_1,
    b.Order_ID AS Order_2,
    a.Customer_ID,
    a.Product_ID,
    a.Order_Date,
    a.Sales
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND a.Product_ID = b.Product_ID
    AND a.Order_Date = b.Order_Date
    AND a.Sales = b.Sales
    AND a.Order_ID < b.Order_ID;
```

```
-- Query 2 --
```

```
SELECT
    a.Order_ID AS Order_1,
    b.Order_ID AS Order_2,
    a.Customer_ID,
    a.Order_Date
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND a.Order_Date = b.Order_Date
    AND a.Order_ID < b.Order_ID;
```

```
-- Query 3 --
```

```
SELECT COUNT(*) AS Duplicate_Pairs
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND a.Product_ID = b.Product_ID
    AND a.Order_Date = b.Order_Date
    AND a.Sales = b.Sales
    AND a.Order_ID < b.Order_ID;
```

```
-- Query 4 --
```

```
SELECT
    a.Customer_ID,
```

```

        a.Order_Date,
        COUNT(*) AS Duplicate_Pairs
FROM Dirty_Sales_Dataset a
JOIN Dirty_Sales_Dataset b
    ON a.Customer_ID = b.Customer_ID
    AND a.Order_Date = b.Order_Date
    AND a.Order_ID < b.Order_ID
GROUP BY a.Customer_ID, a.Order_Date
ORDER BY Duplicate_Pairs DESC;

```

-- C2.5.1 What CROSS JOIN does – Cartesian product --

-- Query 1 --

```

SELECT DISTINCT
    r.Region,
    c.Category
FROM (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) r
CROSS JOIN (SELECT DISTINCT Category FROM Dirty_Sales_Dataset) c;

```

-- Query 2 --

```

SELECT
    p.Salesperson,
    reg.Region
FROM (SELECT DISTINCT Salesperson FROM Dirty_Sales_Dataset) p
CROSS JOIN (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) reg
ORDER BY p.Salesperson, reg.Region;

```

-- Query 3 --

```

SELECT
    r.Region,
    c.Category
FROM (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) r
CROSS JOIN (SELECT DISTINCT Category FROM Dirty_Sales_Dataset) c
WHERE NOT EXISTS (
    SELECT 1 FROM Dirty_Sales_Dataset d
    WHERE d.Region = r.Region AND d.Category = c.Category
);

```

-- Query 4 --

```

SELECT COUNT(*) AS Total_Combinations
FROM (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) r
CROSS JOIN (SELECT DISTINCT Category FROM Dirty_Sales_Dataset) c;

```

-- C2.5.2 Legitimate use cases in Data Analysis (date × product combinations) --

-- Query 1 --

```

SELECT
    grid.Order_Date,
    grid.Product_ID,
    s.Sales
FROM (
    SELECT d.Order_Date, p.Product_ID
    FROM (SELECT DISTINCT Order_Date FROM Dirty_Sales_Dataset) d
    CROSS JOIN (SELECT DISTINCT Product_ID FROM Dirty_Sales_Dataset) p
) grid

```

```

LEFT JOIN Dirty_Sales_Dataset s
    ON grid.Order_Date = s.Order_Date
    AND grid.Product_ID = s.Product_ID
WHERE s.Sales IS NULL;

```

-- Query 2 --

```

SELECT
    grid.Region,
    grid.Category
FROM (
    SELECT r.Region, c.Category
    FROM (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) r
    CROSS JOIN (SELECT DISTINCT Category FROM Dirty_Sales_Dataset) c
) grid
LEFT JOIN Dirty_Sales_Dataset s
    ON grid.Region = s.Region
    AND grid.Category = s.Category
WHERE s.Category IS NULL;

```

-- Query 3 --

```

SELECT
    grid.Salesperson,
    grid.Region
FROM (
    SELECT sp.Salesperson, r.Region
    FROM (SELECT DISTINCT Salesperson FROM Dirty_Sales_Dataset) sp
    CROSS JOIN (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) r
) grid
LEFT JOIN Dirty_Sales_Dataset s
    ON grid.Salesperson = s.Salesperson
    AND grid.Region = s.Region
WHERE s.Salesperson IS NULL;

```

-- Query 4 --

```

SELECT
    COUNT(*) AS Total_Expected_Combinations,
    (SELECT COUNT(*) FROM (SELECT DISTINCT Region, Category FROM Dirty_Sales_Dataset)
x) AS Actual_Combinations
FROM (
    SELECT r.Region, c.Category
    FROM (SELECT DISTINCT Region FROM Dirty_Sales_Dataset) r
    CROSS JOIN (SELECT DISTINCT Category FROM Dirty_Sales_Dataset) c
) grid;

```

-- C2.5.3 CROSS JOIN for generating test data --

-- Query 1 --

```

SELECT
    pm.Payment_Method,
    sm.Ship_Mode
FROM (SELECT DISTINCT Payment_Method FROM Dirty_Sales_Dataset) pm
CROSS JOIN (SELECT DISTINCT Ship_Mode FROM Dirty_Sales_Dataset) sm;

```

-- Query 2 --

```

CREATE TABLE Digits (n INTEGER);

```

```

INSERT INTO Digits VALUES (0),(1),(2),(3),(4),(5),(6),(7),(8),(9);

SELECT (a.n * 10 + b.n) AS Test_ID
FROM Digits a
CROSS JOIN Digits b
ORDER BY Test_ID;

-- Query 3 --
SELECT
    c.Customer_Name,
    p.Product_Name,
    p.Category
FROM (SELECT DISTINCT Customer_Name FROM Dirty_Sales_Dataset LIMIT 5) c
CROSS JOIN (SELECT DISTINCT Product_Name, Category FROM Dirty_Sales_Dataset) p;

-- Query 4 --
CREATE TABLE Years (Year INTEGER);
INSERT INTO Years VALUES (2023),(2024),(2025);

CREATE TABLE Months (Month INTEGER);
INSERT INTO Months VALUES (1),(2),(3),(4),(5),(6),(7),(8),(9),(10),(11),(12);

SELECT
    y.Year,
    m.Month
FROM Years y
CROSS JOIN Months m
ORDER BY y.Year, m.Month;

-- C3.1.1 Subquery in WHERE clause --

-- Query 1 --
SELECT Order_ID, Customer_Name, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT AVG(Sales) FROM Dirty_Sales_Dataset);

-- Query 2 --
SELECT Order_ID, Customer_ID, Category, Sales
FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Dirty_Sales_Dataset
    WHERE Category = 'Electronics'
);

-- Query 3 --
SELECT Order_ID, Customer_Name, Sales
FROM Dirty_Sales_Dataset
WHERE Sales = (SELECT MAX(Sales) FROM Dirty_Sales_Dataset);

-- Query 4 --
SELECT DISTINCT Customer_ID, Customer_Name
FROM Dirty_Sales_Dataset
WHERE Customer_ID NOT IN (
    SELECT Customer_ID

```

```

        FROM Dirty_Sales_Dataset
        WHERE Return_Status = 'Yes' AND Customer_ID IS NOT NULL
    );

-- Query 5 --
SELECT Order_ID, Customer_Name, Sales
FROM Dirty_Sales_Dataset
WHERE Sales = (SELECT MIN(Sales) FROM Dirty_Sales_Dataset);

-- Query 6 --
SELECT Order_ID, Customer_ID, Product_ID, Sales
FROM Dirty_Sales_Dataset
WHERE Product_ID IN (
    SELECT Product_ID
    FROM Dirty_Sales_Dataset
    GROUP BY Product_ID
    HAVING SUM(Sales) > (SELECT AVG(Sales) * 10 FROM Dirty_Sales_Dataset)
);

-- C3.1.2 Subquery in FROM clause (inline view) --

-- Query 1 --
SELECT * FROM (
    SELECT Region, SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
) AS region_summary
WHERE Total_Sales > 500000;

-- Query 2 --
SELECT * FROM (
    SELECT Customer_ID, COUNT(*) AS Total_Orders, SUM(Sales) AS Total_Spent
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
) AS customer_summary
WHERE Total_Spent > 50000
ORDER BY Total_Spent DESC;

-- Query 3 --
SELECT AVG(Category_Avg_Sales) AS Overall_Avg_Of_Category_Averages
FROM (
    SELECT Category, AVG(Sales) AS Category_Avg_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Category
) AS category_summary;

-- Query 4 --
SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
HAVING SUM(Sales) > 500000;

-- Query 5 --
SELECT * FROM (
    SELECT Region, SUM(Sales) AS Total_Sales

```

```

        FROM Dirty_Sales_Dataset
        GROUP BY Region
    ) AS summary
WHERE Total_Sales > 500000;

-- Query 6 --
SELECT * FROM (
    SELECT Region, COUNT(*) AS Order_Count, SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
) AS region_summary
WHERE Total_Sales > 500000
ORDER BY Total_Sales DESC;

-- C3.1.3 Subquery in SELECT clause (scalar subquery) --

-- Query 1 --
SELECT
    Order_ID,
    Sales,
    (SELECT AVG(Sales) FROM Dirty_Sales_Dataset) AS Overall_Avg_Sales
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 2 --
SELECT
    Order_ID,
    Sales,
    (SELECT AVG(Sales) FROM Dirty_Sales_Dataset) AS Avg_Sales,
    Sales - (SELECT AVG(Sales) FROM Dirty_Sales_Dataset) AS Difference_From_Avg
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 3 --
SELECT
    Order_ID,
    Sales,
    (Sales * 100.0 / (SELECT SUM(Sales) FROM Dirty_Sales_Dataset)) AS
Percent_Of_Total
FROM Dirty_Sales_Dataset
LIMIT 20;

-- Query 4 --
SELECT
    DISTINCT Customer_ID,
    (SELECT COUNT(*) FROM Dirty_Sales_Dataset d2 WHERE d2.Customer_ID =
d1.Customer_ID) AS Total_Orders
FROM Dirty_Sales_Dataset d1
LIMIT 20;

-- Query 5 --
SELECT AVG(Sales) FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Sales FROM Dirty_Sales_Dataset;

```

-- Query 7 --

```
SELECT
    Order_ID,
    Sales,
    (SELECT AVG(Sales) FROM Dirty_Sales_Dataset) AS Overall_Avg_Sales,
    Sales - (SELECT AVG(Sales) FROM Dirty_Sales_Dataset) AS Diff_From_Avg
FROM Dirty_Sales_Dataset
LIMIT 20;
```

-- C3.1.4 Correlated vs Non-Correlated subquery --

-- Query 1 --

```
SELECT Order_ID, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT AVG(Sales) FROM Dirty_Sales_Dataset);
```

-- Query 2 --

```
SELECT
    d1.Customer_ID,
    d1.Order_ID,
    d1.Sales,
    (SELECT COUNT(*) FROM Dirty_Sales_Dataset d2 WHERE d2.Customer_ID =
d1.Customer_ID) AS Total_Orders_By_This_Customer
FROM Dirty_Sales_Dataset d1
LIMIT 20;
```

-- Query 3 --

```
SELECT
    Order_ID,
    Customer_ID,
    Sales,
    (SELECT SUM(Sales) FROM Dirty_Sales_Dataset d2 WHERE d2.Customer_ID =
d1.Customer_ID) AS Customer_Total_Sales
FROM Dirty_Sales_Dataset d1
LIMIT 20;
```

-- Query 4 --

```
SELECT
    Order_ID,
    Customer_ID,
    Sales
FROM Dirty_Sales_Dataset d1
WHERE Sales = (
    SELECT MAX(Sales)
    FROM Dirty_Sales_Dataset d2
    WHERE d2.Customer_ID = d1.Customer_ID
);
```

-- Query 5 --

```
SELECT d1.Customer_ID, d1.Sales,
    (SELECT SUM(Sales) FROM Dirty_Sales_Dataset d2 WHERE d2.Customer_ID =
d1.Customer_ID) AS Total
FROM Dirty_Sales_Dataset d1;
```

```

-- Query 6 --
SELECT d1.Customer_ID, d1.Sales, summary.Total
FROM Dirty_Sales_Dataset d1
JOIN (
    SELECT Customer_ID, SUM(Sales) AS Total
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
) summary ON d1.Customer_ID = summary.Customer_ID;

-- Query 7 --
SELECT DISTINCT
    d1.Customer_ID,
    (SELECT SUM(Sales) FROM Dirty_Sales_Dataset d2 WHERE d2.Customer_ID =
d1.Customer_ID) AS Customer_Total_Sales
FROM Dirty_Sales_Dataset d1
LIMIT 20;

-- C3.2.1 EXISTS and NOT EXISTS – faster alternative to IN --

-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (SELECT Customer_ID FROM Salesperson_Info);

-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset d
WHERE EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson);

-- Query 3 --
SELECT d.Order_ID, d.Salesperson, d.Sales
FROM Dirty_Sales_Dataset d
WHERE EXISTS (
    SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson
)
LIMIT 20;

-- Query 4 --
SELECT d.Order_ID, d.Salesperson, d.Sales
FROM Dirty_Sales_Dataset d
WHERE NOT EXISTS (
    SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson
);

-- Query 5 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Salesperson NOT IN (SELECT Salesperson FROM Salesperson_Info);

-- Query 6 --
SELECT * FROM Dirty_Sales_Dataset d
WHERE NOT EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson =
d.Salesperson);

-- Query 7 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Salesperson IN (SELECT Salesperson FROM Salesperson_Info);

```

```
-- Query 8 --
SELECT * FROM Dirty_Sales_Dataset d
WHERE EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson);
```

```
-- Query 9 --
SELECT DISTINCT d.* FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```
-- Query 10 --
SELECT COUNT(*) AS Matched_Orders
FROM Dirty_Sales_Dataset d
WHERE EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson);
```

```
-- Query 11 --
SELECT DISTINCT d.Salesperson
FROM Dirty_Sales_Dataset d
WHERE NOT EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson =
d.Salesperson);
```

-- C3.2.2 IN vs EXISTS – performance difference explained --

```
-- Query 1 --
SELECT * FROM Dirty_Sales_Dataset
WHERE Salesperson IN (SELECT Salesperson FROM Salesperson_Info);
```

```
-- Query 2 --
SELECT * FROM Dirty_Sales_Dataset d
WHERE EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson);
```

```
-- Query 3 --
SELECT COUNT(*) FROM Dirty_Sales_Dataset
WHERE Salesperson IN (SELECT Salesperson FROM Salesperson_Info);
```

```
-- Query 4 --
SELECT COUNT(*) FROM Dirty_Sales_Dataset d
WHERE EXISTS (SELECT 1 FROM Salesperson_Info s WHERE s.Salesperson = d.Salesperson);
```

-- C3.2.3 ANY and ALL operators with subqueries --

```
-- Query 1 --
SELECT Order_ID, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > ANY (
    SELECT Sales FROM Dirty_Sales_Dataset WHERE Region = 'North'
);
```

```
-- Query 2 --
SELECT Order_ID, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > ALL (
    SELECT Sales FROM Dirty_Sales_Dataset WHERE Region = 'North'
);
```

```
-- Query 3 --
SELECT Order_ID, Region, Sales
```

```

FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT MIN(Sales) FROM Dirty_Sales_Dataset WHERE Region = 'North')
LIMIT 20;

```

-- Query 4 --

```

SELECT Order_ID, Region, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT MAX(Sales) FROM Dirty_Sales_Dataset WHERE Region = 'North')
LIMIT 20;

```

-- Query 5 --

```

WHERE Salesperson = ANY (SELECT Salesperson FROM Salesperson_Info)

```

-- Query 6 --

```

WHERE Salesperson IN (SELECT Salesperson FROM Salesperson_Info)

```

-- Query 7 --

```

SELECT Order_ID, Region, Sales
FROM Dirty_Sales_Dataset
WHERE Sales > (SELECT MIN(Sales) FROM Dirty_Sales_Dataset WHERE Region = 'North')
LIMIT 20;

```

-- C3.2.4 Nested subqueries – 3 levels deep example --

-- Query 1 --

```

SELECT AVG(Customer_Total) AS Region_Avg_Customer_Spending
FROM (
    SELECT Customer_ID, SUM(Sales) AS Customer_Total
    FROM Dirty_Sales_Dataset
    WHERE Order_Status = 'Delivered'
    GROUP BY Customer_ID
)

```

-- Query 2 --

```

SELECT Customer_ID, SUM(Sales) AS Customer_Total
FROM Dirty_Sales_Dataset
WHERE Order_Status = 'Delivered'
GROUP BY Customer_ID

```

-- Query 3 --

```

SELECT Order_ID, Customer_ID, Sales
FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM (
        SELECT Customer_ID, SUM(Sales) AS Customer_Total
        FROM Dirty_Sales_Dataset
        WHERE Order_Status = 'Delivered'
        GROUP BY Customer_ID
    ) AS customer_totals
    WHERE Customer_Total > (
        SELECT AVG(Customer_Total)
        FROM (
            SELECT Customer_ID, SUM(Sales) AS Customer_Total
            FROM Dirty_Sales_Dataset

```

```

        WHERE Order_Status = 'Delivered'
        GROUP BY Customer_ID
    )
)
);

-- Query 4 --
SELECT Product_ID, Product_Name, SUM(Sales) AS Product_Sales
FROM Dirty_Sales_Dataset
WHERE Category IN (
    SELECT Category
    FROM (
        SELECT Category, SUM(Sales) AS Category_Sales
        FROM Dirty_Sales_Dataset
        GROUP BY Category
    ) AS category_totals
    WHERE Category_Sales > (
        SELECT AVG(Category_Sales) FROM (
            SELECT Category, SUM(Sales) AS Category_Sales
            FROM Dirty_Sales_Dataset
            GROUP BY Category
        )
    )
)
GROUP BY Product_ID, Product_Name
ORDER BY Product_Sales DESC
LIMIT 20;

-- Query 5 --
WITH customer_totals AS (
    SELECT Customer_ID, SUM(Sales) AS Customer_Total
    FROM Dirty_Sales_Dataset
    WHERE Order_Status = 'Delivered'
    GROUP BY Customer_ID
),
avg_spending AS (
    SELECT AVG(Customer_Total) AS Overall_Avg FROM customer_totals
)
SELECT d.Order_ID, d.Customer_ID, d.Sales
FROM Dirty_Sales_Dataset d
WHERE d.Customer_ID IN (
    SELECT Customer_ID FROM customer_totals, avg_spending
    WHERE Customer_Total > Overall_Avg
);

-- Query 6 --
SELECT Category, Category_Sales
FROM (
    SELECT Category, SUM(Sales) AS Category_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Category
) AS category_totals
WHERE Category_Sales > (
    SELECT AVG(Category_Sales) FROM (
        SELECT Category, SUM(Sales) AS Category_Sales

```

```

        FROM Dirty_Sales_Dataset
        GROUP BY Category
    )
);

-- C4.1.1 UNION – combine results, remove duplicates --

-- Query 1 --
SELECT Customer_ID, 'High Value' AS Category
FROM Dirty_Sales_Dataset
WHERE Sales > 50000

UNION

SELECT Customer_ID, 'Needs Attention' AS Category
FROM Dirty_Sales_Dataset
WHERE Sales < 0;

-- Query 2 --
SELECT DISTINCT Customer_ID, Customer_Name
FROM Dirty_Sales_Dataset
WHERE Region = 'North'

UNION

SELECT DISTINCT Customer_ID, Customer_Name
FROM Dirty_Sales_Dataset
WHERE Region = 'South';

-- Query 3 --
SELECT Salesperson FROM Dirty_Sales_Dataset

UNION

SELECT Salesperson FROM Salesperson_Info;

-- Query 4 --
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset
UNION
SELECT Customer_ID FROM Dirty_Sales_Dataset;

-- Query 5 --
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset
UNION
SELECT Customer_ID, Sales FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT Order_ID, Sales, 'Negative Sales' AS Issue
FROM Dirty_Sales_Dataset
WHERE Sales < 0

UNION

SELECT Order_ID, Sales, 'Negative Quantity' AS Issue
FROM Dirty_Sales_Dataset

```

```
WHERE Quantity < 0
```

```
UNION
```

```
SELECT Order_ID, Sales, 'Invalid Discount' AS Issue
FROM Dirty_Sales_Dataset
WHERE Discount < 0 OR Discount > 1;
```

```
-- Query 7 --
```

```
SELECT Order_ID, Sales, 'Negative Sales' AS Issue
FROM Dirty_Sales_Dataset
WHERE Sales < 0
```

```
UNION
```

```
SELECT Order_ID, Sales, 'Invalid Discount' AS Issue
FROM Dirty_Sales_Dataset
WHERE Discount < 0 OR Discount > 1
LIMIT 30;
```

```
-- C4.1.2 UNION ALL – combine results, keep duplicates (faster) --
```

```
-- Query 1 --
```

```
SELECT Order_ID, Sales, 'North Region' AS Source
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
```

```
UNION ALL
```

```
SELECT Order_ID, Sales, 'High Value' AS Source
FROM Dirty_Sales_Dataset
WHERE Sales > 50000;
```

```
-- Query 2 --
```

```
SELECT Order_ID, Sales, 'Negative Sales' AS Issue
FROM Dirty_Sales_Dataset
WHERE Sales < 0
```

```
UNION ALL
```

```
SELECT Order_ID, Sales, 'Invalid Discount' AS Issue
FROM Dirty_Sales_Dataset
WHERE Discount < 0 OR Discount > 1;
```

```
-- Query 3 --
```

```
SELECT Salesperson FROM Dirty_Sales_Dataset
UNION
SELECT Salesperson FROM Salesperson_Info;
```

```
-- Query 4 --
```

```
SELECT Salesperson FROM Dirty_Sales_Dataset
UNION ALL
SELECT Salesperson FROM Salesperson_Info;
```

```
-- Query 5 --
```

```

SELECT COUNT(*) FROM (
    SELECT Salesperson FROM Dirty_Sales_Dataset
    UNION
    SELECT Salesperson FROM Salesperson_Info
) AS union_result;

```

```

SELECT COUNT(*) FROM (
    SELECT Salesperson FROM Dirty_Sales_Dataset
    UNION ALL
    SELECT Salesperson FROM Salesperson_Info
) AS union_all_result;

```

```

-- Query 6 --
SELECT COUNT(*) AS Union_Count FROM (
    SELECT Salesperson FROM Dirty_Sales_Dataset
    UNION
    SELECT Salesperson FROM Salesperson_Info
);

```

```

SELECT COUNT(*) AS Union_All_Count FROM (
    SELECT Salesperson FROM Dirty_Sales_Dataset
    UNION ALL
    SELECT Salesperson FROM Salesperson_Info
);

```

-- C4.1.3 UNION vs JOIN – they are completely different things --

```

-- Query 1 --
SELECT Order_ID, 'North' AS Region_Type
FROM Dirty_Sales_Dataset
WHERE Region = 'North'

```

UNION

```

SELECT Order_ID, 'South' AS Region_Type
FROM Dirty_Sales_Dataset
WHERE Region = 'South';

```

```

-- Query 2 --
SELECT d.Order_ID, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;

```

```

-- Query 3 --
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset
UNION
SELECT Customer_ID FROM Dirty_Sales_Dataset;

```

```

-- Query 4 --
SELECT d.Order_ID, d.Sales, s.Region_Assigned, s.Years_Experience
FROM Dirty_Sales_Dataset d
JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;

```

```

-- Query 5 --
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset WHERE Region = 'North'

```

```

UNION
SELECT Order_ID, Sales FROM Dirty_Sales_Dataset WHERE Region = 'South'
LIMIT 20;

-- Query 6 --
SELECT d.Order_ID, d.Sales, s.Region_Assigned
FROM Dirty_Sales_Dataset d
JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
LIMIT 20;

-- C4.1.4 Stacking monthly/yearly reports using UNION ALL --

-- Query 1 --
SELECT 'January' AS Month, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE strftime('%m', Order_Date) = '01'

UNION ALL

SELECT 'February' AS Month, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE strftime('%m', Order_Date) = '02'

UNION ALL

SELECT 'March' AS Month, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE strftime('%m', Order_Date) = '03';

-- Query 2 --
SELECT '2023' AS Year, SUM(Sales) AS Total_Sales, COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2023'

UNION ALL

SELECT '2024' AS Year, SUM(Sales) AS Total_Sales, COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2024'

UNION ALL

SELECT '2025' AS Year, SUM(Sales) AS Total_Sales, COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2025';

-- Query 3 --
SELECT strftime('%Y', Order_Date) AS Year, SUM(Sales) AS Total_Sales, COUNT(*) AS
Total_Orders
FROM Dirty_Sales_Dataset
GROUP BY Year
ORDER BY Year;

-- Query 4 --
SELECT 'Current Month' AS Period, SUM(Sales) AS Total_Sales

```

```
FROM Current_Month_Sales_Table
```

```
UNION ALL
```

```
SELECT 'Archived Data' AS Period, SUM(Sales) AS Total_Sales  
FROM Historical_Sales_Archive_Table;
```

```
-- Query 5 --
```

```
SELECT 'India Sales' AS Source, SUM(Sales) AS Total  
FROM India_Sales_Table
```

```
UNION ALL
```

```
SELECT 'US Sales' AS Source, SUM(Sales) AS Total  
FROM US_Sales_Table
```

```
UNION ALL
```

```
SELECT 'Europe Sales' AS Source, SUM(Sales) AS Total  
FROM Europe_Sales_Table;
```

```
-- Query 6 --
```

```
SELECT 'North' AS Region, SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset WHERE  
Region = 'North'
```

```
UNION ALL
```

```
SELECT 'South' AS Region, SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset WHERE  
Region = 'South'
```

```
UNION ALL
```

```
SELECT 'East' AS Region, SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset WHERE  
Region = 'East'
```

```
UNION ALL
```

```
SELECT 'West' AS Region, SUM(Sales) AS Total_Sales FROM Dirty_Sales_Dataset WHERE  
Region = 'West';
```

```
-- Query 7 --
```

```
SELECT Region, SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Region;
```

```
-- C4.2.1 INTERSECT – rows common in both queries --
```

```
-- Query 1 --
```

```
SELECT Salesperson FROM Dirty_Sales_Dataset  
INTERSECT  
SELECT Salesperson FROM Salesperson_Info;
```

```
-- Query 2 --
```

```
SELECT Salesperson FROM Dirty_Sales_Dataset  
INTERSECT  
SELECT Salesperson FROM Salesperson_Info;
```

```
-- Query 3 --
```

```
SELECT d.Salesperson, d.Sales, s.Region_Assigned  
FROM Dirty_Sales_Dataset d  
INNER JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson;
```

```

-- Query 4 --
SELECT DISTINCT Category FROM Dirty_Sales_Dataset
INTERSECT
SELECT DISTINCT Category FROM Salesperson_Info;

-- Query 5 --
SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date)
= '2023'
INTERSECT
SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date)
= '2024';

-- Query 6 --
SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2023'

INTERSECT

SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2024';

-- C4.2.2 EXCEPT / MINUS – rows in first query but not second --

-- Query 1 --
SELECT Name FROM Query1
EXCEPT
SELECT Name FROM Query2;

-- Query 2 --
SELECT Name FROM Query2
EXCEPT
SELECT Name FROM Query1;

-- Query 3 --
SELECT Salesperson FROM Dirty_Sales_Dataset
EXCEPT
SELECT Salesperson FROM Salesperson_Info;

-- Query 4 --
SELECT DISTINCT d.Salesperson
FROM Dirty_Sales_Dataset d
LEFT JOIN Salesperson_Info s ON d.Salesperson = s.Salesperson
WHERE s.Salesperson IS NULL;

-- Query 5 --
SELECT Salesperson FROM Salesperson_Info
EXCEPT
SELECT Salesperson FROM Dirty_Sales_Dataset;

-- Query 6 --
SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date)
= '2023'

```

```
EXCEPT
SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date)
= '2024';
```

```
-- Query 7 --
```

```
SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2023'
```

```
EXCEPT
```

```
SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2024';
```

```
-- C4.2.3 Real use case – find customers who bought in 2023 but not 2024 --
```

```
-- Query 1 --
```

```
SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2023'
```

```
EXCEPT
```

```
SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2024';
```

```
-- Query 2 --
```

```
SELECT DISTINCT c2023.Customer_ID
FROM (SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2023') c2023
LEFT JOIN (SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2024') c2024
ON c2023.Customer_ID = c2024.Customer_ID
WHERE c2024.Customer_ID IS NULL;
```

```
-- Query 3 --
```

```
SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2023'
AND Customer_ID NOT IN (
    SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date) =
'2024'
);
```

```
-- Query 4 --
```

```
SELECT COUNT(*) AS Churned_Customers_Count
FROM (
    SELECT DISTINCT Customer_ID
    FROM Dirty_Sales_Dataset
    WHERE strftime('%Y', Order_Date) = '2023'
```

```
EXCEPT
```

```

SELECT DISTINCT Customer_ID
FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2024'
);

-- Query 5 --
SELECT
    (SELECT COUNT(DISTINCT Customer_ID) FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2023') AS Total_2023_Customers,
    (SELECT COUNT(*) FROM (
        SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2023'
        EXCEPT
        SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2024'
    )) AS Churned_Customers,
    ROUND(
        (SELECT COUNT(*) FROM (
            SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2023'
            EXCEPT
            SELECT DISTINCT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y',
Order_Date) = '2024'
        )) * 100.0 / (SELECT COUNT(DISTINCT Customer_ID) FROM Dirty_Sales_Dataset
WHERE strftime('%Y', Order_Date) = '2023')
        , 2) AS Churn_Rate_Percent;

-- Query 6 --
SELECT
    d.Customer_ID,
    d.Customer_Name,
    SUM(d.Sales) AS Total_2023_Sales,
    COUNT(*) AS Total_2023_Orders
FROM Dirty_Sales_Dataset d
WHERE strftime('%Y', d.Order_Date) = '2023'
AND d.Customer_ID IN (
    SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date) =
'2023'
    EXCEPT
    SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date) =
'2024'
)
GROUP BY d.Customer_ID, d.Customer_Name
ORDER BY Total_2023_Sales DESC;

-- Query 7 --
SELECT
    d.Customer_ID,
    d.Customer_Name,
    SUM(d.Sales) AS Total_2023_Sales
FROM Dirty_Sales_Dataset d
WHERE strftime('%Y', d.Order_Date) = '2023'
AND d.Customer_ID IN (
    SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date) =
'2023'

```

```

    EXCEPT
    SELECT Customer_ID FROM Dirty_Sales_Dataset WHERE strftime('%Y', Order_Date) =
'2024'
)
GROUP BY d.Customer_ID, d.Customer_Name
ORDER BY Total_2023_Sales DESC
LIMIT 20;

```

-- D1.1.1 What is a Window Function – OVER() clause explained --

-- Query 1 --

```

SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Region,
    Sales,
    SUM(Sales) OVER (PARTITION BY Region) AS Region_Total_Sales
FROM Dirty_Sales_Dataset;

```

-- Query 3 --

```

SELECT
    Order_ID,
    Region,
    Sales,
    SUM(Sales) OVER (PARTITION BY Region) AS Total_Sales_In_This_Region
FROM Dirty_Sales_Dataset
LIMIT 20;

```

-- Query 4 --

```

SELECT
    Order_ID,
    Sales,
    SUM(Sales) OVER () AS Grand_Total_Sales
FROM Dirty_Sales_Dataset
LIMIT 20;

```

-- Query 5 --

```

SELECT
    Order_ID,
    Region,
    Sales,
    SUM(Sales) OVER (PARTITION BY Region) AS Region_Total_Sales,
    COUNT(*) OVER (PARTITION BY Region) AS Region_Order_Count
FROM Dirty_Sales_Dataset
LIMIT 30;

```

-- D1.1.2 Difference between Window Function and GROUP BY --

-- Query 1 --

```

SELECT Region, SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset

```

```
GROUP BY Region;
```

```
-- Query 2 --
```

```
SELECT Order_ID, Region, Sales, SUM(Sales) OVER (PARTITION BY Region) AS Region_Total  
FROM Dirty_Sales_Dataset;
```

```
-- D1.1.3 PARTITION BY – creating groups without collapsing rows --
```

```
-- Query 1 --
```

```
SELECT  
    Order_ID,  
    Region,  
    Sales,  
    AVG(Sales) OVER (PARTITION BY Region) AS Region_Avg_Sales  
FROM Dirty_Sales_Dataset  
LIMIT 20;
```

```
-- Query 2 --
```

```
SELECT  
    Order_ID,  
    Category,  
    Sales,  
    COUNT(*) OVER (PARTITION BY Category) AS Orders_In_This_Category  
FROM Dirty_Sales_Dataset  
LIMIT 20;
```

```
-- Query 3 --
```

```
SELECT  
    Order_ID,  
    Region,  
    Category,  
    Sales,  
    SUM(Sales) OVER (PARTITION BY Region, Category) AS Region_Category_Total  
FROM Dirty_Sales_Dataset  
LIMIT 20;
```

```
-- Query 4 --
```

```
SELECT  
    Order_ID,  
    Sales,  
    AVG(Sales) OVER () AS Overall_Avg_Sales  
FROM Dirty_Sales_Dataset  
LIMIT 20;
```

```
-- Query 5 --
```

```
SELECT  
    Order_ID,  
    Category,  
    Sales,  
    AVG(Sales) OVER (PARTITION BY Category) AS Category_Avg,  
    Sales - AVG(Sales) OVER (PARTITION BY Category) AS Diff_From_Category_Avg  
FROM Dirty_Sales_Dataset  
LIMIT 20;
```

```
-- Query 6 --
```

```
SELECT Region, AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 7 --

```
SELECT Order_ID, Region, Sales, AVG(Sales) OVER (PARTITION BY Region) AS Avg_Sales
FROM Dirty_Sales_Dataset;
```

-- Query 8 --

```
SELECT
    Order_ID,
    Category,
    Sales,
    ROUND(AVG(Sales) OVER (PARTITION BY Category), 2) AS Category_Avg,
    ROUND(Sales - AVG(Sales) OVER (PARTITION BY Category), 2) AS Diff_From_Avg
FROM Dirty_Sales_Dataset
LIMIT 30;
```

-- D1.1.4 ORDER BY inside OVER() – row ordering within window --

-- Query 1 --

```
SELECT Order_ID, Region, Sales, SUM(Sales) OVER (PARTITION BY Region) AS Region_Total
FROM Dirty_Sales_Dataset
LIMIT 10;
```

-- Query 2 --

```
SELECT Order_ID, Region, Order_Date, Sales,
    SUM(Sales) OVER (PARTITION BY Region ORDER BY Order_Date) AS Running_Total
FROM Dirty_Sales_Dataset
LIMIT 10;
```

-- Query 3 --

```
SELECT
    Order_ID,
    Region,
    Order_Date,
    Sales,
    SUM(Sales) OVER (PARTITION BY Region ORDER BY Order_Date) AS Running_Sales_Total
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 20;
```

-- Query 4 --

```
SELECT
    Order_ID,
    Customer_ID,
    Sales,
    ROW_NUMBER() OVER (PARTITION BY Customer_ID ORDER BY Sales DESC) AS
Rank_Within_Customer
FROM Dirty_Sales_Dataset
LIMIT 20;
```

-- Query 5 --

```
SELECT
```

```

    Order_ID,
    Region,
    Order_Date,
    Sales,
    SUM(Sales) OVER (PARTITION BY Region ORDER BY Order_Date) AS Running_Total
FROM Dirty_Sales_Dataset
ORDER BY Region, Order_Date;

```

-- Query 6 --

```

SELECT
    Order_ID,
    Region,
    Order_Date,
    Sales,
    SUM(Sales) OVER (PARTITION BY Region ORDER BY Order_Date) AS Running_Total
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 30;

```

-- D1.1.5 ROWS vs RANGE framing – the most misunderstood concept --

-- Query 1 --

```

SUM(Sales) OVER (ORDER BY Sales ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

```

-- Query 2 --

```

SUM(Sales) OVER (ORDER BY Sales RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

```

-- Query 3 --

```

SELECT Order_Date, COUNT(*) AS Orders_On_This_Date
FROM Dirty_Sales_Dataset
GROUP BY Order_Date
HAVING COUNT(*) > 1
LIMIT 5;

```

-- Query 4 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    SUM(Sales) OVER (ORDER BY Order_Date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS Running_Total_ROWS,
    SUM(Sales) OVER (ORDER BY Order_Date RANGE BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS Running_Total_RANGE
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 30;

```

-- Query 5 --

```

SUM(Sales) OVER (PARTITION BY Region ORDER BY Order_Date)

```

-- Query 6 --

```

SUM(Sales) OVER (
    PARTITION BY Region

```

```

        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    )

-- Query 7 --
SELECT Order_Date, COUNT(*) AS Orders_On_This_Date
FROM Dirty_Sales_Dataset
GROUP BY Order_Date
HAVING COUNT(*) > 1
LIMIT 10;

-- D1.2.1 ROW_NUMBER() – unique sequential number per row --

-- Query 1 --
SELECT
    Order_ID,
    Customer_ID,
    Sales,
    ROW_NUMBER() OVER (ORDER BY Sales DESC) AS Row_Num
FROM Dirty_Sales_Dataset
LIMIT 10;

-- Query 2 --
SELECT
    Order_ID,
    Region,
    Sales,
    ROW_NUMBER() OVER (PARTITION BY Region ORDER BY Sales DESC) AS Rank_In_Region
FROM Dirty_Sales_Dataset
LIMIT 30;

-- Query 3 --
SELECT * FROM (
    SELECT
        Order_ID,
        Region,
        Sales,
        ROW_NUMBER() OVER (PARTITION BY Region ORDER BY Sales DESC) AS Rank_In_Region
    FROM Dirty_Sales_Dataset
) AS ranked_orders
WHERE Rank_In_Region <= 3;

-- Query 4 --
SELECT * FROM (
    SELECT
        Order_ID,
        Customer_ID,
        Sales,
        ROW_NUMBER() OVER (PARTITION BY Customer_ID ORDER BY Sales DESC) AS Rank_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Rank_Num = 1;

-- Query 5 --
SELECT * FROM (

```

```

    SELECT *, ROW_NUMBER() OVER (PARTITION BY Order_ID ORDER BY Order_Date) AS
Row_Num
    FROM Dirty_Sales_Dataset
) AS numbered
WHERE Row_Num = 1;

```

-- Query 6 --

```

SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Order_ID ORDER BY Order_Date) AS
Row_Num
    FROM Dirty_Sales_Dataset
) AS numbered
WHERE Row_Num > 1
LIMIT 20;

```

D1.2.2 RANK() – same rank for ties, gaps after ties

-- Query 1 --

```

SELECT
    Order_ID,
    Customer_ID,
    Sales,
    RANK() OVER (ORDER BY Sales DESC) AS Sales_Rank
FROM Dirty_Sales_Dataset
LIMIT 10;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Region,
    Sales,
    RANK() OVER (PARTITION BY Region ORDER BY Sales DESC) AS Sales_Rank
FROM Dirty_Sales_Dataset
LIMIT 30;

```

-- Query 3 --

```

SELECT * FROM (
    SELECT
        Order_ID,
        Region,
        Sales,
        RANK() OVER (PARTITION BY Region ORDER BY Sales DESC) AS Sales_Rank
    FROM Dirty_Sales_Dataset
) AS ranked_orders
WHERE Sales_Rank <= 3;

```

-- Query 4 --

```

SELECT Sales, COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Sales
HAVING COUNT(*) > 1
ORDER BY Frequency DESC
LIMIT 10;

```

-- Query 5 --

```

SELECT * FROM (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Spending,
        RANK() OVER (ORDER BY SUM(Sales) DESC) AS Spending_Rank
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
) AS ranked_customers
WHERE Spending_Rank <= 5;

```

-- Query 6 --

```

SELECT
    Order_ID,
    Region,
    Sales,
    ROW_NUMBER() OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
    ) AS Row_Num,
    RANK() OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
    ) AS Rank_Num
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Sales DESC
LIMIT 20;

```

D1.2.3 DENSE_RANK() – same rank for ties, NO gaps

-- Query 1 --

```

SELECT
    Order_ID,
    Customer_ID,
    Sales,
    DENSE_RANK() OVER (ORDER BY Sales DESC) AS Sales_Dense_Rank
FROM Dirty_Sales_Dataset
LIMIT 10;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Region,
    Sales,
    RANK() OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
    ) AS Rank_Num,
    DENSE_RANK() OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
    ) AS Dense_Rank_Num
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Sales DESC

```

```
LIMIT 20;
```

```
-- Query 3 --
```

```
SELECT * FROM (  
    SELECT  
        Order_ID,  
        Region,  
        Sales,  
        DENSE_RANK() OVER (  
            PARTITION BY Region  
            ORDER BY Sales DESC  
        ) AS Sales_Tier  
    FROM Dirty_Sales_Dataset  
    ) AS ranked  
WHERE Sales_Tier <= 3;
```

```
-- Query 4 --
```

```
SELECT  
    Customer_ID,  
    SUM(Sales) AS Total_Spending,  
    DENSE_RANK() OVER (  
        ORDER BY SUM(Sales) DESC  
    ) AS Spending_Tier  
FROM Dirty_Sales_Dataset  
GROUP BY Customer_ID  
LIMIT 20;
```

```
-- Query 5 --
```

```
SELECT  
    Order_ID,  
    Region,  
    Quantity,  
    RANK() OVER (  
        PARTITION BY Region  
        ORDER BY Quantity DESC  
    ) AS Rank_Num,  
    DENSE_RANK() OVER (  
        PARTITION BY Region  
        ORDER BY Quantity DESC  
    ) AS Dense_Rank_Num  
FROM Dirty_Sales_Dataset  
WHERE Region = 'North'  
ORDER BY Quantity DESC  
LIMIT 30;
```

D1.2.4 NTILE(n) – divide rows into N equal buckets

```
-- Query 1 --
```

```
SELECT  
    Order_ID,  
    Sales,  
    NTILE(4) OVER (ORDER BY Sales DESC) AS Sales_Quartile  
FROM Dirty_Sales_Dataset  
LIMIT 30;
```

-- Query 2 --

```
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Spending,
    NTILE(10) OVER (
        ORDER BY SUM(Sales) DESC
    ) AS Spending_Decile
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 3 --

```
SELECT * FROM (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Spending,
        NTILE(4) OVER (
            ORDER BY SUM(Sales) DESC
        ) AS Quartile
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
) AS ranked_customers
WHERE Quartile = 1;
```

-- Query 4 --

```
SELECT
    Order_ID,
    Region,
    Sales,
    NTILE(3) OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
    ) AS Sales_Tier
FROM Dirty_Sales_Dataset
LIMIT 30;
```

-- Query 5 --

```
SELECT * FROM (
    SELECT
        Order_ID,
        Sales,
        NTILE(4) OVER (ORDER BY Sales DESC) AS Quartile_Num
    FROM Dirty_Sales_Dataset
) AS segmented
SELECT
    Order_ID,
    Sales,
    Quartile_Num,
    CASE Quartile_Num
        WHEN 1 THEN 'Top 25%'
        WHEN 2 THEN 'Upper Mid 25%'
        WHEN 3 THEN 'Lower Mid 25%'
        WHEN 4 THEN 'Bottom 25%'
    END AS Sales_Segment
FROM segmented
LIMIT 30;
```

```
-- Query 6 --
SELECT
    NTILE(4) OVER (ORDER BY Sales DESC) AS Quartile
FROM Dirty_Sales_Dataset;
```

```
-- Query 7 --
SELECT
    Order_ID,
    Category,
    Sales,
    NTILE(4) OVER (
        PARTITION BY Category
        ORDER BY Sales DESC
    ) AS Sales_Quartile
FROM Dirty_Sales_Dataset
WHERE Category = 'Electronics'
ORDER BY Sales DESC
LIMIT 40;
```

D1.2.5 Top N per Group using ROW_NUMBER()

```
-- Query 1 --
SELECT * FROM (
    SELECT
        Order_ID,
        Region,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Region
            ORDER BY Sales DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num <= 3;
```

```
-- Query 2 --
SELECT * FROM (
    SELECT
        Order_ID,
        Customer_ID,
        Order_Date,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num = 1;
```

```
-- Query 3 --
SELECT * FROM (
    SELECT
        Product_ID,
```

```

        Product_Name,
        Category,
        SUM(Sales) AS Total_Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Category
            ORDER BY SUM(Sales) DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
    GROUP BY Product_ID, Product_Name, Category
) AS ranked
WHERE Row_Num <= 5;

```

```

-- Query 4 --
SELECT * FROM (
    SELECT
        Order_ID,
        Salesperson,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Salesperson
            ORDER BY Sales DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num = 1;

```

```

-- Query 5 --
WITH Ranked_Orders AS (
    SELECT
        Order_ID,
        Region,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Region
            ORDER BY Sales DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
)
SELECT *
FROM Ranked_Orders
WHERE Row_Num <= 3;

```

```

-- Query 6 --
SELECT * FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY Order_ID
            ORDER BY Order_Date
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS numbered
WHERE Row_Num > 1;

```

```

-- Query 7 --

```

```

SELECT * FROM (
    SELECT
        Order_ID,
        Category,
        Product_Name,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Category
            ORDER BY Sales DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num <= 3
ORDER BY Category, Row_Num;

```

D1.3.1 SUM() OVER() – running total / cumulative sum

-- Query 1 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    SUM(Sales) OVER (
        ORDER BY Order_Date
    ) AS Running_Total
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Region,
    Order_Date,
    Sales,
    SUM(Sales) OVER (
        PARTITION BY Region
        ORDER BY Order_Date
    ) AS Region_Running_Total
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 30;

```

-- Query 3 --

```

SELECT
    strftime('%Y-%m', Order_Date) AS Order_Month,
    SUM(Sales) AS Monthly_Sales,
    SUM(SUM(Sales)) OVER (
        ORDER BY strftime('%Y-%m', Order_Date)
    ) AS Cumulative_Sales
FROM Dirty_Sales_Dataset
GROUP BY Order_Month
ORDER BY Order_Month;

```

-- Query 4 --

```
SELECT
    Customer_ID,
    Order_Date,
    Sales,
    SUM(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Customer_Running_Spend
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;
```

-- Query 5 --

```
SELECT
    Order_ID,
    Sales,
    SUM(Sales) OVER () AS Grand_Total,
    SUM(Sales) OVER (
        ORDER BY Order_Date
    ) AS Running_Total
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 10;
```

-- Query 6 --

```
SELECT
    Order_ID,
    Order_Date,
    Sales,
    SUM(Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Precise_Running_Total
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;
```

-- Query 7 --

```
SELECT
    Order_ID,
    Order_Date,
    Sales,
    SUM(Sales) OVER (
        ORDER BY Order_Date, Order_ID
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Precise_Running_Total
FROM Dirty_Sales_Dataset
ORDER BY Order_Date, Order_ID
LIMIT 20;
```

-- Query 8 --

```
SELECT
    Order_ID,
    Order_Date,
```

```

Sales,
SUM(Sales) OVER (
    ORDER BY Order_Date
) AS Running_Total,
SUM(Sales) OVER () AS Grand_Total
FROM Dirty_Sales_Dataset
WHERE Region = 'South'
ORDER BY Order_Date
LIMIT 40;

```

D1.3.2 AVG() OVER() – moving average

-- Query 1 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    AVG(Sales) OVER (
        ORDER BY Order_Date
    ) AS Running_Avg
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;

```

-- Query 2 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    AVG(Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS Moving_Avg_3
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;

```

-- Query 3 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    AVG(Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS Moving_Avg_7Day
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 40;

```

-- Query 4 --

```

SELECT
    Order_ID,
    Region,
    Order_Date,

```

```

Sales,
AVG(Sales) OVER (
    PARTITION BY Region
    ORDER BY Order_Date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS Region_Moving_Avg_3
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 30;

```

-- Query 5 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    AVG(Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS Centered_Moving_Avg_3
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;

```

-- Query 6 --

```

SELECT
    Order_Month,
    Monthly_Sales,
    AVG(Monthly_Sales) OVER (
        ORDER BY Order_Month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS Moving_Avg_3Month
FROM (
    SELECT
        strftime('%Y-%m', Order_Date) AS Order_Month,
        SUM(Sales) AS Monthly_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Order_Month
) AS monthly_data
ORDER BY Order_Month;

```

-- Query 7 --

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    AVG(Sales) OVER (
        ORDER BY Order_Date
    ) AS Running_Avg,
    AVG(Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
    ) AS Moving_Avg_5
FROM Dirty_Sales_Dataset
WHERE Region = 'East'

```

```
ORDER BY Order_Date
LIMIT 40;
```

D1.3.3 COUNT() OVER() – running count

-- Query 1 --

```
SELECT
    Order_ID,
    Order_Date,
    COUNT(*) OVER (
        ORDER BY Order_Date
    ) AS Running_Order_Count
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;
```

-- Query 2 --

```
SELECT
    Order_ID,
    Region,
    Order_Date,
    COUNT(*) OVER (
        PARTITION BY Region
        ORDER BY Order_Date
    ) AS Region_Running_Count
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 30;
```

-- Query 3 --

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    COUNT(*) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Order_Sequence_Number
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;
```

-- Query 4 --

```
SELECT
    Order_ID,
    Order_Date,
    COUNT(*) OVER () AS Total_Rows,
    COUNT(*) OVER (
        ORDER BY Order_Date
    ) AS Running_Count
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;
```

-- Query 5 --

```
SELECT
    Order_ID,
    Order_Date,
    Customer_Rating,
    COUNT(Customer_Rating) OVER (
        ORDER BY Order_Date
    ) AS Running_Rated_Count,
    COUNT(*) OVER (
        ORDER BY Order_Date
    ) AS Running_Total_Count
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;
```

-- Query 6 --

```
SELECT
    Order_ID,
    Order_Date,
    ROW_NUMBER() OVER (
        ORDER BY Order_Date
    ) AS Row_Num,
    COUNT(*) OVER (
        ORDER BY Order_Date
    ) AS Running_Count
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;
```

-- Query 7 --

```
SELECT
    Order_ID,
    Order_Date,
    COUNT(*) OVER (
        ORDER BY Order_Date, Order_ID
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Precise_Running_Count
FROM Dirty_Sales_Dataset
ORDER BY Order_Date, Order_ID
LIMIT 20;
```

-- Query 8 --

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    COUNT(*) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Customer_Order_Number
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 40;
```

D1.3.4 MIN() / MAX() OVER() – Running Min/Max

```
-- Query 1 --  
-- Simple running max/min (date-wise, poore dataset ka)
```

```
SELECT  
    Order_ID,  
    Order_Date,  
    Sales,  
    MAX(Sales) OVER (ORDER BY Order_Date) AS Running_Max_Sales,  
    MIN(Sales) OVER (ORDER BY Order_Date) AS Running_Min_Sales  
FROM Dirty_Sales_Dataset  
ORDER BY Order_Date  
LIMIT 30;
```

```
-- Query 2 --  
-- Region-wise running max (highest sales ab tak, us region mein)
```

```
SELECT  
    Order_ID,  
    Region,  
    Order_Date,  
    Sales,  
    MAX(Sales) OVER (  
        PARTITION BY Region  
        ORDER BY Order_Date  
    ) AS Region_Running_Max  
FROM Dirty_Sales_Dataset  
WHERE Region = 'North'  
ORDER BY Order_Date  
LIMIT 30;
```

```
-- Query 3 --  
-- Customer ka sabse bada order ab tak (spending peak track karna)
```

```
SELECT  
    Order_ID,  
    Customer_ID,  
    Order_Date,  
    Sales,  
    MAX(Sales) OVER (  
        PARTITION BY Customer_ID  
        ORDER BY Order_Date  
    ) AS Customer_Best_Order_So_Far  
FROM Dirty_Sales_Dataset  
ORDER BY Customer_ID, Order_Date  
LIMIT 30;
```

```
-- Query 4 --  
-- Grand Max/Min vs Running Max/Min
```

```
SELECT  
    Order_ID,
```

```

    Order_Date,
    Sales,
    MAX(Sales) OVER () AS Overall_Max,
    MAX(Sales) OVER (ORDER BY Order_Date) AS Running_Max,
    MIN(Sales) OVER () AS Overall_Min,
    MIN(Sales) OVER (ORDER BY Order_Date) AS Running_Min
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;

```

```

-- Query 5 --
-- Naya record detect karna (jab peak break ho)

```

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    MAX(Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    ) AS Previous_Max,
    CASE
        WHEN Sales > MAX(Sales) OVER (
            ORDER BY Order_Date
            ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
        )
        THEN 'New Record!'
        ELSE 'Normal'
    END AS Record_Status
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 30;

```

```

-- Query 6 --
-- Negative Sales values ko running min ke through track karna

```

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    MIN(Sales) OVER (
        ORDER BY Order_Date
    ) AS Running_Min
FROM Dirty_Sales_Dataset
ORDER BY Order_Date;

```

```

-- Query 7 --
-- West region mein running max/min check karna

```

```

SELECT
    Order_ID,
    Order_Date,

```

```

Sales,
MAX(Sales) OVER (
    ORDER BY Order_Date
) AS Running_Max,
MIN(Sales) OVER (
    ORDER BY Order_Date
) AS Running_Min
FROM Dirty_Sales_Dataset
WHERE Region = 'West'
ORDER BY Order_Date
LIMIT 40;

```

D1.3.5 7-Day Rolling Average Using ROWS BETWEEN

```

-- Query 1 --
-- Daily sales total nikalna (one row per date)

```

```

SELECT
    Order_Date,
    SUM(Sales) AS Daily_Sales
FROM Dirty_Sales_Dataset
GROUP BY Order_Date
ORDER BY Order_Date
LIMIT 20;

```

```

-- Query 2 --
-- 7-day rolling average using daily sales

```

```

SELECT
    Order_Date,
    Daily_Sales,
    AVG(Daily_Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS Rolling_7Day_Avg
FROM (
    SELECT
        Order_Date,
        SUM(Sales) AS Daily_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Order_Date
) AS daily_totals
ORDER BY Order_Date
LIMIT 30;

```

```

-- Query 3 --
-- Region-wise 7-day rolling average

```

```

SELECT
    Region,
    Order_Date,
    Daily_Sales,
    AVG(Daily_Sales) OVER (

```

```

        PARTITION BY Region
        ORDER BY Order_Date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS Region_Rolling_7Day_Avg
FROM (
    SELECT
        Region,
        Order_Date,
        SUM(Sales) AS Daily_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region, Order_Date
) AS daily_region_totals
ORDER BY Region, Order_Date
LIMIT 40;

```

```

-- Query 4 --
-- Daily sales vs rolling average (deviation from trend)

```

```

SELECT
    Order_Date,
    Daily_Sales,
    AVG(Daily_Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS Rolling_7Day_Avg,
    ROUND(
        Daily_Sales - AVG(Daily_Sales) OVER (
            ORDER BY Order_Date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ), 2
    ) AS Deviation_From_Trend
FROM (
    SELECT
        Order_Date,
        SUM(Sales) AS Daily_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Order_Date
) AS daily_totals
ORDER BY Order_Date
LIMIT 30;

```

```

-- Query 5 --
-- Recent data par 7-day rolling average (2025 onwards)

```

```

SELECT
    Order_Date,
    Daily_Sales,
    AVG(Daily_Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS Rolling_7Day_Avg
FROM (
    SELECT

```

```

        Order_Date,
        SUM(Sales) AS Daily_Sales
    FROM Dirty_Sales_Dataset
    WHERE Order_Date >= date('2025-01-01')
    GROUP BY Order_Date
) AS daily_totals
ORDER BY Order_Date;

```

```

-- Query 6 --
-- West region ka 7-day rolling average

```

```

SELECT
    Order_Date,
    Daily_Sales,
    AVG(Daily_Sales) OVER (
        ORDER BY Order_Date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS Rolling_7Day_Avg
FROM (
    SELECT
        Order_Date,
        SUM(Sales) AS Daily_Sales
    FROM Dirty_Sales_Dataset
    WHERE Region = 'West'
    GROUP BY Order_Date
) AS daily_totals
ORDER BY Order_Date
LIMIT 50;

```

```

-- Query 7 --
-- 7-day rolling average with rounded values

```

```

SELECT
    Order_Date,
    Daily_Sales,
    ROUND(
        AVG(Daily_Sales) OVER (
            ORDER BY Order_Date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ), 2
    ) AS Rolling_7Day_Avg
FROM (
    SELECT
        Order_Date,
        SUM(Sales) AS Daily_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Order_Date
) AS daily_totals
ORDER BY Order_Date
LIMIT 50;

```

D1.4.1 LAG() – Access Previous Row Value

```
-- Query 1 --  
-- Simple LAG (pichle order ki Sales dekhna)
```

```
SELECT  
    Order_ID,  
    Order_Date,  
    Sales,  
    LAG(Sales) OVER (  
        ORDER BY Order_Date  
    ) AS Previous_Sales  
FROM Dirty_Sales_Dataset  
ORDER BY Order_Date  
LIMIT 20;
```

```
-- Query 2 --  
-- Day-over-day change nikalna
```

```
SELECT  
    Order_ID,  
    Order_Date,  
    Sales,  
    LAG(Sales) OVER (  
        ORDER BY Order_Date  
    ) AS Previous_Sales,  
    Sales - LAG(Sales) OVER (  
        ORDER BY Order_Date  
    ) AS Change_From_Previous  
FROM Dirty_Sales_Dataset  
ORDER BY Order_Date  
LIMIT 20;
```

```
-- Query 3 --  
-- Customer ka pichla order (PARTITION BY ke saath)
```

```
SELECT  
    Order_ID,  
    Customer_ID,  
    Order_Date,  
    Sales,  
    LAG(Order_Date) OVER (  
        PARTITION BY Customer_ID  
        ORDER BY Order_Date  
    ) AS Previous_Order_Date,  
    LAG(Sales) OVER (  
        PARTITION BY Customer_ID  
        ORDER BY Order_Date  
    ) AS Previous_Order_Sales  
FROM Dirty_Sales_Dataset  
ORDER BY Customer_ID, Order_Date  
LIMIT 30;
```

```
-- Query 4 --
```

-- Default value use karna (NULL ki jagah 0)

```
SELECT
    Order_ID,
    Order_Date,
    Sales,
    LAG(Sales, 1, 0) OVER (
        ORDER BY Order_Date
    ) AS Previous_Sales_Or_Zero
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 10;
```

-- Query 5 --

-- 1 row aur 2 rows peeche dekhna (offset)

```
SELECT
    Order_ID,
    Order_Date,
    Sales,
    LAG(Sales, 1) OVER (
        ORDER BY Order_Date
    ) AS One_Row_Back,
    LAG(Sales, 2) OVER (
        ORDER BY Order_Date
    ) AS Two_Rows_Back
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 10;
```

-- Query 6 --

-- Days between consecutive customer orders

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    LAG(Order_Date) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Previous_Order_Date,
    julianday(Order_Date) - julianday(
        LAG(Order_Date) OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        )
    ) AS Days_Since_Last_Order
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;
```

-- Query 7 --

-- Customer ke previous order sales aur sales change

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    LAG(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Previous_Order_Sales,
    Sales - LAG(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Sales_Change
FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
    HAVING COUNT(*) > 3
)
ORDER BY Customer_ID, Order_Date
LIMIT 40;
```

D1.4.2 LEAD() – Access Next Row Value

-- Query 1 --
-- Simple LEAD (agle order ki Sales dekhna)

```
SELECT
    Order_ID,
    Order_Date,
    Sales,
    LEAD(Sales) OVER (
        ORDER BY Order_Date
    ) AS Next_Sales
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;
```

-- Query 2 --
-- Customer ka next order kab hoga

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    LEAD(Order_Date) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Next_Order_Date
FROM Dirty_Sales_Dataset
```

```
ORDER BY Customer_ID, Order_Date
LIMIT 30;
```

```
-- Query 3 --
-- Customer ka last order dhoondna (LEAD IS NULL)
```

```
SELECT *
FROM (
    SELECT
        Order_ID,
        Customer_ID,
        Order_Date,
        Sales,
        LEAD(Order_Date) OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        ) AS Next_Order_Date
    FROM Dirty_Sales_Dataset
) AS with_lead
WHERE Next_Order_Date IS NULL;
```

```
-- Query 4 --
-- Days until next order
```

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    LEAD(Order_Date) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Next_Order_Date,
    julianday(
        LEAD(Order_Date) OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        )
    ) - julianday(Order_Date) AS Days_Until_Next_Order
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;
```

```
-- Query 5 --
-- LAG aur LEAD ek saath
```

```
SELECT
    Order_ID,
    Order_Date,
    Sales,
    LAG(Sales) OVER (
        ORDER BY Order_Date
    ) AS Previous_Sales,
```

```

Sales AS Current_Sales,
LEAD(Sales) OVER (
    ORDER BY Order_Date
) AS Next_Sales
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;

```

```

-- Query 6 --
-- Next order Sales se comparison

```

```

SELECT
    Order_ID,
    Order_Date,
    Sales,
    LEAD(Sales) OVER (
        ORDER BY Order_Date
    ) AS Next_Sales,
    CASE
        WHEN LEAD(Sales) OVER (
            ORDER BY Order_Date
        ) > Sales
        THEN 'Sales Increased Next'

        WHEN LEAD(Sales) OVER (
            ORDER BY Order_Date
        ) < Sales
        THEN 'Sales Decreased Next'

        ELSE 'No Change / Last Row'
    END AS Trend_Direction
FROM Dirty_Sales_Dataset
ORDER BY Order_Date
LIMIT 20;

```

```

-- Query 7 --
-- First aur last order per customer

```

```

SELECT *
FROM (
    SELECT
        Order_ID,
        Customer_ID,
        Order_Date,
        Sales,
        LAG(Order_Date) OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        ) AS Prev_Order,
        LEAD(Order_Date) OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        ) AS Next_Order

```

```

    FROM Dirty_Sales_Dataset
) AS with_lag_lead
WHERE Prev_Order IS NULL
    OR Next_Order IS NULL
ORDER BY Customer_ID, Order_Date
LIMIT 40;

```

D1.4.3 FIRST_VALUE() – First Value in the Window

```

-- Query 1 --
-- Customer ka pehla order date har row mein dikhana

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    FIRST_VALUE(Order_Date) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Order_Date
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 2 --
-- Customer ke first order ki Sales value dikhana

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    FIRST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Order_Sales
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 3 --
-- Region ka sabse pehla order kab hua

```

```

SELECT
    Order_ID,
    Region,
    Order_Date,
    Sales,
    FIRST_VALUE(Order_Date) OVER (
        PARTITION BY Region

```

```

        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Region_First_Order
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Order_Date
LIMIT 20;

```

```

-- Query 4 --
-- First order Sales se current order ka comparison

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    FIRST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Order_Sales,
    Sales - FIRST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Change_Since_First_Order
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 5 --
-- Region ke highest sales wale order ko identify karna

```

```

SELECT
    Order_ID,
    Region,
    Sales,
    FIRST_VALUE(Order_ID) OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Top_Order_ID_In_Region
FROM Dirty_Sales_Dataset
WHERE Region = 'South'
ORDER BY Sales DESC
LIMIT 20;

```

```

-- Query 6 --
-- MIN() vs FIRST_VALUE() comparison

```

```

SELECT
    Order_ID,

```

```

Customer_ID,
Order_Date,
Sales,
MIN(Sales) OVER (
    PARTITION BY Customer_ID
) AS Customer_Min_Sales,
FIRST_VALUE(Sales) OVER (
    PARTITION BY Customer_ID
    ORDER BY Order_Date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS Customer_First_Order_Sales
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 7 --
-- Multiple-order customers ka first order date aur sales

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    FIRST_VALUE(Order_Date) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Order_Date,
    FIRST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Order_Sales
FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
    HAVING COUNT(*) > 3
)
ORDER BY Customer_ID, Order_Date
LIMIT 40;

```

D1.4.4 LAST_VALUE() – Last Value in the Window

```

-- Query 1 --
-- Customer ka last order date har row mein dikhana

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    LAST_VALUE(Order_Date) OVER (

```

```

        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Last_Order_Date
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 2 --
-- Customer ke last order ki Sales value dikhana

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    LAST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Last_Order_Sales
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 3 --
-- FIRST_VALUE aur LAST_VALUE ek saath

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    FIRST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Order_Sales,
    LAST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Last_Order_Sales
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 4 --
-- First order se last order tak Sales comparison

```

```

SELECT DISTINCT
    Customer_ID,

```

```

FIRST_VALUE(Sales) OVER (
    PARTITION BY Customer_ID
    ORDER BY Order_Date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS First_Order_Sales,
LAST_VALUE(Sales) OVER (
    PARTITION BY Customer_ID
    ORDER BY Order_Date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
) AS Last_Order_Sales
FROM Dirty_Sales_Dataset
LIMIT 20;

```

```

-- Query 5 --
-- Region ka sabse latest order date

```

```

SELECT
    Order_ID,
    Region,
    Order_Date,
    LAST_VALUE(Order_Date) OVER (
        PARTITION BY Region
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Region_Last_Order_Date
FROM Dirty_Sales_Dataset
WHERE Region = 'East'
ORDER BY Order_Date
LIMIT 20;

```

```

-- Query 6 --
-- LAST_VALUE ka default frame vs explicit frame

```

```

SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    LAST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Last_Value_Without_Fix,
    LAST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Last_Value_With_Fix
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 7 --

```

-- Last row per customer using ROW_NUMBER()

```
SELECT *
FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num = 1;
```

-- Query 8 --

-- Multiple-order customers ka first aur last Sales

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    FIRST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS First_Sales,
    LAST_VALUE(Sales) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Last_Sales
FROM Dirty_Sales_Dataset
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
    HAVING COUNT(*) > 3
)
ORDER BY Customer_ID, Order_Date
LIMIT 40;
```

D1.4.5 NTH_VALUE() – Get Nth Row Value in Window

-- Query 1 --

-- Customer ka 2nd order ka Sales

```
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    NTH_VALUE(Sales, 2) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
```

```

        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Second_Order_Sales
FROM Dirty_Sales_Dataset
ORDER BY Customer_ID, Order_Date
LIMIT 30;

```

```

-- Query 2 --
-- Region ka 3rd highest sales order ID
SELECT
    Order_ID,
    Region,
    Sales,
    NTH_VALUE(Order_ID, 3) OVER (
        PARTITION BY Region
        ORDER BY Sales DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Third_Highest_Order_ID
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
ORDER BY Sales DESC
LIMIT 20;

```

```

-- Query 3 --
-- 1st, 2nd aur 3rd order Sales comparison
SELECT DISTINCT
    Customer_ID,
    NTH_VALUE(Sales, 1) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS First_Order,

    NTH_VALUE(Sales, 2) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Second_Order,

    NTH_VALUE(Sales, 3) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Third_Order
FROM Dirty_Sales_Dataset
LIMIT 20;

```

```

-- Query 4 --
-- Category mein 2nd best-selling product
SELECT DISTINCT
    Category,
    NTH_VALUE(Product_Name, 2) OVER (
        PARTITION BY Category

```

```

        ORDER BY Sales DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Second_Best_Seller
FROM Dirty_Sales_Dataset;

```

```

-- Query 5 --
-- RANK() se 2nd highest sales
SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,
        RANK() OVER (
            ORDER BY Sales DESC
        ) AS Rnk
    FROM Dirty_Sales_Dataset
)
WHERE Rnk = 2;

```

```

-- Query 6 --
-- NTH_VALUE() se 2nd highest sales
SELECT DISTINCT
    NTH_VALUE(Sales, 2) OVER (
        ORDER BY Sales DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Second_Highest_Sales
FROM Dirty_Sales_Dataset;

```

```

-- Query 7 --
-- SQLite-friendly alternative using ROW_NUMBER()
-- Customer ka 2nd order
SELECT *
FROM (
    SELECT
        Customer_ID,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num = 2;

```

```

-- Query 8 --
-- DBeaver/SQLite alternative with complete order details
SELECT *
FROM (
    SELECT
        Order_ID,
        Customer_ID,

```

```

        Order_Date,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Customer_ID
            ORDER BY Order_Date
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Row_Num = 2
ORDER BY Customer_ID
LIMIT 20;

```

```

-- Query 9 --
-- DBEaver mein NTH_VALUE() test karo
SELECT DISTINCT
    Customer_ID,
    NTH_VALUE(Sales, 2) OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS Second_Order_Sales
FROM Dirty_Sales_Dataset
LIMIT 20;

```

D1.5.1 PERCENT_RANK() – Relative Rank as Percentage

```

-- Query 1 --
-- Sales ka percent rank poore dataset mein
SELECT
    Order_ID,
    Sales,
    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ),
        4
    ) AS Sales_Percent_Rank
FROM Dirty_Sales_Dataset
ORDER BY Sales
LIMIT 20;

```

```

-- Query 2 --
-- Customer spending ka percentile
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Spending,
    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY SUM(Sales)
        ),
        4
    ) AS Spending_Percentile
FROM Dirty_Sales_Dataset

```

```

GROUP BY Customer_ID
ORDER BY Total_Spending DESC
LIMIT 20;

```

```

-- Query 3 --
-- Region ke andar Sales ka percent rank

```

```

SELECT
    Order_ID,
    Region,
    Sales,
    ROUND(
        PERCENT_RANK() OVER (
            PARTITION BY Region
            ORDER BY Sales
        ),
        4
    ) AS Region_Percent_Rank
FROM Dirty_Sales_Dataset
WHERE Region = 'West'
ORDER BY Sales
LIMIT 20;

```

```

-- Query 4 --
-- Top 10% high-value orders

```

```

SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) AS Percent_Rank
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Percent_Rank >= 0.90
ORDER BY Sales DESC;

```

```

-- Query 5 --
-- Percentile ko readable bucket mein convert karna

```

```

SELECT
    Order_ID,
    Sales,
    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) * 100,
        1
    ) AS Percentile,

    CASE
        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales

```

```

        ) >= 0.90
            THEN 'Top 10%'

        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales
        ) >= 0.75
            THEN 'Top 25%'

        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales
        ) >= 0.50
            THEN 'Above Median'

        ELSE 'Below Median'
    END AS Percentile_Bucket
FROM Dirty_Sales_Dataset
ORDER BY Sales DESC
LIMIT 30;

```

```

-- Query 6 --
-- RANK() aur PERCENT_RANK() ka comparison
SELECT
    Order_ID,
    Sales,

    RANK() OVER (
        ORDER BY Sales
    ) AS Sales_Rank,

    PERCENT_RANK() OVER (
        ORDER BY Sales
    ) AS Sales_Percent_Rank
FROM Dirty_Sales_Dataset
ORDER BY Sales
LIMIT 30;

```

```

-- Query 7 --
-- Region-wise percent rank comparison
SELECT
    Order_ID,
    Region,
    Sales,

    ROUND(
        PERCENT_RANK() OVER (
            PARTITION BY Region
            ORDER BY Sales
        ) * 100,
        2
    ) AS Region_Percentile
FROM Dirty_Sales_Dataset
ORDER BY Region, Sales
LIMIT 40;

```

```

-- Query 8 --
-- Top 10%, Bottom 10% aur Middle 80% segmentation
SELECT
    Order_ID,
    Sales,

    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) * 100,
        2
    ) AS Percentile,

    CASE
        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales
        ) >= 0.90
            THEN 'Top 10%'

        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales
        ) <= 0.10
            THEN 'Bottom 10%'

        ELSE 'Middle 80%'
    END AS Segment
FROM Dirty_Sales_Dataset
ORDER BY Sales DESC
LIMIT 40;

```

D1.5.2 CUME_DIST() – Cumulative Distribution Value

```

-- Query 1 --
-- Sales ka cumulative distribution
SELECT
    Order_ID,
    Sales,
    ROUND(
        CUME_DIST() OVER (
            ORDER BY Sales
        ),
        4
    ) AS Sales_Cume_Dist
FROM Dirty_Sales_Dataset
ORDER BY Sales
LIMIT 20;

```

```

-- Query 2 --
-- Kitne percent orders ki Sales current order se kam ya barabar hai
SELECT
    Order_ID,
    Sales,

```

```

ROUND(
    CUME_DIST() OVER (
        ORDER BY Sales
    ) * 100,
    2
) AS Percent_Orders_Below_Or_Equal
FROM Dirty_Sales_Dataset
ORDER BY Sales DESC
LIMIT 20;

```

```

-- Query 3 --
-- Region ke andar cumulative distribution
SELECT
    Order_ID,
    Region,
    Sales,
    ROUND(
        CUME_DIST() OVER (
            PARTITION BY Region
            ORDER BY Sales
        ),
        4
    ) AS Region_Cume_Dist
FROM Dirty_Sales_Dataset
WHERE Region = 'South'
ORDER BY Sales
LIMIT 20;

```

```

-- Query 4 --
-- Top 20% orders dhoondna
SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,
        CUME_DIST() OVER (
            ORDER BY Sales
        ) AS Cume_Dist
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Cume_Dist > 0.80
ORDER BY Sales DESC;

```

```

-- Query 5 --
-- PERCENT_RANK() aur CUME_DIST() comparison
SELECT
    Order_ID,
    Sales,

    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales

```

```

    ),
    4
) AS Percent_Rank,

ROUND(
    CUME_DIST() OVER (
        ORDER BY Sales
    ),
    4
) AS Cume_Dist
FROM Dirty_Sales_Dataset
ORDER BY Sales
LIMIT 20;

```

```

-- Query 6 --
-- Region-wise CUME_DIST() comparison
SELECT
    Order_ID,
    Region,
    Sales,

    ROUND(
        CUME_DIST() OVER (
            PARTITION BY Region
            ORDER BY Sales
        ) * 100,
        2
    ) AS Region_Cume_Dist_Pct
FROM Dirty_Sales_Dataset
ORDER BY Region, Sales
LIMIT 40;

```

```

-- Query 7 --
-- Duplicate Sales values ke saath CUME_DIST()
SELECT
    Order_ID,
    Sales,

    COUNT(*) OVER (
        PARTITION BY Sales
    ) AS Same_Sales_Count,

    ROUND(
        CUME_DIST() OVER (
            ORDER BY Sales
        ),
        4
    ) AS Cume_Dist
FROM Dirty_Sales_Dataset
ORDER BY Sales
LIMIT 30;

```

```
-- Query 8 --
-- PERCENT_RANK() aur CUME_DIST() percentage mein
SELECT
    Order_ID,
    Sales,

    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) * 100,
        2
    ) AS Percent_Rank_Pct,

    ROUND(
        CUME_DIST() OVER (
            ORDER BY Sales
        ) * 100,
        2
    ) AS Cume_Dist_Pct
FROM Dirty_Sales_Dataset
ORDER BY Sales
LIMIT 15;
```

D1.5.3 PERCENTILE CALCULATION USING WINDOW FUNCTIONS

```
-- Query 1 --
-- PERCENTILE_CONT() se median (PostgreSQL / SQL Server)
SELECT
    PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY Sales) AS Median_Sales
FROM Dirty_Sales_Dataset;
```

```
-- Query 2 --
-- Region-wise P25, Median aur P75
SELECT
    Region,

    PERCENTILE_CONT(0.25)
    WITHIN GROUP (ORDER BY Sales) AS P25_Sales,

    PERCENTILE_CONT(0.50)
    WITHIN GROUP (ORDER BY Sales) AS Median_Sales,

    PERCENTILE_CONT(0.75)
    WITHIN GROUP (ORDER BY Sales) AS P75_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 3 --
-- SQLite-friendly median calculation
WITH Ranked_Sales AS (
    SELECT
        Sales,
```

```

        ROW_NUMBER() OVER (
            ORDER BY Sales
        ) AS Row_Num,
        COUNT(*) OVER () AS Total_Rows
    FROM Dirty_Sales_Dataset
    WHERE Sales IS NOT NULL
)
SELECT
    AVG(Sales) AS Median_Sales
FROM Ranked_Sales
WHERE Row_Num IN (
    (Total_Rows + 1) / 2,
    (Total_Rows + 2) / 2
);

```

```

-- Query 4 --
-- 25th, 50th aur 75th percentile (Quartiles)
WITH Ranked_Sales AS (
    SELECT
        Sales,
        ROW_NUMBER() OVER (
            ORDER BY Sales
        ) AS Row_Num,
        COUNT(*) OVER () AS Total_Rows
    FROM Dirty_Sales_Dataset
    WHERE Sales IS NOT NULL
        AND Sales > 0
)
SELECT
    MAX(
        CASE
            WHEN Row_Num = CAST(0.25 * Total_Rows AS INTEGER)
            THEN Sales
        END
    ) AS P25,
    MAX(
        CASE
            WHEN Row_Num = CAST(0.50 * Total_Rows AS INTEGER)
            THEN Sales
        END
    ) AS P50_Median,
    MAX(
        CASE
            WHEN Row_Num = CAST(0.75 * Total_Rows AS INTEGER)
            THEN Sales
        END
    ) AS P75
FROM Ranked_Sales;

```

```

-- Query 5 --
-- NTILE(100) se approximate percentile groups

```

```

SELECT
    NTile_Group,
    MIN(Sales) AS Min_In_Group,
    MAX(Sales) AS Max_In_Group
FROM (
    SELECT
        Sales,
        NTILE(100) OVER (
            ORDER BY Sales
        ) AS NTile_Group
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
) AS percentile_groups
WHERE NTile_Group IN (25, 50, 75)
GROUP BY NTile_Group;

```

```

-- Query 6 --
-- Region-wise median using PARTITION BY
WITH Ranked_Sales AS (
    SELECT
        Region,
        Sales,

        ROW_NUMBER() OVER (
            PARTITION BY Region
            ORDER BY Sales
        ) AS Row_Num,

        COUNT(*) OVER (
            PARTITION BY Region
        ) AS Total_Rows
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
)
SELECT
    Region,
    AVG(Sales) AS Median_Sales
FROM Ranked_Sales
WHERE Row_Num IN (
    (Total_Rows + 1) / 2,
    (Total_Rows + 2) / 2
)
GROUP BY Region;

```

```

-- Query 7 --
-- CUME_DIST() se approximate median
SELECT
    MIN(Sales) AS Approx_Median
FROM (
    SELECT
        Sales,
        CUME_DIST() OVER (
            ORDER BY Sales

```

```

        ) AS Cume_Dist
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
) AS ranked
WHERE Cume_Dist >= 0.50;

```

```

-- Query 8 --
-- Average aur Median ka comparison
SELECT

```

```

    AVG(Sales) AS Average_Sales,

    (
        SELECT AVG(Sales)
        FROM (
            SELECT
                Sales,
                ROW_NUMBER() OVER (
                    ORDER BY Sales
                ) AS Row_Num,
                COUNT(*) OVER () AS Total_Rows
            FROM Dirty_Sales_Dataset
            WHERE Sales > 0
        ) AS Ranked_Sales
        WHERE Row_Num IN (
            (Total_Rows + 1) / 2,
            (Total_Rows + 2) / 2
        )
    ) AS Median_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0;

```

```

-- Query 9 --
-- DBeaver mein Median aur Average side-by-side compare
WITH Ranked_Sales AS (

```

```

    SELECT
        Sales,
        ROW_NUMBER() OVER (
            ORDER BY Sales
        ) AS Row_Num,
        COUNT(*) OVER () AS Total_Rows
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
)
SELECT
    AVG(Sales) AS Median_Sales,

    (
        SELECT AVG(Sales)
        FROM Dirty_Sales_Dataset
        WHERE Sales > 0
    ) AS Average_Sales
FROM Ranked_Sales
WHERE Row_Num IN (

```

```

        (Total_Rows + 1) / 2,
        (Total_Rows + 2) / 2
    );

```

D1.5.4 FINDING OUTLIERS USING PERCENT_RANK() AND CUME_DIST()

```

-- Query 1 --
-- Top 1% aur Bottom 1% Sales values

```

```

SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) AS Percent_Rank
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Percent_Rank <= 0.01
    OR Percent_Rank >= 0.99
ORDER BY Sales;

```

```

-- Query 2 --
-- Outlier status label karna

```

```

SELECT
    Order_ID,
    Sales,

    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) * 100,
        2
    ) AS Percentile,

    CASE
        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales
        ) <= 0.01
            THEN 'Low Outlier'

        WHEN PERCENT_RANK() OVER (
            ORDER BY Sales
        ) >= 0.99
            THEN 'High Outlier'

        ELSE 'Normal'
    END AS Outlier_Status
FROM Dirty_Sales_Dataset
ORDER BY Sales;

```

```

-- Query 3 --
-- Negative Sales values ka outlier cross-check

```

```

SELECT
    Order_ID,
    Sales,

    ROUND(
        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) * 100,
        2
    ) AS Percentile
FROM Dirty_Sales_Dataset
WHERE Sales < 0
ORDER BY Sales;

```

```

-- Query 4 --
-- CUME_DIST() se outliers identify karna
SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,
        CUME_DIST() OVER (
            ORDER BY Sales
        ) AS Cume_Dist
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Cume_Dist <= 0.01
    OR Cume_Dist >= 0.99
ORDER BY Sales;

```

```

-- Query 5 --
-- Region-wise outlier detection
SELECT *
FROM (
    SELECT
        Order_ID,
        Region,
        Sales,

        PERCENT_RANK() OVER (
            PARTITION BY Region
            ORDER BY Sales
        ) AS Region_Percent_Rank
    FROM Dirty_Sales_Dataset
) AS ranked
WHERE Region_Percent_Rank <= 0.02
    OR Region_Percent_Rank >= 0.98
ORDER BY Region, Sales;

```

```

-- Query 6 --
-- Outlier count summary
SELECT

```

```

Outlier_Status,
COUNT(*) AS Count,
ROUND(AVG(Sales), 2) AS Avg_Sales_In_Group
FROM (
    SELECT
        Sales,

        CASE
            WHEN PERCENT_RANK() OVER (
                ORDER BY Sales
            ) <= 0.01
            THEN 'Low Outlier'

            WHEN PERCENT_RANK() OVER (
                ORDER BY Sales
            ) >= 0.99
            THEN 'High Outlier'

            ELSE 'Normal'
        END AS Outlier_Status
    FROM Dirty_Sales_Dataset
) AS categorized
GROUP BY Outlier_Status;

```

```

-- Query 7 --
-- Outliers exclude karke average comparison

```

```

SELECT
    AVG(Sales) AS Avg_With_Outliers,

    (
        SELECT AVG(Sales)
        FROM (
            SELECT
                Sales,
                PERCENT_RANK() OVER (
                    ORDER BY Sales
                ) AS Pr
            FROM Dirty_Sales_Dataset
        ) AS ranked
        WHERE Pr > 0.01
        AND Pr < 0.99
    ) AS Avg_Without_Outliers
FROM Dirty_Sales_Dataset;

```

```

-- Query 8 --
-- Loose outlier detection (Bottom 5% aur Top 5%)

```

```

SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,

        PERCENT_RANK() OVER (

```

```

        ORDER BY Sales
    ) AS Percent_Rank
FROM Dirty_Sales_Dataset
) AS ranked
WHERE Percent_Rank <= 0.05
    OR Percent_Rank >= 0.95
ORDER BY Sales;

```

```

-- Query 9 --
-- DBeaver mein Top 2% aur Bottom 2% outlier check
WITH Ranked_Data AS (
    SELECT
        Order_ID,
        Sales,
        Quantity,
        Discount,

        PERCENT_RANK() OVER (
            ORDER BY Sales
        ) AS Percent_Rank
    FROM Dirty_Sales_Dataset
)
SELECT
    Order_ID,
    Sales,
    Quantity,
    Discount,

    ROUND(Percent_Rank * 100, 2) AS Sales_Percentile,

    CASE
        WHEN Percent_Rank <= 0.02
            THEN 'Bottom 2% - Check for Errors'

        WHEN Percent_Rank >= 0.98
            THEN 'Top 2% - Check for Errors'

        ELSE 'Normal Range'
    END AS Flag
FROM Ranked_Data
WHERE Percent_Rank <= 0.02
    OR Percent_Rank >= 0.98
ORDER BY Sales
LIMIT 30;

```

```

-- Query 10 --
-- Corrected outlier query using percentile percentage
SELECT *
FROM (
    SELECT
        Order_ID,
        Sales,
        Quantity,

```

```

Discount,

ROUND(
    PERCENT_RANK() OVER (
        ORDER BY Sales
    ) * 100,
    2
) AS Sales_Percentile
FROM Dirty_Sales_Dataset
) AS ranked
WHERE Sales_Percentile <= 2
    OR Sales_Percentile >= 98
ORDER BY Sales
LIMIT 30;

```

D2.1.1 WITH CLAUSE SYNTAX – HOW TO WRITE A CTE

```

-- Query 1 --
-- Basic CTE: High-value orders
WITH High_Value_Orders AS (
    SELECT
        Order_ID,
        Customer_Name,
        Sales
    FROM Dirty_Sales_Dataset
    WHERE Sales > 10000
)
SELECT *
FROM High_Value_Orders;

-- Query 2 --
-- CTE ke baad further filtering aur aggregation
WITH High_Value_Orders AS (
    SELECT
        Order_ID,
        Customer_Name,
        Region,
        Sales
    FROM Dirty_Sales_Dataset
    WHERE Sales > 10000
)
SELECT
    Region,
    COUNT(*) AS Total_High_Value_Orders,
    SUM(Sales) AS Total_Sales
FROM High_Value_Orders
GROUP BY Region
ORDER BY Total_Sales DESC;

```

```

-- Query 3 --
-- Region summary ko CTE se readable banana
WITH Region_Summary AS (
    SELECT

```

```

        Region,
        SUM(Sales) AS Total_Sales,
        COUNT(*) AS Order_Count
    FROM Dirty_Sales_Dataset
    GROUP BY Region
)
SELECT *
FROM Region_Summary
WHERE Total_Sales > 500000;

```

```

-- Query 4 --
-- Multiple CTEs ko JOIN karna
WITH Region_Sales AS (
    SELECT
        Region,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
),
Region_Orders AS (
    SELECT
        Region,
        COUNT(*) AS Order_Count
    FROM Dirty_Sales_Dataset
    GROUP BY Region
)
SELECT
    rs.Region,
    rs.Total_Sales,
    ro.Order_Count
FROM Region_Sales rs
JOIN Region_Orders ro
    ON rs.Region = ro.Region;

```

```

-- Query 5 --
-- Window function ke saath CTE
WITH Ranked_Sales AS (
    SELECT
        Order_ID,
        Region,
        Sales,

        ROW_NUMBER() OVER (
            PARTITION BY Region
            ORDER BY Sales DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
)
SELECT *
FROM Ranked_Sales
WHERE Row_Num <= 3;

```

```

-- Query 6 --
-- Subquery approach
SELECT *
FROM (
    SELECT
        Region,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
) AS region_summary
WHERE Total_Sales > 500000;

-- Query 7 --
-- CTE approach: Subquery ka alternative
WITH Region_Summary AS (
    SELECT
        Region,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
)
SELECT *
FROM Region_Summary
WHERE Total_Sales > 500000;

-- Query 8 --
-- Customer-wise total spending aur order count
WITH Customer_Totals AS (
    SELECT
        Customer_ID,
        Customer_Name,
        SUM(Sales) AS Total_Spending,
        COUNT(*) AS Total_Orders
    FROM Dirty_Sales_Dataset
    GROUP BY
        Customer_ID,
        Customer_Name
)
SELECT *
FROM Customer_Totals
WHERE Total_Spending > 50000
ORDER BY Total_Spending DESC
LIMIT 20;

```

D2.1.2 CTE vs Subquery – Readability and Reusability

```

-- Query 1: Nested Subquery Approach --
SELECT *
FROM (
    SELECT
        Category,
        SUM(Sales) AS Total_Sales

```

```

FROM (
    SELECT
        Category,
        Sales
    FROM Dirty_Sales_Dataset
    WHERE Order_Status != 'Cancelled'
) AS filtered_data
GROUP BY Category
) AS category_totals
WHERE Total_Sales > 100000;

```

-- Query 2: Step-by-Step CTE Approach --

```

WITH Filtered_Data AS (
    SELECT
        Category,
        Sales
    FROM Dirty_Sales_Dataset
    WHERE Order_Status != 'Cancelled'
),
Category_Totals AS (
    SELECT
        Category,
        SUM(Sales) AS Total_Sales
    FROM Filtered_Data
    GROUP BY Category
)
SELECT *
FROM Category_Totals
WHERE Total_Sales > 100000;

```

-- Query 3: Subquery Repeated for Company Total Sales --

```

SELECT
    Region,
    (
        SELECT SUM(Sales)
        FROM Dirty_Sales_Dataset
        WHERE Sales > 0
    ) AS Company_Total_Sales,
    SUM(Sales) AS Region_Sales,
    ROUND(
        SUM(Sales) * 100.0 /
        (
            SELECT SUM(Sales)
            FROM Dirty_Sales_Dataset
            WHERE Sales > 0
        ),
        2
    ) AS Percent_Of_Total
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;

```

-- Query 4: CTE Reused for Company Total Sales --

```
WITH Company_Total AS (  
    SELECT  
        SUM(Sales) AS Total_Sales  
    FROM Dirty_Sales_Dataset  
    WHERE Sales > 0  
)  
SELECT  
    d.Region,  
    ct.Total_Sales AS Company_Total_Sales,  
    SUM(d.Sales) AS Region_Sales,  
    ROUND(  
        SUM(d.Sales) * 100.0 / ct.Total_Sales,  
        2  
    ) AS Percent_Of_Total  
FROM Dirty_Sales_Dataset d  
CROSS JOIN Company_Total ct  
WHERE d.Sales > 0  
GROUP BY  
    d.Region,  
    ct.Total_Sales;
```

-- Query 5: Reusing CTE in JOIN --

```
WITH Customer_Summary AS (  
    SELECT  
        Customer_ID,  
        SUM(Sales) AS Total_Spending,  
        COUNT(*) AS Total_Orders  
    FROM Dirty_Sales_Dataset  
    GROUP BY Customer_ID  
)  
SELECT  
    d.Order_ID,  
    d.Customer_ID,  
    d.Sales,  
    cs.Total_Spending,  
    cs.Total_Orders  
FROM Dirty_Sales_Dataset d  
JOIN Customer_Summary cs  
    ON d.Customer_ID = cs.Customer_ID  
WHERE cs.Total_Spending > 50000  
LIMIT 30;
```

-- Query 6: Complex Multi-Step Logic Using CTE --

```
WITH Valid_Orders AS (  
    SELECT *  
    FROM Dirty_Sales_Dataset  
    WHERE Sales > 0  
        AND Order_Status != 'Cancelled'  
)  
Customer_Totals AS (  
    SELECT  
        Customer_ID,
```

```

        SUM(Sales) AS Total_Sales
    FROM Valid_Orders
    GROUP BY Customer_ID
),
High_Value_Customers AS (
    SELECT
        Customer_ID
    FROM Customer_Totals
    WHERE Total_Sales > (
        SELECT AVG(Total_Sales)
        FROM Customer_Totals
    )
)
SELECT COUNT(*) AS High_Value_Customer_Count
FROM High_Value_Customers;

```

```

-- Query 7: Subquery Version for Sales Summary --
SELECT *
FROM (
    SELECT
        Salesperson,
        SUM(Sales) AS Total_Sales,
        COUNT(*) AS Order_Count
    FROM Dirty_Sales_Dataset
    WHERE Order_Status != 'Cancelled'
    GROUP BY Salesperson
) AS sales_summary
WHERE Total_Sales > 200000
ORDER BY Total_Sales DESC;

```

```

-- Query 8: CTE Version for Sales Summary --
WITH Sales_Summary AS (
    SELECT
        Salesperson,
        SUM(Sales) AS Total_Sales,
        COUNT(*) AS Order_Count
    FROM Dirty_Sales_Dataset
    WHERE Order_Status != 'Cancelled'
    GROUP BY Salesperson
)
SELECT *
FROM Sales_Summary
WHERE Total_Sales > 200000
ORDER BY Total_Sales DESC;

```

D2.1.3 Multiple CTEs in One Query – Chaining CTEs

```

-- Query 1: Basic CTE Chaining Syntax --
WITH CTE_1 AS (
    SELECT *
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0

```

```

),
CTE_2 AS (
    SELECT *
    FROM CTE_1
    WHERE Order_Status != 'Cancelled'
),
CTE_3 AS (
    SELECT *
    FROM CTE_2
    WHERE Sales > 1000
)
SELECT *
FROM CTE_3;

```

-- Query 2: Three-Step Pipeline – Filter → Aggregate → Rank --

```

WITH Valid_Orders AS (
    SELECT
        Customer_ID,
        Sales,
        Order_Date
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
    AND Order_Status != 'Cancelled'
),
Customer_Totals AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Sales,
        COUNT(*) AS Order_Count
    FROM Valid_Orders
    GROUP BY Customer_ID
),
Ranked_Customers AS (
    SELECT
        Customer_ID,
        Total_Sales,
        Order_Count,
        RANK() OVER (
            ORDER BY Total_Sales DESC
        ) AS Spending_Rank
    FROM Customer_Totals
)
SELECT *
FROM Ranked_Customers
WHERE Spending_Rank <= 10;

```

-- Query 3: Category-Wise Analysis Pipeline --

```

WITH Category_Sales AS (
    SELECT
        Category,
        Order_ID,
        Sales
    FROM Dirty_Sales_Dataset

```

```

        WHERE Sales > 0
    ),
    Category_Totals AS (
        SELECT
            Category,
            SUM(Sales) AS Total_Sales,
            COUNT(*) AS Order_Count,
            AVG(Sales) AS Avg_Sales
        FROM Category_Sales
        GROUP BY Category
    ),
    Category_With_Avg_Comparison AS (
        SELECT
            Category,
            Total_Sales,
            Order_Count,
            Avg_Sales,
            (
                SELECT AVG(Total_Sales)
                FROM Category_Totals
            ) AS Overall_Avg_Category_Sales
        FROM Category_Totals
    )
    SELECT
        Category,
        Total_Sales,
        Overall_Avg_Category_Sales,
        CASE
            WHEN Total_Sales > Overall_Avg_Category_Sales
            THEN 'Above Average'
            ELSE 'Below Average'
        END AS Performance
    FROM Category_With_Avg_Comparison
    ORDER BY Total_Sales DESC;

```

-- Query 4: Customer Churn Pipeline – 2023 vs 2024 --

```

WITH Customers_2023 AS (
    SELECT DISTINCT
        Customer_ID
    FROM Dirty_Sales_Dataset
    WHERE strftime('%Y', Order_Date) = '2023'
),
Customers_2024 AS (
    SELECT DISTINCT
        Customer_ID
    FROM Dirty_Sales_Dataset
    WHERE strftime('%Y', Order_Date) = '2024'
),
Churned_Customers AS (
    SELECT Customer_ID
    FROM Customers_2023

    EXCEPT

```

```

        SELECT Customer_ID
        FROM Customers_2024
    )
    SELECT
        d.Customer_ID,
        d.Customer_Name,
        SUM(d.Sales) AS Total_2023_Sales
    FROM Dirty_Sales_Dataset d
    JOIN Churned_Customers cc
        ON d.Customer_ID = cc.Customer_ID
    WHERE strftime('%Y', d.Order_Date) = '2023'
    GROUP BY
        d.Customer_ID,
        d.Customer_Name
    ORDER BY Total_2023_Sales DESC
    LIMIT 20;

```

-- Query 5: Four-Level Deep Chain – Regional Category Contribution --

```

WITH Regional_Orders AS (
    SELECT
        Region,
        Category,
        Sales
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
),
Regional_Category_Totals AS (
    SELECT
        Region,
        Category,
        SUM(Sales) AS Category_Sales
    FROM Regional_Orders
    GROUP BY
        Region,
        Category
),
Regional_Totals AS (
    SELECT
        Region,
        SUM(Category_Sales) AS Region_Total
    FROM Regional_Category_Totals
    GROUP BY Region
),
Category_Contribution AS (
    SELECT
        rct.Region,
        rct.Category,
        rct.Category_Sales,
        rt.Region_Total,
        ROUND(
            rct.Category_Sales * 100.0 / rt.Region_Total,
            2
        ) AS Contribution_Percent
    FROM Regional_Category_Totals rct

```

```

        JOIN Regional_Totals rt
        ON rct.Region = rt.Region
    )
    SELECT *
    FROM Category_Contribution
    ORDER BY
        Region,
        Contribution_Percent DESC;

```

-- Query 6: Incorrect CTE Order Example --
 -- CTE_2 references CTE_1 before CTE_1 is defined.
 -- This may result in an error depending on the database.

```

WITH CTE_2 AS (
    SELECT *
    FROM CTE_1
),
CTE_1 AS (
    SELECT *
    FROM Dirty_Sales_Dataset
)
SELECT *
FROM CTE_2;

```

-- Query 7: Correct CTE Order Example --

```

WITH CTE_1 AS (
    SELECT *
    FROM Dirty_Sales_Dataset
),
CTE_2 AS (
    SELECT *
    FROM CTE_1
)
SELECT *
FROM CTE_2;

```

-- Query 8: Reusing One CTE in Multiple Subsequent CTEs --

```

WITH Base_Data AS (
    SELECT *
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
),
Summary_A AS (
    SELECT
        Region,
        SUM(Sales) AS Total
    FROM Base_Data
    GROUP BY Region
),
Summary_B AS (
    SELECT
        Category,

```

```

        SUM(Sales) AS Total
    FROM Base_Data
    GROUP BY Category
)
SELECT
    sa.Region,
    sa.Total AS Regional_Total,
    sb.Category,
    sb.Total AS Category_Total
FROM Summary_A sa
CROSS JOIN Summary_B sb;

-- Query 9: Monthly Sales Chaining Pipeline --
WITH Monthly_Sales AS (
    SELECT
        strftime('%Y-%m', Order_Date) AS Sales_Month,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
    GROUP BY Sales_Month
),
Monthly_With_Avg AS (
    SELECT
        Sales_Month,
        Total_Sales,
        (
            SELECT AVG(Total_Sales)
            FROM Monthly_Sales
        ) AS Overall_Monthly_Avg
    FROM Monthly_Sales
),
Monthly_Performance AS (
    SELECT
        Sales_Month,
        Total_Sales,
        Overall_Monthly_Avg,
        CASE
            WHEN Total_Sales > Overall_Monthly_Avg
            THEN 'Above Average'
            ELSE 'Below Average'
        END AS Performance_Status
    FROM Monthly_With_Avg
)
SELECT *
FROM Monthly_Performance
ORDER BY Sales_Month;

```

D2.1.4 CTE vs Temp Table vs View – When to Use What

```

-- Query 1: CTE – Region Summary --
WITH Region_Summary AS (
    SELECT
        Region,

```

```

        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
)
SELECT *
FROM Region_Summary
WHERE Total_Sales > 500000;

```

```

-- Query 2: Create Temporary Table --
CREATE TEMP TABLE Region_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

```

-- Query 3: Use Temporary Table – Filter --
SELECT *
FROM Region_Summary
WHERE Total_Sales > 500000;

```

```

-- Query 4: Use Temporary Table – Sorting --
SELECT *
FROM Region_Summary
ORDER BY Total_Sales DESC;

```

```

-- Query 5: Drop Temporary Table --
DROP TABLE Region_Summary;

```

```

-- Query 6: Create View – Region Summary --
CREATE VIEW Region_Summary_View AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

```

-- Query 7: Use View – Filter --
SELECT *
FROM Region_Summary_View
WHERE Total_Sales > 500000;

```

```

-- Query 8: Drop View --
DROP VIEW Region_Summary_View;

```

```

-- Query 9: CTE – Top Customers --
WITH Top_Customers AS (

```

```

    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Spending
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
    HAVING SUM(Sales) > 50000
)
SELECT *
FROM Top_Customers
ORDER BY Total_Spending DESC;

```

```

-- Query 10: Create Temporary Table – Top Customers --
CREATE TEMP TABLE Top_Customers AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Spending
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID
HAVING SUM(Sales) > 50000;

```

```

-- Query 11: Temp Table – Order by Spending --
SELECT *
FROM Top_Customers
ORDER BY Total_Spending DESC;

```

```

-- Query 12: Temp Table – Count Customers --
SELECT COUNT(*)
FROM Top_Customers;

```

```

-- Query 13: Temp Table – JOIN with Dataset --
SELECT
    tc.*,
    d.Region
FROM Top_Customers tc
JOIN Dirty_Sales_Dataset d
    ON tc.Customer_ID = d.Customer_ID
LIMIT 20;

```

```

-- Query 14: Create View – Regional Sales Report --
CREATE VIEW Regional_Sales_Report AS
SELECT
    Region,
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    AVG(Sales) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY
    Region,

```

Category;

-- Query 15: Use View – Specific Region --

```
SELECT *
FROM Regional_Sales_Report
WHERE Region = 'North';
```

-- Query 16: SQLite – Create Temporary Table --

```
CREATE TEMPORARY TABLE Top_Customers AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Spending
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 17: CTE – Customer Summary --

```
WITH Customer_Summary AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Sales,
        COUNT(*) AS Order_Count
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
)
SELECT *
FROM Customer_Summary
WHERE Total_Sales > 30000
ORDER BY Total_Sales DESC
LIMIT 10;
```

-- Query 18: Temp Table – Customer Summary --

```
CREATE TEMP TABLE Customer_Summary AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 19: Temp Table – Filter and Sort --

```
SELECT *
FROM Customer_Summary
WHERE Total_Sales > 30000
ORDER BY Total_Sales DESC
LIMIT 10;
```

-- Query 20: Temp Table – Reuse Existing Result --

```
SELECT COUNT(*)
FROM Customer_Summary
```

```
WHERE Order_Count > 5;
```

```
-- Query 21: Drop Customer Summary Temp Table --  
DROP TABLE Customer_Summary;
```

```
-- Query 22: Create View – Customer Summary --  
CREATE VIEW Customer_Summary_View AS  
SELECT  
    Customer_ID,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Customer_ID;
```

```
-- Query 23: Use Customer Summary View --  
SELECT *  
FROM Customer_Summary_View  
WHERE Total_Sales > 30000  
ORDER BY Total_Sales DESC  
LIMIT 10;
```

D2.2.1 What is a Recursive CTE

```
-- Query 1: Basic Recursive CTE – Generate Numbers 1 to 10 --  
WITH RECURSIVE Numbers AS (  
    SELECT 1 AS Num  
  
    UNION ALL  
  
    SELECT Num + 1  
    FROM Numbers  
    WHERE Num < 10  
)  
SELECT *  
FROM Numbers;
```

```
-- Query 2: Generate Date Series – January 2023 --  
WITH RECURSIVE Date_Series AS (  
    SELECT date('2023-01-01') AS Calendar_Date  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < date('2023-01-31')  
)  
SELECT *  
FROM Date_Series;
```

-- Query 3: Find Missing Dates – Gap Analysis --

```
WITH RECURSIVE Date_Series AS (  
    SELECT date('2023-01-01') AS Calendar_Date  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < date('2023-01-31')  
)  
SELECT  
    ds.Calendar_Date  
FROM Date_Series ds  
LEFT JOIN Dirty_Sales_Dataset d  
    ON ds.Calendar_Date = d.Order_Date  
WHERE d.Order_Date IS NULL;
```

-- Query 4: Create Employee-Manager Table --

```
CREATE TABLE Salesperson_Info (  
    Salesperson TEXT PRIMARY KEY,  
    Region_Assigned TEXT,  
    Reports_To TEXT  
);
```

-- Query 5: Insert Employee-Manager Data --

```
INSERT INTO Salesperson_Info VALUES  
( 'Ravi Kumar', 'North', NULL),  
( 'Anjali Mehta', 'South', 'Ravi Kumar'),  
( 'Suresh Rao', 'East', 'Ravi Kumar'),  
( 'Priya Nair', 'West', 'Anjali Mehta'),  
( 'Karan Singh', 'West', 'Priya Nair');
```

-- Query 6: Employee-Manager Hierarchy Using Recursive CTE --

```
WITH RECURSIVE Employee_Hierarchy AS (  
    -- Anchor Member: Top-level employee  
    SELECT  
        Salesperson,  
        Reports_To,  
        1 AS Level  
    FROM Salesperson_Info  
    WHERE Reports_To IS NULL  
  
    UNION ALL  
  
    -- Recursive Member: Find next-level employees  
    SELECT  
        s.Salesperson,  
        s.Reports_To,  
        eh.Level + 1  
    FROM Salesperson_Info s  
    JOIN Employee_Hierarchy eh  
        ON s.Reports_To = eh.Salesperson
```

```
)  
SELECT *  
FROM Employee_Hierarchy  
ORDER BY Level;
```

-- Query 7: Simple Recursive Counter – 1 to 5 --

```
WITH RECURSIVE Counter AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Counter  
    WHERE N < 5  
)  
SELECT *  
FROM Counter;
```

-- Query 8: Incorrect Recursive CTE – Missing RECURSIVE Keyword --
-- This syntax is invalid in databases that require RECURSIVE.

```
WITH Numbers AS (  
    SELECT 1 AS Num  
  
    UNION ALL  
  
    SELECT Num + 1  
    FROM Numbers  
    WHERE Num < 10  
)  
SELECT *  
FROM Numbers;
```

-- Query 9: Recursive CTE with No Stopping Condition --
-- WARNING: Do not execute this query.
-- It may run indefinitely or generate an error.

```
WITH RECURSIVE Numbers AS (  
    SELECT 1 AS Num  
  
    UNION ALL  
  
    SELECT Num + 1  
    FROM Numbers  
)  
SELECT *  
FROM Numbers;
```

-- Query 10: Safe Recursive Counter – Practice Example --

```
WITH RECURSIVE Simple_Counter AS (  
    SELECT 1 AS Num
```

```

        UNION ALL

        SELECT Num + 1
        FROM Simple_Counter
        WHERE Num < 10
    )
    SELECT *
    FROM Simple_Counter;

```

D2.2.2 Anchor Member and Recursive Member

```

-- Query 1 --
-- Basic Recursive CTE Structure

WITH RECURSIVE cte_name AS (
    SELECT ...                -- ANCHOR MEMBER
    UNION ALL
    SELECT ...                -- RECURSIVE MEMBER
    FROM cte_name
    WHERE ...
)
SELECT * FROM cte_name;

```

```

-- Query 2 --
-- Anchor Member Separately

```

```

SELECT 1 AS Num;

```

```

-- Query 3 --
-- Anchor and Recursive Member: Internal Working

```

```

WITH RECURSIVE Numbers AS (
    -- ANCHOR MEMBER
    SELECT 1 AS Num

    UNION ALL

    -- RECURSIVE MEMBER
    SELECT Num + 1
    FROM Numbers
    WHERE Num < 5
)
SELECT * FROM Numbers;

```

```

-- Query 4 --
-- Anchor and Recursive Member: Date Sequence

```

```

WITH RECURSIVE Order_Sequence AS (
    -- ANCHOR MEMBER
    SELECT MIN(Order_Date) AS Seq_Date
    FROM Dirty_Sales_Dataset

```

```

UNION ALL

-- RECURSIVE MEMBER
SELECT date(Seq_Date, '+1 day')
FROM Order_Sequence
WHERE Seq_Date < (
    SELECT MAX(Order_Date)
    FROM Dirty_Sales_Dataset
)
)
SELECT COUNT(*) AS Total_Days_In_Range
FROM Order_Sequence;

-- Query 5 --
-- Multi-Column Anchor and Recursive Member: Employee Hierarchy

WITH RECURSIVE Employee_Hierarchy AS (
    -- ANCHOR MEMBER
    SELECT
        Salesperson,
        Reports_To,
        1 AS Level,
        Salesperson AS Path
    FROM Salesperson_Info
    WHERE Reports_To IS NULL

    UNION ALL

    -- RECURSIVE MEMBER
    SELECT
        s.Salesperson,
        s.Reports_To,
        eh.Level + 1,
        eh.Path || ' > ' || s.Salesperson
    FROM Salesperson_Info s
    JOIN Employee_Hierarchy eh
    ON s.Reports_To = eh.Salesperson
)
SELECT *
FROM Employee_Hierarchy
ORDER BY Level;

-- Query 6 --
-- Multiple Starting Rows in Anchor Member

WITH RECURSIVE Multi_Start AS (
    -- ANCHOR MEMBER
    SELECT
        Salesperson,
        Reports_To,
        1 AS Level
    FROM Salesperson_Info

```

```

WHERE Region_Assigned IN ('North', 'South')

UNION ALL

-- RECURSIVE MEMBER
SELECT
    s.Salesperson,
    s.Reports_To,
    m.Level + 1
FROM Salesperson_Info s
JOIN Multi_Start m
    ON s.Reports_To = m.Salesperson
)
SELECT *
FROM Multi_Start;

-- Query 7 --
-- Incorrect: Column Count Mismatch

WITH RECURSIVE Bad_Example AS (
    -- ANCHOR MEMBER: 2 columns
    SELECT
        1 AS Num,
        'Start' AS Label

    UNION ALL

    -- RECURSIVE MEMBER: 3 columns
    SELECT
        Num + 1,
        'Next',
        'Extra'
    FROM Bad_Example
    WHERE Num < 5
)
SELECT *
FROM Bad_Example;

-- Query 8 --
-- Incorrect: Multiple Self-References in Recursive Member

WITH RECURSIVE Bad AS (
    SELECT 1 AS N

    UNION ALL

    SELECT
        a.N + b.N
    FROM Bad a, Bad b
    WHERE a.N < 5
);

```

```
-- Query 9 --  
-- Anchor Member with JOIN and Aggregate
```

```
WITH RECURSIVE Example AS (  
    -- ANCHOR MEMBER  
    SELECT  
        Region,  
        MIN(Order_Date) AS First_Date  
    FROM Dirty_Sales_Dataset  
    GROUP BY Region  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER  
    SELECT  
        Region,  
        date(First_Date, '+30 days')  
    FROM Example  
    WHERE First_Date < date('2023-12-31')  
)  
SELECT *  
FROM Example;
```

```
-- Query 10 --  
-- Anchor Member Separately: Dataset Starting Date
```

```
SELECT MIN(Order_Date) AS Start_Date  
FROM Dirty_Sales_Dataset;
```

```
-- Query 11 --  
-- Recursive Employee Chain with Full Path
```

```
WITH RECURSIVE Employee_Chain AS (  
    -- ANCHOR MEMBER  
    SELECT  
        Salesperson,  
        Reports_To,  
        1 AS Level,  
        Salesperson AS Full_Path  
    FROM Salesperson_Info  
    WHERE Reports_To IS NULL  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER  
    SELECT  
        s.Salesperson,  
        s.Reports_To,  
        ec.Level + 1,  
        ec.Full_Path || ' -> ' || s.Salesperson  
    FROM Salesperson_Info s  
    JOIN Employee_Chain ec  
    ON s.Reports_To = ec.Salesperson
```

```
)
SELECT *
FROM Employee_Chain
ORDER BY Level;
```

D2.2.3 Traversing Organizational **Hierarchy** (Manager → Employee **Chain**)

```
-- Query 1 --
-- Setup: Create Salesperson_Info Table
```

```
CREATE TABLE Salesperson_Info (
    Salesperson TEXT PRIMARY KEY,
    Region_Assigned TEXT,
    Years_Experience INTEGER,
    Reports_To TEXT
);
```

```
-- Query 2 --
-- Setup: Insert Organizational Hierarchy Data
```

```
INSERT INTO Salesperson_Info VALUES
('Ravi Kumar', 'North', 10, NULL),
('Anjali Mehta', 'South', 7, 'Ravi Kumar'),
('Suresh Rao', 'East', 8, 'Ravi Kumar'),
('Priya Nair', 'West', 4, 'Anjali Mehta'),
('Karan Singh', 'North', 2, 'Priya Nair'),
('Meera Joshi', 'South', 3, 'Anjali Mehta'),
('Rohit Verma', 'East', 5, 'Suresh Rao');
```

```
-- Query 3 --
-- Downward Traversal: Complete Organizational Hierarchy
```

```
WITH RECURSIVE Org_Chart AS (
    -- ANCHOR: Top-level employees
    SELECT
        Salesperson,
        Reports_To,
        1 AS Level
    FROM Salesperson_Info
    WHERE Reports_To IS NULL

    UNION ALL

    -- RECURSIVE: Find employees at lower levels
    SELECT
        s.Salesperson,
        s.Reports_To,
        oc.Level + 1
    FROM Salesperson_Info s
    JOIN Org_Chart oc
        ON s.Reports_To = oc.Salesperson
)
```

```
SELECT *
FROM Org_Chart
ORDER BY Level, Salesperson;
```

```
-- Query 4 --
-- Specific Manager's Complete Subtree
```

```
WITH RECURSIVE Anjali_Team AS (
  -- ANCHOR: Start from Anjali Mehta
  SELECT
    Salesperson,
    Reports_To,
    1 AS Level
  FROM Salesperson_Info
  WHERE Salesperson = 'Anjali Mehta'

  UNION ALL

  -- RECURSIVE: Find direct and indirect reports
  SELECT
    s.Salesperson,
    s.Reports_To,
    at.Level + 1
  FROM Salesperson_Info s
  JOIN Anjali_Team at
    ON s.Reports_To = at.Salesperson
)
SELECT *
FROM Anjali_Team
ORDER BY Level;
```

```
-- Query 5 --
-- Upward Traversal: Chain of Command
```

```
WITH RECURSIVE Chain_Of_Command AS (
  -- ANCHOR: Start from Karan Singh
  SELECT
    Salesperson,
    Reports_To,
    1 AS Level
  FROM Salesperson_Info
  WHERE Salesperson = 'Karan Singh'

  UNION ALL

  -- RECURSIVE: Find manager and manager's manager
  SELECT
    s.Salesperson,
    s.Reports_To,
    coc.Level + 1
  FROM Salesperson_Info s
  JOIN Chain_Of_Command coc
    ON s.Salesperson = coc.Reports_To
```

```
)
SELECT *
FROM Chain_Of_Command
ORDER BY Level;
```

```
-- Query 6 --
-- Complete Hierarchy with Full Path
```

```
WITH RECURSIVE Org_Path AS (
  -- ANCHOR: Top-level employees
  SELECT
    Salesperson,
    Reports_To,
    1 AS Level,
    Salesperson AS Full_Path
  FROM Salesperson_Info
  WHERE Reports_To IS NULL

  UNION ALL

  -- RECURSIVE: Extend the hierarchy path
  SELECT
    s.Salesperson,
    s.Reports_To,
    op.Level + 1,
    op.Full_Path || ' → ' || s.Salesperson
  FROM Salesperson_Info s
  JOIN Org_Path op
    ON s.Reports_To = op.Salesperson
)
SELECT
  Salesperson,
  Level,
  Full_Path
FROM Org_Path
ORDER BY Level, Salesperson;
```

```
-- Query 7 --
-- Full Hierarchy: Track Top Manager
```

```
WITH RECURSIVE Full_Hierarchy AS (
  -- ANCHOR: Top-level employees
  SELECT
    Salesperson,
    Reports_To,
    Salesperson AS Top_Manager
  FROM Salesperson_Info
  WHERE Reports_To IS NULL

  UNION ALL

  -- RECURSIVE: Pass the top manager to lower levels
  SELECT
```

```

        s.Salesperson,
        s.Reports_To,
        fh.Top_Manager
    FROM Salesperson_Info s
    JOIN Full_Hierarchy fh
        ON s.Reports_To = fh.Salesperson
)
SELECT *
FROM Full_Hierarchy;

```

```

-- Query 8 --
-- Advanced: Total Direct and Indirect Reports

```

```

WITH RECURSIVE Full_Hierarchy AS (
    SELECT
        Salesperson,
        Reports_To,
        Salesperson AS Top_Manager
    FROM Salesperson_Info
    WHERE Reports_To IS NULL

    UNION ALL

    SELECT
        s.Salesperson,
        s.Reports_To,
        fh.Top_Manager
    FROM Salesperson_Info s
    JOIN Full_Hierarchy fh
        ON s.Reports_To = fh.Salesperson
),
Manager_Downline AS (
    WITH RECURSIVE Upward AS (
        -- ANCHOR: Every employee starts as their own ancestor
        SELECT
            Salesperson AS Employee,
            Salesperson AS Ancestor
        FROM Salesperson_Info

        UNION ALL

        -- RECURSIVE: Find each employee's manager
        SELECT
            u.Employee,
            s.Reports_To
        FROM Upward u
        JOIN Salesperson_Info s
            ON u.Ancestor = s.Salesperson
        WHERE s.Reports_To IS NOT NULL
    )
    SELECT
        Employee,
        Ancestor
    FROM Upward

```

```

        WHERE Employee != Ancestor
    )
    SELECT
        Ancestor AS Manager,
        COUNT(*) AS Total_Reports_Including_Indirect
    FROM Manager_Downline
    GROUP BY Ancestor
    ORDER BY Total_Reports_Including_Indirect DESC;

```

```

-- Query 9 --
-- Simpler Version: Total Team Size of One Manager

```

```

WITH RECURSIVE Team_Members AS (
    -- ANCHOR: Start from Ravi Kumar
    SELECT
        Salesperson,
        Reports_To,
        0 AS Level
    FROM Salesperson_Info
    WHERE Salesperson = 'Ravi Kumar'

    UNION ALL

    -- RECURSIVE: Find all direct and indirect reports
    SELECT
        s.Salesperson,
        s.Reports_To,
        tm.Level + 1
    FROM Salesperson_Info s
    JOIN Team_Members tm
        ON s.Reports_To = tm.Salesperson
)
SELECT COUNT(*) - 1 AS Total_Team_Size
FROM Team_Members;

```

```

-- Query 10 --
-- Setup: Recreate Salesperson_Info Table

```

```

DROP TABLE IF EXISTS Salesperson_Info;

```

```

CREATE TABLE Salesperson_Info (
    Salesperson TEXT PRIMARY KEY,
    Region_Assigned TEXT,
    Reports_To TEXT
);

```

```

INSERT INTO Salesperson_Info VALUES
('Ravi Kumar', 'North', NULL),
('Anjali Mehta', 'South', 'Ravi Kumar'),
('Suresh Rao', 'East', 'Ravi Kumar'),
('Priya Nair', 'West', 'Anjali Mehta'),
('Karan Singh', 'North', 'Priya Nair'),
('Meera Joshi', 'South', 'Anjali Mehta'),

```

```
('Rohit Verma', 'East', 'Suresh Rao');
```

```
-- Query 11 --
```

```
-- Full Path Organizational Chart
```

```
WITH RECURSIVE Org_Path AS (  
    -- ANCHOR: Top-level employees  
    SELECT  
        Salesperson,  
        Reports_To,  
        1 AS Level,  
        Salesperson AS Full_Path  
    FROM Salesperson_Info  
    WHERE Reports_To IS NULL  
  
    UNION ALL  
  
    -- RECURSIVE: Build the full path  
    SELECT  
        s.Salesperson,  
        s.Reports_To,  
        op.Level + 1,  
        op.Full_Path || ' -> ' || s.Salesperson  
    FROM Salesperson_Info s  
    JOIN Org_Path op  
        ON s.Reports_To = op.Salesperson  
)  
SELECT  
    Salesperson,  
    Level,  
    Full_Path  
FROM Org_Path  
ORDER BY Level, Salesperson;
```

```
-- Query 12 --
```

```
-- Downward Traversal: Show Only Top Two Levels
```

```
WITH RECURSIVE Org_Chart AS (  
    SELECT  
        Salesperson,  
        Reports_To,  
        1 AS Level  
    FROM Salesperson_Info  
    WHERE Reports_To IS NULL  
  
    UNION ALL  
  
    SELECT  
        s.Salesperson,  
        s.Reports_To,  
        oc.Level + 1  
    FROM Salesperson_Info s  
    JOIN Org_Chart oc
```

```

        ON s.Reports_To = oc.Salesperson
    )
    SELECT *
    FROM Org_Chart
    WHERE Level <= 2
    ORDER BY Level, Salesperson;

```

D2.2.4 Generating Number Series / Date Series Using Recursive CTE

```

-- Query 1 --
-- Basic Number Series: 1 to 100

```

```

WITH RECURSIVE Number_Series AS (
    SELECT 1 AS N

    UNION ALL

    SELECT N + 1
    FROM Number_Series
    WHERE N < 100
)
SELECT *
FROM Number_Series;

```

```

-- Query 2 --
-- Number Series with Custom Start, End, and Step Size

```

```

WITH RECURSIVE Number_Series AS (
    SELECT 50 AS N

    UNION ALL

    SELECT N + 5
    FROM Number_Series
    WHERE N < 100
)
SELECT *
FROM Number_Series;

```

```

-- Query 3 --
-- Basic Date Series: Complete Year 2023

```

```

WITH RECURSIVE Date_Series AS (
    SELECT date('2023-01-01') AS Calendar_Date

    UNION ALL

    SELECT date(Calendar_Date, '+1 day')
    FROM Date_Series
    WHERE Calendar_Date < date('2023-12-31')
)
SELECT *

```

```
FROM Date_Series;
```

```
-- Query 4 --
```

```
-- Dynamic Date Series Using Dataset's Actual Date Range
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT MIN(Order_Date) AS Calendar_Date  
    FROM Dirty_Sales_Dataset
```

```
    UNION ALL
```

```
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < (  
        SELECT MAX(Order_Date)  
        FROM Dirty_Sales_Dataset  
    )  
)
```

```
SELECT *  
FROM Date_Series;
```

```
-- Query 5 --
```

```
-- Missing Dates: Complete Gap Analysis
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT MIN(Order_Date) AS Calendar_Date  
    FROM Dirty_Sales_Dataset
```

```
    UNION ALL
```

```
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < (  
        SELECT MAX(Order_Date)  
        FROM Dirty_Sales_Dataset  
    )  
)
```

```
SELECT  
    ds.Calendar_Date,  
    COALESCE(d.Order_Count, 0) AS Orders_On_This_Date  
FROM Date_Series ds  
LEFT JOIN (  
    SELECT  
        Order_Date,  
        COUNT(*) AS Order_Count  
    FROM Dirty_Sales_Dataset  
    GROUP BY Order_Date  
) d  
ON ds.Calendar_Date = d.Order_Date  
WHERE d.Order_Date IS NULL  
ORDER BY ds.Calendar_Date;
```

```
-- Query 6 --  
-- Complete Daily Sales Report with Zero-Filled Dates
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT MIN(Order_Date) AS Calendar_Date  
    FROM Dirty_Sales_Dataset  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < (  
        SELECT MAX(Order_Date)  
        FROM Dirty_Sales_Dataset  
    )  
)  
SELECT  
    ds.Calendar_Date,  
    COALESCE(SUM(d.Sales), 0) AS Daily_Sales  
FROM Date_Series ds  
LEFT JOIN Dirty_Sales_Dataset d  
    ON ds.Calendar_Date = d.Order_Date  
GROUP BY ds.Calendar_Date  
ORDER BY ds.Calendar_Date  
LIMIT 40;
```

```
-- Query 7 --  
-- Month Series with Complete Monthly Sales Report
```

```
WITH RECURSIVE Month_Series AS (  
    SELECT date('2023-01-01') AS Month_Start  
  
    UNION ALL  
  
    SELECT date(Month_Start, '+1 month')  
    FROM Month_Series  
    WHERE Month_Start < date('2025-06-01')  
)  
SELECT  
    strftime('%Y-%m', ms.Month_Start) AS Sales_Month,  
    COALESCE(SUM(d.Sales), 0) AS Monthly_Sales  
FROM Month_Series ms  
LEFT JOIN Dirty_Sales_Dataset d  
    ON strftime('%Y-%m', d.Order_Date)  
    = strftime('%Y-%m', ms.Month_Start)  
GROUP BY Sales_Month  
ORDER BY Sales_Month;
```

```
-- Query 8 --  
-- Number Series for Dynamic Sales Buckets
```

```
WITH RECURSIVE Sales_Buckets AS (  
    SELECT
```

```

        0 AS Bucket_Start,
        1000 AS Bucket_End

    UNION ALL

    SELECT
        Bucket_End,
        Bucket_End + 1000
    FROM Sales_Buckets
    WHERE Bucket_End < 10000
)
SELECT
    sb.Bucket_Start,
    sb.Bucket_End,
    COUNT(d.Order_ID) AS Orders_In_Bucket
FROM Sales_Buckets sb
LEFT JOIN Dirty_Sales_Dataset d
    ON d.Sales >= sb.Bucket_Start
    AND d.Sales < sb.Bucket_End
GROUP BY
    sb.Bucket_Start,
    sb.Bucket_End
ORDER BY sb.Bucket_Start;

-- Query 9 --
-- Weekly Series: Seven-Day Intervals

WITH RECURSIVE Week_Series AS (
    SELECT MIN(Order_Date) AS Week_Start
    FROM Dirty_Sales_Dataset

    UNION ALL

    SELECT date(Week_Start, '+7 days')
    FROM Week_Series
    WHERE Week_Start < (
        SELECT MAX(Order_Date)
        FROM Dirty_Sales_Dataset
    )
)
SELECT
    ws.Week_Start,
    date(ws.Week_Start, '+6 days') AS Week_End,
    COALESCE(SUM(d.Sales), 0) AS Weekly_Sales
FROM Week_Series ws
LEFT JOIN Dirty_Sales_Dataset d
    ON d.Order_Date BETWEEN
        ws.Week_Start
        AND date(ws.Week_Start, '+6 days')
GROUP BY ws.Week_Start
ORDER BY ws.Week_Start
LIMIT 20;

```

```
-- Query 10 --  
-- Number Series: Increment Example
```

```
WITH RECURSIVE Number_Series AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Number_Series  
    WHERE N < 100  
)  
SELECT *  
FROM Number_Series;
```

```
-- Query 11 --  
-- Date Series: Increment Example
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT date('2023-01-01') AS Calendar_Date  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < '2023-12-31'  
)  
SELECT *  
FROM Date_Series;
```

```
-- Query 12 --  
-- LEFT JOIN: Show Missing Dates
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT MIN(Order_Date) AS Calendar_Date  
    FROM Dirty_Sales_Dataset  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < (  
        SELECT MAX(Order_Date)  
        FROM Dirty_Sales_Dataset  
    )  
)  
SELECT  
    ds.Calendar_Date,  
    d.Order_ID,  
    d.Sales  
FROM Date_Series ds  
LEFT JOIN Dirty_Sales_Dataset d  
    ON ds.Calendar_Date = d.Order_Date;
```

```
-- Query 13 --  
-- PRAGMA: Recursive Trigger Setting
```

```
PRAGMA recursive_triggers = ON;
```

```
-- Query 14 --  
-- Count Total Calendar Days in Dataset Date Range
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT MIN(Order_Date) AS Calendar_Date  
    FROM Dirty_Sales_Dataset  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < (  
        SELECT MAX(Order_Date)  
        FROM Dirty_Sales_Dataset  
    )  
)  
SELECT COUNT(*) AS Total_Calendar_Days  
FROM Date_Series;
```

```
-- Query 15 --  
-- Count Distinct Days with Orders
```

```
SELECT  
    COUNT(DISTINCT Order_Date) AS Days_With_Orders  
FROM Dirty_Sales_Dataset;
```

D2.2.5 MAXRECURSION – Preventing Infinite Loops

```
-- Query 1 --  
-- SQL Server: MAXRECURSION Safety Limit
```

```
WITH Numbers AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Numbers  
    WHERE N < 1000000  
)  
SELECT *  
FROM Numbers  
OPTION (MAXRECURSION 100);
```

```
-- Query 2 --  
-- SQL Server: Remove Recursion Limit (Risky)
```

```
WITH Numbers AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Numbers  
    WHERE N < 1000000  
)  
SELECT *  
FROM Numbers  
OPTION (MAXRECURSION 0);
```

```
-- Query 3 --  
-- SQLite: Number Series with LIMIT Safety Net
```

```
WITH RECURSIVE Numbers AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Numbers  
    WHERE N < 1000000  
)  
SELECT *  
FROM Numbers  
LIMIT 1000;
```

```
-- Query 4 --  
-- Dataset: Date Series with WHERE and LIMIT
```

```
WITH RECURSIVE Date_Series AS (  
    SELECT MIN(Order_Date) AS Calendar_Date  
    FROM Dirty_Sales_Dataset  
  
    UNION ALL  
  
    SELECT date(Calendar_Date, '+1 day')  
    FROM Date_Series  
    WHERE Calendar_Date < (  
        SELECT MAX(Order_Date)  
        FROM Dirty_Sales_Dataset  
    )  
)  
SELECT *  
FROM Date_Series  
LIMIT 5000;
```

```
-- Query 5 --  
-- Risky Example: Missing WHERE Condition
```

```
WITH RECURSIVE Bad_Example AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Bad_Example  
)  
SELECT *  
FROM Bad_Example  
LIMIT 10;
```

```
-- Query 6 --  
-- Employee Hierarchy with Explicit Level Limit
```

```
WITH RECURSIVE Employee_Hierarchy AS (  
    -- ANCHOR MEMBER  
    SELECT  
        Salesperson,  
        Reports_To,  
        1 AS Level  
    FROM Salesperson_Info  
    WHERE Reports_To IS NULL  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER  
    SELECT  
        s.Salesperson,  
        s.Reports_To,  
        eh.Level + 1  
    FROM Salesperson_Info s  
    JOIN Employee_Hierarchy eh  
        ON s.Reports_To = eh.Salesperson  
    WHERE eh.Level < 20  
)  
SELECT *  
FROM Employee_Hierarchy  
ORDER BY Level;
```

```
-- Query 7 --  
-- Universal Level-Based Safety Pattern
```

```
WITH RECURSIVE Safe_Recursive AS (  
    -- ANCHOR MEMBER  
    SELECT  
        ...,  
        1 AS Level  
    FROM table_name  
    WHERE ...
```

```

UNION ALL

-- RECURSIVE MEMBER
SELECT
    ...,
    sr.Level + 1
FROM table_name t
JOIN Safe_Recursive sr
    ON ...
WHERE sr.Level < 50
)
SELECT *
FROM Safe_Recursive;

-- Query 8 --
-- Testing: Small Recursion Range

WITH RECURSIVE Test_Numbers AS (
    SELECT 1 AS N

    UNION ALL

    SELECT N + 1
    FROM Test_Numbers
    WHERE N < 10
)
SELECT *
FROM Test_Numbers;

-- Query 9 --
-- Production Example: Larger Recursion Range

WITH RECURSIVE Production_Numbers AS (
    SELECT 1 AS N

    UNION ALL

    SELECT N + 1
    FROM Production_Numbers
    WHERE N < 10000
)
SELECT *
FROM Production_Numbers;

-- Query 10 --
-- Safe Numbers: WHERE Condition with Level Control

WITH RECURSIVE Safe_Numbers AS (
    SELECT
        1 AS N,
        1 AS Level

```

```

UNION ALL

SELECT
    N + 1,
    Level + 1
FROM Safe_Numbers
WHERE N < 50
    AND Level < 50
)
SELECT COUNT(*) AS Total_Numbers_Generated
FROM Safe_Numbers;

-- Query 11 --
-- Risky Numbers: Missing WHERE Condition with LIMIT

WITH RECURSIVE Risky_Numbers AS (
    SELECT 1 AS N

    UNION ALL

    SELECT N + 1
    FROM Risky_Numbers
)
SELECT COUNT(*) AS Total
FROM Risky_Numbers
LIMIT 100;

-- Query 12 --
-- Date Series with Explicit Maximum Depth

WITH RECURSIVE Safe_Date_Series AS (
    SELECT
        MIN(Order_Date) AS Calendar_Date,
        1 AS Level
    FROM Dirty_Sales_Dataset

    UNION ALL

    SELECT
        date(Calendar_Date, '+1 day'),
        Level + 1
    FROM Safe_Date_Series
    WHERE Calendar_Date < (
        SELECT MAX(Order_Date)
        FROM Dirty_Sales_Dataset
    )
    AND Level < 5000
)
SELECT *
FROM Safe_Date_Series;

```

```
-- Query 13 --  
-- Compare WHERE and LIMIT: Logical Stopping Condition
```

```
WITH RECURSIVE Number_Series AS (  
    SELECT 1 AS N  
  
    UNION ALL  
  
    SELECT N + 1  
    FROM Number_Series  
    WHERE N < 100  
)  
SELECT *  
FROM Number_Series  
LIMIT 10;
```

```
-- Query 14 --  
-- SQLite Recursive Trigger Setting
```

```
PRAGMA recursive_triggers = ON;
```

D3.1.1 PIVOT – Rows to Columns

```
-- Query 1: Native PIVOT Syntax (SQL Server / Oracle Reference) --
```

```
SELECT  
    Region,  
    Electronics,  
    Furniture,  
    Clothing  
FROM (  
    SELECT  
        Region,  
        Category,  
        Sales  
    FROM Dirty_Sales_Dataset  
) AS source_table  
PIVOT (  
    SUM(Sales)  
    FOR Category IN (Electronics, Furniture, Clothing)  
) AS pivot_table;
```

```
-- Query 2: Manual PIVOT Using CASE WHEN (SQLite-Friendly) --
```

```
SELECT  
    Region,  
    SUM(CASE  
        WHEN Category = 'Electronics' THEN Sales  
        ELSE 0  
    END) AS Electronics,  
    SUM(CASE  
        WHEN Category = 'Furniture' THEN Sales  
        ELSE 0  
    END) AS Furniture
```

```

        END) AS Furniture,
    SUM(CASE
        WHEN Category = 'Clothing' THEN Sales
        ELSE 0
    END) AS Clothing
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 3: Payment_Method Ko Columns Mein Convert Karna --

```

SELECT
    Region,
    COUNT(CASE
        WHEN Payment_Method = 'Credit Card' THEN 1
    END) AS Credit_Card_Orders,
    COUNT(CASE
        WHEN Payment_Method = 'UPI' THEN 1
    END) AS UPI_Orders,
    COUNT(CASE
        WHEN Payment_Method = 'Cash' THEN 1
    END) AS Cash_Orders
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 4: Year-Wise Sales Ko Columns Mein Convert Karna --

```

SELECT
    Category,
    SUM(CASE
        WHEN strftime('%Y', Order_Date) = '2023' THEN Sales
        ELSE 0
    END) AS Sales_2023,
    SUM(CASE
        WHEN strftime('%Y', Order_Date) = '2024' THEN Sales
        ELSE 0
    END) AS Sales_2024,
    SUM(CASE
        WHEN strftime('%Y', Order_Date) = '2025' THEN Sales
        ELSE 0
    END) AS Sales_2025
FROM Dirty_Sales_Dataset
GROUP BY Category;

```

-- Query 5: Order_Status Ko Columns Mein Convert Karna --

```

SELECT
    Region,
    COUNT(CASE
        WHEN Order_Status = 'Delivered' THEN 1
    END) AS Delivered,
    COUNT(CASE
        WHEN Order_Status = 'Cancelled' THEN 1
    END) AS Cancelled
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

```

END) AS Cancelled,
COUNT(CASE
    WHEN Order_Status = 'Returned' THEN 1
END) AS Returned,
COUNT(*) AS Total_Orders,
ROUND(
    COUNT(CASE
        WHEN Order_Status = 'Cancelled' THEN 1
    END) * 100.0 / COUNT(*),
    2
) AS Cancellation_Rate_Percent
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 6: Ship_Mode Ko Columns Mein Convert Karna --

```

SELECT
    Category,
    ROUND(
        AVG(CASE
            WHEN Ship_Mode = 'Standard' THEN Sales
        END),
        2
    ) AS Avg_Standard_Sales,
    ROUND(
        AVG(CASE
            WHEN Ship_Mode = 'Express' THEN Sales
        END),
        2
    ) AS Avg_Express_Sales,
    ROUND(
        AVG(CASE
            WHEN Ship_Mode = 'Same Day' THEN Sales
        END),
        2
    ) AS Avg_SameDay_Sales
FROM Dirty_Sales_Dataset
GROUP BY Category;

```

-- Query 7: SUM() Mein ELSE 0 Ka Use --

```

SELECT
    Region,
    SUM(
        CASE
            WHEN Category = 'Electronics' THEN Sales
            ELSE 0
        END
    ) AS Electronics_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 8: COUNT() Mein ELSE Ki Zaroorat Nahi --

```
SELECT
    Region,
    COUNT(
        CASE
            WHEN Category = 'Electronics' THEN 1
        END
    ) AS Electronics_Orders
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 9: AVG() Mein ELSE 0 Avoid Karna --

```
SELECT
    Category,
    AVG(
        CASE
            WHEN Ship_Mode = 'Standard' THEN Sales
        END
    ) AS Avg_Standard_Sales
FROM Dirty_Sales_Dataset
GROUP BY Category;
```

-- Query 10: Complete Cross-Tab Report with Total Sales --

```
SELECT
    Region,
    SUM(
        CASE
            WHEN Category = 'Electronics' THEN Sales
            ELSE 0
        END
    ) AS Electronics,
    SUM(
        CASE
            WHEN Category = 'Furniture' THEN Sales
            ELSE 0
        END
    ) AS Furniture,
    SUM(
        CASE
            WHEN Category = 'Clothing' THEN Sales
            ELSE 0
        END
    ) AS Clothing,
    SUM(Sales) AS Total_All_Categories
FROM Dirty_Sales_Dataset
GROUP BY Region
ORDER BY Total_All_Categories DESC;
```

D3.1.2 Manual PIVOT Using CASE + GROUP BY

-- Query 1: Normal GROUP BY – Row Format --

```
SELECT
    Region,
    Payment_Method,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region, Payment_Method
ORDER BY Region, Payment_Method;
```

-- Query 2: Check Distinct Payment Methods --

```
SELECT DISTINCT
    Payment_Method
FROM Dirty_Sales_Dataset;
```

-- Query 3: Manual PIVOT Using CASE WHEN --

```
SELECT
    Region,
    SUM(CASE
        WHEN Payment_Method = 'Cash' THEN Sales
        ELSE 0
    END) AS Cash,
    SUM(CASE
        WHEN Payment_Method = 'Credit Card' THEN Sales
        ELSE 0
    END) AS Credit_Card,
    SUM(CASE
        WHEN Payment_Method = 'UPI' THEN Sales
        ELSE 0
    END) AS UPI,
    SUM(CASE
        WHEN Payment_Method = 'Debit Card' THEN Sales
        ELSE 0
    END) AS Debit_Card,
    SUM(CASE
        WHEN Payment_Method = 'Net Banking' THEN Sales
        ELSE 0
    END) AS Net_Banking
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 4: Check Distinct Segments --

```
SELECT DISTINCT
    Segment
FROM Dirty_Sales_Dataset;
```

-- Query 5: Segment-Wise Sales Pivot --

```

SELECT
    Category,
    SUM(CASE
        WHEN Segment = 'Consumer' THEN Sales
        ELSE 0
    END) AS Consumer,
    SUM(CASE
        WHEN Segment = 'Corporate' THEN Sales
        ELSE 0
    END) AS Corporate,
    SUM(CASE
        WHEN Segment = 'Home Office' THEN Sales
        ELSE 0
    END) AS Home_Office,
    SUM(CASE
        WHEN Segment = 'Small Business' THEN Sales
        ELSE 0
    END) AS Small_Business
FROM Dirty_Sales_Dataset
GROUP BY Category;

```

-- Query 6: Count-Based Pivot – Order Status --

```

SELECT
    Region,
    COUNT(CASE
        WHEN Order_Status = 'Delivered' THEN 1
    END) AS Delivered,
    COUNT(CASE
        WHEN Order_Status = 'Pending' THEN 1
    END) AS Pending,
    COUNT(CASE
        WHEN Order_Status = 'Cancelled' THEN 1
    END) AS Cancelled,
    COUNT(CASE
        WHEN Order_Status = 'Returned' THEN 1
    END) AS Returned
FROM Dirty_Sales_Dataset
GROUP BY Region
ORDER BY Region;

```

-- Query 7: Multiple Row Dimensions – Region + Category --

```

SELECT
    Region,
    Category,
    SUM(CASE
        WHEN Payment_Method = 'Credit Card' THEN Sales
        ELSE 0
    END) AS Credit_Card,
    SUM(CASE
        WHEN Payment_Method = 'UPI' THEN Sales
        ELSE 0
    END) AS UPI

```

```

        END) AS UPI,
        SUM(CASE
            WHEN Payment_Method = 'Cash' THEN Sales
            ELSE 0
        END) AS Cash
FROM Dirty_Sales_Dataset
GROUP BY Region, Category
ORDER BY Region, Category;

```

-- Query 8: Percentage-Based Pivot --

```

SELECT
    Region,
    SUM(CASE
        WHEN Category = 'Electronics' THEN Sales
        ELSE 0
    END) AS Electronics,
    SUM(CASE
        WHEN Category = 'Furniture' THEN Sales
        ELSE 0
    END) AS Furniture,
    SUM(CASE
        WHEN Category = 'Clothing' THEN Sales
        ELSE 0
    END) AS Clothing,
    SUM(Sales) AS Total,
    ROUND(
        SUM(CASE
            WHEN Category = 'Electronics' THEN Sales
            ELSE 0
        END) * 100.0 / SUM(Sales),
        1
    ) AS Electronics_Percent
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 9: Ship_Mode Pivot with Grand Total Using UNION ALL --

```

SELECT
    Category,
    SUM(CASE
        WHEN Ship_Mode = 'Standard' THEN Sales
        ELSE 0
    END) AS Standard,
    SUM(CASE
        WHEN Ship_Mode = 'Express' THEN Sales
        ELSE 0
    END) AS Express,
    SUM(CASE
        WHEN Ship_Mode = 'Same Day' THEN Sales
        ELSE 0
    END) AS Same_Day
FROM Dirty_Sales_Dataset

```

```
GROUP BY Category
```

```
UNION ALL
```

```
SELECT
```

```
    'GRAND TOTAL' AS Category,  
    SUM(CASE  
        WHEN Ship_Mode = 'Standard' THEN Sales  
        ELSE 0  
    END),  
    SUM(CASE  
        WHEN Ship_Mode = 'Express' THEN Sales  
        ELSE 0  
    END),  
    SUM(CASE  
        WHEN Ship_Mode = 'Same Day' THEN Sales  
        ELSE 0  
    END)  
FROM Dirty_Sales_Dataset;
```

```
-- Query 10: Incorrect Alias with Space (Reference) --
```

```
SELECT
```

```
    Category,  
    SUM(CASE  
        WHEN Segment = 'Home Office' THEN Sales  
        ELSE 0  
    END) AS Home Office  
FROM Dirty_Sales_Dataset  
GROUP BY Category;
```

```
-- Query 11: Correct Alias Using Underscore --
```

```
SELECT
```

```
    Category,  
    SUM(CASE  
        WHEN Segment = 'Home Office' THEN Sales  
        ELSE 0  
    END) AS Home_Office  
FROM Dirty_Sales_Dataset  
GROUP BY Category;
```

```
-- Query 12: Alias with Quoted Column Name --
```

```
SELECT
```

```
    Category,  
    SUM(CASE  
        WHEN Segment = 'Home Office' THEN Sales  
        ELSE 0  
    END) AS "Home Office"  
FROM Dirty_Sales_Dataset  
GROUP BY Category;
```

-- Query 13: Data Value vs Column Alias --

```
SELECT
    Category,
    SUM(
        CASE
            WHEN Segment = 'Home Office' THEN Sales
            ELSE 0
        END
    ) AS Home_Office
FROM Dirty_Sales_Dataset
GROUP BY Category;
```

-- Query 14: Check Distinct Order Status Values --

```
SELECT DISTINCT
    Order_Status
FROM Dirty_Sales_Dataset;
```

-- Query 15: Final Order Status Pivot with Total Orders --

```
SELECT
    Region,
    COUNT(CASE
        WHEN Order_Status = 'Delivered' THEN 1
    END) AS Delivered,
    COUNT(CASE
        WHEN Order_Status = 'Pending' THEN 1
    END) AS Pending,
    COUNT(CASE
        WHEN Order_Status = 'Cancelled' THEN 1
    END) AS Cancelled,
    COUNT(CASE
        WHEN Order_Status = 'Returned' THEN 1
    END) AS Returned,
    COUNT(*) AS Total_Orders
FROM Dirty_Sales_Dataset
GROUP BY Region
ORDER BY Total_Orders DESC;
```

D3.1.3 Native PIVOT Syntax (SQL Server)

-- Query 1: Native PIVOT Syntax Structure (Reference) --

```
SELECT
    <non_pivoted_column>,
    <pivoted_column_1>,
    <pivoted_column_2>
FROM
(
    SELECT
```

```

        <non_pivoted_column>,
        <column_to_pivot>,
        <value_column>
    FROM table_name
) AS SourceTable
PIVOT
(
    AGG_FUNCTION(<value_column>)
    FOR <column_to_pivot> IN (
        [value1],
        [value2],
        [value3]
    )
) AS PivotTable;

```

-- Query 2: Region-Wise Category Sales Using Native PIVOT --

```

SELECT
    Region,
    Electronics,
    Furniture,
    Clothing
FROM
(
    SELECT
        Region,
        Category,
        Sales
    FROM Dirty_Sales_Dataset
) AS SourceTable
PIVOT
(
    SUM(Sales)
    FOR Category IN (
        [Electronics],
        [Furniture],
        [Clothing]
    )
) AS PivotTable;

```

-- Query 3: Region-Wise Category Sales Using Manual CASE WHEN --

```

SELECT
    Region,
    SUM(CASE
        WHEN Category = 'Electronics' THEN Sales
        ELSE 0
    END) AS Electronics,
    SUM(CASE
        WHEN Category = 'Furniture' THEN Sales
        ELSE 0
    END) AS Furniture,
    SUM(CASE

```

```

        WHEN Category = 'Clothing' THEN Sales
        ELSE 0
    END) AS Clothing
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 4: Payment Method Pivot Using COUNT() --

```

SELECT
    Region,
    Cash,
    [Credit Card],
    UPI
FROM
    (
        SELECT
            Region,
            Payment_Method,
            Order_ID
        FROM Dirty_Sales_Dataset
    ) AS SourceTable
PIVOT
(
    COUNT(Order_ID)
    FOR Payment_Method IN (
        [Cash],
        [Credit Card],
        [UPI]
    )
) AS PivotTable;

```

-- Query 5: Order Status Pivot Using COUNT() --

```

SELECT
    Region,
    Delivered,
    Cancelled,
    Pending,
    Returned
FROM
    (
        SELECT
            Region,
            Order_Status,
            Order_ID
        FROM Dirty_Sales_Dataset
    ) AS SourceTable
PIVOT
(
    COUNT(Order_ID)
    FOR Order_Status IN (
        [Delivered],
        [Cancelled],

```

```

        [Pending],
        [Returned]
    )
) AS PivotTable;

```

-- Query 6: Native PIVOT with Multiple Aggregate Functions (Invalid Reference) --

```

-- This syntax is not valid:
-- PIVOT (
--     SUM(Sales),
--     COUNT(Order_ID)
--     FOR Category IN (...)
-- );

```

-- Query 7: SQLite-Compatible Alternative for SUM and COUNT --

```

SELECT
    Region,
    SUM(CASE
        WHEN Category = 'Electronics' THEN Sales
        ELSE 0
    END) AS Electronics_Sales,
    COUNT(CASE
        WHEN Category = 'Electronics' THEN Order_ID
    END) AS Electronics_Orders
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 8: Static PIVOT – Explicit Category Values --

```

SELECT
    Region,
    Electronics,
    Furniture,
    Clothing
FROM
    (
        SELECT
            Region,
            Category,
            Sales
        FROM Dirty_Sales_Dataset
    ) AS SourceTable
PIVOT
(
    SUM(Sales)
    FOR Category IN (
        [Electronics],
        [Furniture],
        [Clothing]
    )
) AS PivotTable;

```

-- Query 9: SQLite PIVOT Syntax Check (Error Expected) --

```
SELECT
    Region,
    Electronics,
    Furniture
FROM Dirty_Sales_Dataset
PIVOT (
    SUM(Sales)
    FOR Category IN (
        Electronics,
        Furniture
    )
) AS PivotTable;
```

D3.1.4 Dynamic PIVOT – When Column Names Are Not Fixed

-- Query 1: Static PIVOT Limitation --

```
SELECT
    Region,
    SUM(CASE
        WHEN Payment_Method = 'Cash' THEN Sales
        ELSE 0
    END) AS Cash,
    SUM(CASE
        WHEN Payment_Method = 'Credit Card' THEN Sales
        ELSE 0
    END) AS Credit_Card,
    SUM(CASE
        WHEN Payment_Method = 'UPI' THEN Sales
        ELSE 0
    END) AS UPI
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 2: SQL Server – Collect Distinct Values Dynamically --

```
DECLARE @columns NVARCHAR(MAX) = '';

SELECT
    @columns += QUOTENAME(Payment_Method) + ', '
FROM (
    SELECT DISTINCT
        Payment_Method
    FROM Dirty_Sales_Dataset
) AS distinct_values;

SET @columns = LEFT(
    @columns,
    LEN(@columns) - 1
);
```

-- Query 3: SQL Server – Execute Dynamic PIVOT --

```
DECLARE @sql NVARCHAR(MAX);

SET @sql = '
SELECT
    Region,
    ' + @columns + '
FROM
(
    SELECT
        Region,
        Payment_Method,
        Sales
    FROM Dirty_Sales_Dataset
) AS SourceTable
PIVOT
(
    SUM(Sales)
    FOR Payment_Method IN ( ' + @columns + ' )
) AS PivotTable';

EXEC sp_executesql @sql;
```

-- Query 4: SQLite – Check Distinct Payment Methods --

```
SELECT DISTINCT
    Payment_Method
FROM Dirty_Sales_Dataset
ORDER BY Payment_Method;
```

-- Query 5: SQLite – Semi-Dynamic Manual PIVOT --

```
SELECT
    Region,
    SUM(CASE
        WHEN Payment_Method = 'Cash' THEN Sales
        ELSE 0
    END) AS Cash,
    SUM(CASE
        WHEN Payment_Method = 'Credit Card' THEN Sales
        ELSE 0
    END) AS Credit_Card,
    SUM(CASE
        WHEN Payment_Method = 'UPI' THEN Sales
        ELSE 0
    END) AS UPI
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 6: Check Segment Frequency --

```
SELECT
    Segment,
    COUNT(*) AS Frequency
FROM Dirty_Sales_Dataset
GROUP BY Segment
ORDER BY Frequency DESC;
```

-- Query 7: Segment-Wise Manual PIVOT --

```
SELECT
    Region,
    SUM(CASE
        WHEN Segment = 'Consumer' THEN Sales
        ELSE 0
    END) AS Consumer,
    SUM(CASE
        WHEN Segment = 'Corporate' THEN Sales
        ELSE 0
    END) AS Corporate,
    SUM(CASE
        WHEN Segment = 'Home Office' THEN Sales
        ELSE 0
    END) AS Home_Office,
    SUM(CASE
        WHEN Segment = 'Small Business' THEN Sales
        ELSE 0
    END) AS Small_Business
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 8: Check Distinct Categories Before Pivoting --

```
SELECT DISTINCT
    Category
FROM Dirty_Sales_Dataset
ORDER BY Category;
```

-- Query 9: Category-Wise Dynamic-Style Manual PIVOT --

```
SELECT
    Region,
    SUM(CASE
        WHEN Category = 'Electronics' THEN Sales
        ELSE 0
    END) AS Electronics,
    SUM(CASE
        WHEN Category = 'Furniture' THEN Sales
        ELSE 0
    END) AS Furniture,
    SUM(CASE
```

```

        WHEN Category = 'Clothing' THEN Sales
        ELSE 0
    END) AS Clothing
FROM Dirty_Sales_Dataset
GROUP BY Region;

```

-- Query 10: Dynamic PIVOT Security – Risky String Construction (Reference) --

```

-- Avoid directly inserting unsanitized user input into SQL strings.
-- Example pattern:
-- case_statements.append(
--     f"SUM(CASE WHEN Payment_Method = '{method}' "
--     f"THEN Sales ELSE 0 END) AS {safe_alias}"
-- );

```

-- Query 11: Dynamic PIVOT with a Total Column --

```

SELECT
    Region,
    SUM(CASE
        WHEN Payment_Method = 'Cash' THEN Sales
        ELSE 0
    END) AS Cash,
    SUM(CASE
        WHEN Payment_Method = 'Credit Card' THEN Sales
        ELSE 0
    END) AS Credit_Card,
    SUM(CASE
        WHEN Payment_Method = 'UPI' THEN Sales
        ELSE 0
    END) AS UPI,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region
ORDER BY Total_Sales DESC;

```

D3.2.1 UNPIVOT – Columns to Rows

-- Query 1: SQL Server Native UNPIVOT Syntax --

```

SELECT
    Region,
    Category,
    Sales
FROM
(
    SELECT
        Region,
        Electronics,
        Furniture,
        Clothing
    FROM Pivoted_Sales_Table
) AS SourceTable

```

```

UNPIVOT
(
    Sales FOR Category IN (
        Electronics,
        Furniture,
        Clothing
    )
) AS UnpivotedTable;

```

-- Query 2: Manual UNPIVOT Using UNION ALL – Basic Pattern --

```

SELECT
    Row_Identifier_Column,
    'Label11' AS New_Category_Column,
    Column1 AS New_Value_Column
FROM table_name

```

UNION ALL

```

SELECT
    Row_Identifier_Column,
    'Label12' AS New_Category_Column,
    Column2 AS New_Value_Column
FROM table_name

```

UNION ALL

```

SELECT
    Row_Identifier_Column,
    'Label13' AS New_Category_Column,
    Column3 AS New_Value_Column
FROM table_name;

```

-- Query 3: Create Wide/Pivoted Data Using CTE --

```

WITH Pivoted_Data AS (
    SELECT
        Region,
        SUM(CASE
            WHEN Category = 'Electronics' THEN Sales
            ELSE 0
        END) AS Electronics,
        SUM(CASE
            WHEN Category = 'Furniture' THEN Sales
            ELSE 0
        END) AS Furniture,
        SUM(CASE
            WHEN Category = 'Clothing' THEN Sales
            ELSE 0
        END) AS Clothing
    FROM Dirty_Sales_Dataset
    GROUP BY Region
)

```

```
SELECT *
FROM Pivoted_Data;
```

-- Query 4: UNPIVOT Pivoted Data Using UNION ALL --

```
WITH Pivoted_Data AS (
    SELECT
        Region,
        SUM(CASE
            WHEN Category = 'Electronics' THEN Sales
            ELSE 0
        END) AS Electronics,
        SUM(CASE
            WHEN Category = 'Furniture' THEN Sales
            ELSE 0
        END) AS Furniture,
        SUM(CASE
            WHEN Category = 'Clothing' THEN Sales
            ELSE 0
        END) AS Clothing
    FROM Dirty_Sales_Dataset
    GROUP BY Region
)
SELECT
    Region,
    'Electronics' AS Category,
    Electronics AS Sales
FROM Pivoted_Data

UNION ALL

SELECT
    Region,
    'Furniture' AS Category,
    Furniture AS Sales
FROM Pivoted_Data

UNION ALL

SELECT
    Region,
    'Clothing' AS Category,
    Clothing AS Sales
FROM Pivoted_Data

ORDER BY Region, Category;
```

-- Query 5: Create Quarterly Sales Wide Table --

```
DROP TABLE IF EXISTS Quarterly_Sales;
```

```
CREATE TABLE Quarterly_Sales (
    Region TEXT,
```

```
Q1_Sales REAL,  
Q2_Sales REAL,  
Q3_Sales REAL,  
Q4_Sales REAL  
);
```

```
-- Query 6: Insert Data into Quarterly Sales Table --
```

```
INSERT INTO Quarterly_Sales VALUES  
( 'North', 150000, 180000, 160000, 200000),  
( 'South', 130000, 140000, 135000, 155000);
```

```
-- Query 7: UNPIVOT Quarterly Sales Using UNION ALL --
```

```
SELECT  
    Region,  
    'Q1' AS Quarter,  
    Q1_Sales AS Sales  
FROM Quarterly_Sales
```

```
UNION ALL
```

```
SELECT  
    Region,  
    'Q2' AS Quarter,  
    Q2_Sales AS Sales  
FROM Quarterly_Sales
```

```
UNION ALL
```

```
SELECT  
    Region,  
    'Q3' AS Quarter,  
    Q3_Sales AS Sales  
FROM Quarterly_Sales
```

```
UNION ALL
```

```
SELECT  
    Region,  
    'Q4' AS Quarter,  
    Q4_Sales AS Sales  
FROM Quarterly_Sales
```

```
ORDER BY Region, Quarter;
```

```
-- Query 8: UNPIVOT Quarterly Data for Analysis --
```

```
SELECT  
    Quarter,  
    AVG(Sales) AS Average_Sales  
FROM (
```

```

SELECT
    Region,
    'Q1' AS Quarter,
    Q1_Sales AS Sales
FROM Quarterly_Sales

UNION ALL

SELECT
    Region,
    'Q2' AS Quarter,
    Q2_Sales AS Sales
FROM Quarterly_Sales

UNION ALL

SELECT
    Region,
    'Q3' AS Quarter,
    Q3_Sales AS Sales
FROM Quarterly_Sales

UNION ALL

SELECT
    Region,
    'Q4' AS Quarter,
    Q4_Sales AS Sales
FROM Quarterly_Sales
) AS Long_Format_Data
GROUP BY Quarter
ORDER BY Quarter;

```

-- Query 9: UNPIVOT Data While Excluding NULL Values --

```

SELECT
    Region,
    'Q3' AS Quarter,
    Q3_Sales AS Sales
FROM Quarterly_Sales
WHERE Q3_Sales IS NOT NULL;

```

-- Query 10: Create Monthly Sales Wide Table --

```

DROP TABLE IF EXISTS Monthly_Sales_Wide;

CREATE TABLE Monthly_Sales_Wide (
    Customer_ID TEXT,
    Jan_Sales REAL,
    Feb_Sales REAL,
    Mar_Sales REAL
);

```

-- Query 11: Insert Monthly Sales Data --

```
INSERT INTO Monthly_Sales_Wide VALUES
('CUST001', 5000, 6000, 4500),
('CUST002', 3000, 3500, 4000),
('CUST003', 7000, 6500, 8000);
```

-- Query 12: UNPIVOT Monthly Sales Using UNION ALL --

```
SELECT
    Customer_ID,
    'January' AS Month,
    Jan_Sales AS Sales
FROM Monthly_Sales_Wide
```

UNION ALL

```
SELECT
    Customer_ID,
    'February' AS Month,
    Feb_Sales AS Sales
FROM Monthly_Sales_Wide
```

UNION ALL

```
SELECT
    Customer_ID,
    'March' AS Month,
    Mar_Sales AS Sales
FROM Monthly_Sales_Wide
```

ORDER BY Customer_ID, Month;

-- Query 13: Monthly Long Format Analysis --

```
SELECT
    Month,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Average_Sales
FROM (
```

```
    SELECT
        Customer_ID,
        'January' AS Month,
        Jan_Sales AS Sales
    FROM Monthly_Sales_Wide
```

UNION ALL

```
    SELECT
        Customer_ID,
        'February' AS Month,
        Feb_Sales AS Sales
```

```

FROM Monthly_Sales_Wide

UNION ALL

SELECT
    Customer_ID,
    'March' AS Month,
    Mar_Sales AS Sales
FROM Monthly_Sales_Wide
) AS Long_Format_Data
GROUP BY Month
ORDER BY Month;

```

-- Query 14: UNPIVOT Using UNION ALL – Preserve Duplicate Values --

```

SELECT
    Region,
    'Electronics' AS Category,
    Electronics AS Sales
FROM Pivoted_Sales_Table

UNION ALL

SELECT
    Region,
    'Furniture' AS Category,
    Furniture AS Sales
FROM Pivoted_Sales_Table;

```

-- Query 15: UNPIVOT Wide Data with Matching Data Types --

```

SELECT
    Region,
    'Q1' AS Quarter,
    Q1_Sales AS Sales
FROM Quarterly_Sales

UNION ALL

SELECT
    Region,
    'Q2' AS Quarter,
    Q2_Sales AS Sales
FROM Quarterly_Sales

UNION ALL

SELECT
    Region,
    'Q3' AS Quarter,
    Q3_Sales AS Sales
FROM Quarterly_Sales

```

UNION ALL

SELECT

```
    Region,  
    'Q4' AS Quarter,  
    Q4_Sales AS Sales  
FROM Quarterly_Sales;
```

D3.2.2 Manual UNPIVOT using UNION ALL

-- Query 1: Create Wide Region Payment Summary Table --

```
CREATE TABLE Region_Payment_Summary (  
    Region TEXT,  
    Cash_Total REAL,  
    Credit_Card_Total REAL,  
    UPI_Total REAL  
);
```

INSERT INTO Region_Payment_Summary **VALUES**

```
('North', 50000, 120000, 80000),  
('South', 45000, 110000, 75000),  
('East', 40000, 95000, 60000),  
('West', 55000, 130000, 90000);
```

-- Query 2: Manual UNPIVOT using UNION ALL --

```
SELECT  
    Region,  
    'Cash' AS Payment_Method,  
    Cash_Total AS Total_Sales  
FROM Region_Payment_Summary
```

UNION ALL

SELECT

```
    Region,  
    'Credit Card' AS Payment_Method,  
    Credit_Card_Total AS Total_Sales  
FROM Region_Payment_Summary
```

UNION ALL

SELECT

```
    Region,  
    'UPI' AS Payment_Method,  
    UPI_Total AS Total_Sales  
FROM Region_Payment_Summary  
  
ORDER BY Region, Payment_Method;
```

-- Query 3: Create Yearly Category Sales Wide Table --

```
CREATE TABLE Yearly_Category_Sales (  
    Category TEXT,  
    Sales_2023 REAL,
```

```

        Sales_2024 REAL,
        Sales_2025 REAL
    );

INSERT INTO Yearly_Category_Sales VALUES
('Electronics', 1200000, 1450000, 980000),
('Furniture', 800000, 850000, 620000),
('Clothing', 600000, 720000, 510000);

-- Query 4: Unpivot Yearly Category Sales --
SELECT
    Category,
    '2023' AS Year,
    Sales_2023 AS Sales
FROM Yearly_Category_Sales

UNION ALL

SELECT
    Category,
    '2024' AS Year,
    Sales_2024 AS Sales
FROM Yearly_Category_Sales

UNION ALL

SELECT
    Category,
    '2025' AS Year,
    Sales_2025 AS Sales
FROM Yearly_Category_Sales

ORDER BY Category, Year;

-- Query 5: Create Region Category Status Wide Table --
CREATE TABLE Region_Category_Status (
    Region TEXT,
    Category TEXT,
    Delivered_Count INTEGER,
    Cancelled_Count INTEGER,
    Returned_Count INTEGER
);

INSERT INTO Region_Category_Status VALUES
('North', 'Electronics', 500, 30, 20),
('South', 'Electronics', 450, 25, 15);

-- Query 6: Unpivot with Multiple Identifier Columns --
SELECT
    Region,
    Category,
    'Delivered' AS Status,

```

```
    Delivered_Count AS Count
FROM Region_Category_Status
```

```
UNION ALL
```

```
SELECT
    Region,
    Category,
    'Cancelled' AS Status,
    Cancelled_Count AS Count
FROM Region_Category_Status
```

```
UNION ALL
```

```
SELECT
    Region,
    Category,
    'Returned' AS Status,
    Returned_Count AS Count
FROM Region_Category_Status
```

```
ORDER BY Region, Category, Status;
```

```
-- Query 7: Unpivot using CTE and Aggregate Data --
```

```
WITH Unpivoted_Data AS (
```

```
    SELECT
        Region,
        'Cash' AS Payment_Method,
        Cash_Total AS Total_Sales
    FROM Region_Payment_Summary
```

```
UNION ALL
```

```
SELECT
    Region,
    'Credit Card',
    Credit_Card_Total
FROM Region_Payment_Summary
```

```
UNION ALL
```

```
SELECT
    Region,
    'UPI',
    UPI_Total
FROM Region_Payment_Summary
```

```
)
```

```
SELECT
    Payment_Method,
    SUM(Total_Sales) AS Grand_Total_By_Payment_Method
FROM Unpivoted_Data
```

```
GROUP BY Payment_Method
ORDER BY Grand_Total_By_Payment_Method DESC;
```

```
-- Query 8: Create Product Monthly Stock Wide Table --
```

```
CREATE TABLE Product_Monthly_Stock (
    Product_Name TEXT,
    Jan_Stock INTEGER,
    Feb_Stock INTEGER,
    Mar_Stock INTEGER
);
```

```
INSERT INTO Product_Monthly_Stock VALUES
('Laptop', 100, NULL, 80),
('Mouse', 500, 450, 400);
```

```
-- Query 9: Unpivot Monthly Stock with NULL Filtering --
```

```
SELECT
    Product_Name,
    'Jan' AS Month,
    Jan_Stock AS Stock
FROM Product_Monthly_Stock
WHERE Jan_Stock IS NOT NULL
```

```
UNION ALL
```

```
SELECT
    Product_Name,
    'Feb' AS Month,
    Feb_Stock AS Stock
FROM Product_Monthly_Stock
WHERE Feb_Stock IS NOT NULL
```

```
UNION ALL
```

```
SELECT
    Product_Name,
    'Mar' AS Month,
    Mar_Stock AS Stock
FROM Product_Monthly_Stock
WHERE Mar_Stock IS NOT NULL
```

```
ORDER BY Product_Name, Month;
```

```
-- Query 10: Unpivot Monthly Stock without NULL Filtering --
```

```
SELECT
    Product_Name,
    'Jan' AS Month,
    Jan_Stock AS Stock
FROM Product_Monthly_Stock
```

```
UNION ALL
```

```
SELECT
    Product_Name,
    'Feb' AS Month,
    Feb_Stock AS Stock
FROM Product_Monthly_Stock
```

```
UNION ALL
```

```
SELECT
    Product_Name,
    'Mar' AS Month,
    Mar_Stock AS Stock
FROM Product_Monthly_Stock
```

```
ORDER BY Product_Name, Month;
```

```
-- Query 11: Incorrect UNION ALL - Column Count Mismatch --
```

```
SELECT
    Region,
    'Cash',
    Cash_Total
FROM Region_Payment_Summary
```

```
UNION ALL
```

```
SELECT
    Region,
    Credit_Card_Total
FROM Region_Payment_Summary;
```

```
-- Query 12: Correct UNION ALL - Same Number of Columns --
```

```
SELECT
    Region,
    'Cash' AS Payment_Method,
    Cash_Total AS Total_Sales
FROM Region_Payment_Summary
```

```
UNION ALL
```

```
SELECT
    Region,
    'Credit Card' AS Payment_Method,
    Credit_Card_Total AS Total_Sales
FROM Region_Payment_Summary;
```

```
-- Query 13: Correct String Literal Label --
```

```
SELECT
    Region,
    'Cash' AS Payment_Method,
    Cash_Total
FROM Region_Payment_Summary;
```

-- Query 14: Incorrect String Literal Label --

```
SELECT
    Region,
    Cash AS Payment_Method,
    Cash_Total
FROM Region_Payment_Summary;
```

-- Query 15: Create Region Category Sales Wide Table --

```
DROP TABLE IF EXISTS Region_Category_Sales_Wide;
```

```
CREATE TABLE Region_Category_Sales_Wide (
    Region TEXT,
    Electronics_Sales REAL,
    Furniture_Sales REAL,
    Clothing_Sales REAL
);
```

```
INSERT INTO Region_Category_Sales_Wide VALUES
('North', 500000, 300000, 200000),
('South', 450000, 280000, 180000),
('East', 400000, 250000, 160000),
('West', 550000, 320000, 210000);
```

-- Query 16: Unpivot and Aggregate Category Sales --

```
WITH Unpivoted AS (

    SELECT
        Region,
        'Electronics' AS Category,
        Electronics_Sales AS Sales
    FROM Region_Category_Sales_Wide

    UNION ALL

    SELECT
        Region,
        'Furniture',
        Furniture_Sales
    FROM Region_Category_Sales_Wide

    UNION ALL

    SELECT
        Region,
        'Clothing',
        Clothing_Sales
    FROM Region_Category_Sales_Wide

)

SELECT
    Category,
```

```

        SUM(Sales) AS Total_Sales,
        AVG(Sales) AS Avg_Sales_Per_Region
FROM Unpivoted
GROUP BY Category
ORDER BY Total_Sales DESC;

```

D3.2.3 Native UNPIVOT Syntax (SQL Server)

-- Query 1: Native UNPIVOT Syntax Structure --

```

SELECT
    <identifier-column>,
    <new-category-column>,
    <new-value-column>
FROM
(
    SELECT
        <identifier-column>,
        <value-column-1>,
        <value-column-2>
    FROM table_name
) AS SourceTable
UNPIVOT
(
    <new-value-column>
    FOR <new-category-column>
    IN (<value-column-1>, <value-column-2>)
) AS UnpivotTable;

```

-- Query 2: Create Region Payment Summary Table (SQL Server) --

```

CREATE TABLE Region_Payment_Summary (
    Region VARCHAR(50),
    Cash_Total DECIMAL(10,2),
    Credit_Card_Total DECIMAL(10,2),
    UPI_Total DECIMAL(10,2)
);

```

-- Query 3: Native UNPIVOT in SQL Server --

```

SELECT
    Region,
    Payment_Method,
    Total_Sales
FROM
(
    SELECT
        Region,
        Cash_Total,
        Credit_Card_Total,
        UPI_Total
    FROM Region_Payment_Summary
) AS SourceTable
UNPIVOT
(
    Total_Sales

```

```

        FOR Payment_Method IN (
            Cash_Total,
            Credit_Card_Total,
            UPI_Total
        )
    ) AS UnpivotTable;

```

-- Query 4: Manual UNPIVOT using UNION ALL (SQLite Compatible) --

```

SELECT
    Region,
    'Cash_Total' AS Payment_Method,
    Cash_Total AS Total_Sales
FROM Region_Payment_Summary

UNION ALL

SELECT
    Region,
    'Credit_Card_Total' AS Payment_Method,
    Credit_Card_Total AS Total_Sales
FROM Region_Payment_Summary

UNION ALL

SELECT
    Region,
    'UPI_Total' AS Payment_Method,
    UPI_Total AS Total_Sales
FROM Region_Payment_Summary;

```

-- Query 5: Native UNPIVOT with Custom Labels using REPLACE() --

```

SELECT
    Region,
    REPLACE(Payment_Method, '_Total', '') AS Payment_Method,
    Total_Sales
FROM
    (
        SELECT
            Region,
            Cash_Total,
            Credit_Card_Total,
            UPI_Total
        FROM Region_Payment_Summary
    ) AS SourceTable
UNPIVOT
    (
        Total_Sales
        FOR Payment_Method IN (
            Cash_Total,
            Credit_Card_Total,
            UPI_Total
        )
    ) AS UnpivotTable;

```

-- Query 6: Native UNPIVOT Automatically Excludes NULL Values --

```
SELECT
    Region,
    Payment_Method,
    Total_Sales
FROM
(
    SELECT
        Region,
        Cash_Total,
        Credit_Card_Total,
        UPI_Total
    FROM Region_Payment_Summary
) AS SourceTable
UNPIVOT
(
    Total_Sales
    FOR Payment_Method IN (
        Cash_Total,
        Credit_Card_Total,
        UPI_Total
    )
) AS UnpivotTable;
```

-- Query 7: Native UNPIVOT with Compatible Data Types --

```
CREATE TABLE Payment_Summary_Typed (
    Region VARCHAR(50),
    Cash_Total DECIMAL(10,2),
    Credit_Card_Total DECIMAL(10,2),
    UPI_Total DECIMAL(10,2)
);
```

```
SELECT
    Region,
    Payment_Method,
    Total_Sales
FROM
(
    SELECT
        Region,
        Cash_Total,
        Credit_Card_Total,
        UPI_Total
    FROM Payment_Summary_Typed
) AS SourceTable
UNPIVOT
(
    Total_Sales
    FOR Payment_Method IN (
        Cash_Total,
        Credit_Card_Total,
        UPI_Total
    )
)
```

```
)  
) AS UnpivotTable;
```

-- Query 8: Native UNPIVOT with Aggregation --

```
WITH Unpivoted_Data AS (  
  
    SELECT  
        Region,  
        Payment_Method,  
        Total_Sales  
    FROM  
    (  
        SELECT  
            Region,  
            Cash_Total,  
            Credit_Card_Total,  
            UPI_Total  
        FROM Region_Payment_Summary  
    ) AS SourceTable  
    UNPIVOT  
    (  
        Total_Sales  
        FOR Payment_Method IN (  
            Cash_Total,  
            Credit_Card_Total,  
            UPI_Total  
        )  
    ) AS UnpivotTable  
)  
  
SELECT  
    Payment_Method,  
    SUM(Total_Sales) AS Grand_Total_By_Payment_Method  
FROM Unpivoted_Data  
GROUP BY Payment_Method  
ORDER BY Grand_Total_By_Payment_Method DESC;
```

-- Query 9: SQLite - Native UNPIVOT Error Expected --

```
SELECT  
    Region,  
    Payment_Method,  
    Total_Sales  
FROM Region_Payment_Summary  
UNPIVOT (  
    Total_Sales  
    FOR Payment_Method IN (  
        Cash_Total,  
        Credit_Card_Total,  
        UPI_Total  
    )  
)  
) AS unpvt;
```

D3.2.4 Real Use Case – Reshaping Wide Survey Data for Analysis

-- Query 1: Create Employee Survey Wide Table --

DROP TABLE IF EXISTS Employee_Survey_Wide;

CREATE TABLE Employee_Survey_Wide (
 Employee_ID **TEXT**,
 Department **TEXT**,
 Work_Life_Balance **INTEGER**,
 Management_Rating **INTEGER**,
 Growth_Opportunity **INTEGER**,
 Compensation_Rating **INTEGER**,
 Work_Environment **INTEGER**
);

INSERT INTO Employee_Survey_Wide **VALUES**
('EMP001', 'Sales', 4, 3, 5, 4, 4),
('EMP002', 'Sales', 2, 4, 3, 3, 5),
('EMP003', 'Marketing', 5, 5, 4, 5, 4),
('EMP004', 'Marketing', 3, 2, 2, 3, 3),
('EMP005', 'IT', 4, 4, 5, 4, 5),
('EMP006', 'IT', 2, 3, 3, 2, 3),
('EMP007', 'Sales', 5, 4, 4, 5, 5),
('EMP008', 'Marketing', 3, 3, 3, 4, 4);

-- Query 2: UNPIVOT Wide Survey Data into Long Format --

WITH Survey_Long **AS** (

 SELECT
 Employee_ID,
 Department,
 'Work_Life_Balance' **AS** Category,
 Work_Life_Balance **AS** Rating
 FROM Employee_Survey_Wide

 UNION ALL

 SELECT
 Employee_ID,
 Department,
 'Management',
 Management_Rating
 FROM Employee_Survey_Wide

 UNION ALL

 SELECT
 Employee_ID,
 Department,
 'Growth_Opportunity',
 Growth_Opportunity

```

FROM Employee_Survey_Wide

UNION ALL

SELECT
    Employee_ID,
    Department,
    'Compensation',
    Compensation_Rating
FROM Employee_Survey_Wide

UNION ALL

SELECT
    Employee_ID,
    Department,
    'Work_Environment',
    Work_Environment
FROM Employee_Survey_Wide

)

SELECT *
FROM Survey_Long
ORDER BY Employee_ID, Category;

-- Query 3: Company-Wide Average Rating by Category --

WITH Survey_Long AS (

    SELECT
        Employee_ID,
        Department,
        'Work_Life_Balance' AS Category,
        Work_Life_Balance AS Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Management',
        Management_Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Growth_Opportunity',
        Growth_Opportunity
    FROM Employee_Survey_Wide

```

UNION ALL

SELECT

Employee_ID,
Department,
'Compensation',
Compensation_Rating

FROM Employee_Survey_Wide

UNION ALL

SELECT

Employee_ID,
Department,
'Work_Environment',
Work_Environment

FROM Employee_Survey_Wide

)

SELECT

Category,
ROUND(AVG(Rating), 2) **AS** Avg_Rating,
COUNT(*) **AS** Response_Count

FROM Survey_Long

GROUP BY Category

ORDER BY Avg_Rating **ASC**;

-- Query 4: Department-Wise and Category-Wise Average Rating --

WITH Survey_Long **AS** (

SELECT

Employee_ID,
Department,
'Work_Life_Balance' **AS** Category,
Work_Life_Balance **AS** Rating

FROM Employee_Survey_Wide

UNION ALL

SELECT

Employee_ID,
Department,
'Management',
Management_Rating

FROM Employee_Survey_Wide

UNION ALL

SELECT

Employee_ID,
Department,

```
        'Growth_Opportunity',
        Growth_Opportunity
FROM Employee_Survey_Wide
```

```
UNION ALL
```

```
SELECT
    Employee_ID,
    Department,
    'Compensation',
    Compensation_Rating
FROM Employee_Survey_Wide
```

```
UNION ALL
```

```
SELECT
    Employee_ID,
    Department,
    'Work_Environment',
    Work_Environment
FROM Employee_Survey_Wide
```

```
)
```

```
SELECT
    Department,
    Category,
    ROUND(AVG(Rating), 2) AS Avg_Rating
FROM Survey_Long
GROUP BY Department, Category
ORDER BY Department, Avg_Rating ASC;
```

```
-- Query 5: Find Employees with Low Ratings (2 or Below) --
```

```
WITH Survey_Long AS (

    SELECT
        Employee_ID,
        Department,
        'Work_Life_Balance' AS Category,
        Work_Life_Balance AS Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Management',
        Management_Rating
    FROM Employee_Survey_Wide

    UNION ALL
```

```

SELECT
    Employee_ID,
    Department,
    'Growth_Opportunity',
    Growth_Opportunity
FROM Employee_Survey_Wide

```

```

UNION ALL

```

```

SELECT
    Employee_ID,
    Department,
    'Compensation',
    Compensation_Rating
FROM Employee_Survey_Wide

```

```

UNION ALL

```

```

SELECT
    Employee_ID,
    Department,
    'Work_Environment',
    Work_Environment
FROM Employee_Survey_Wide

```

```

)

```

```

SELECT
    Employee_ID,
    Department,
    Category,
    Rating
FROM Survey_Long
WHERE Rating <= 2
ORDER BY Employee_ID;

```

```

-- Query 6: Convert Long Format Back to Wide Format --
-- Manual PIVOT using CASE WHEN --

```

```

WITH Survey_Long AS (

    SELECT
        Employee_ID,
        Department,
        'Work_Life_Balance' AS Category,
        Work_Life_Balance AS Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Management',

```

```
Management_Rating
FROM Employee_Survey_Wide
```

```
UNION ALL
```

```
SELECT
    Employee_ID,
    Department,
    'Growth_Oppportunity',
    Growth_Oppportunity
FROM Employee_Survey_Wide
```

```
UNION ALL
```

```
SELECT
    Employee_ID,
    Department,
    'Compensation',
    Compensation_Rating
FROM Employee_Survey_Wide
```

```
UNION ALL
```

```
SELECT
    Employee_ID,
    Department,
    'Work_Environment',
    Work_Environment
FROM Employee_Survey_Wide
```

```
)
```

```
SELECT
    Department,
    ROUND(
        AVG(
            CASE
                WHEN Category = 'Work_Life_Balance'
                THEN Rating
            END
        ), 2
    ) AS Work_Life_Balance,

    ROUND(
        AVG(
            CASE
                WHEN Category = 'Management'
                THEN Rating
            END
        ), 2
    ) AS Management,

    ROUND(
        AVG(
            CASE
```

```

                WHEN Category = 'Growth_Opportunity'
                THEN Rating
            END
        ), 2
    ) AS Growth_Opportunity,

    ROUND(
        AVG(
            CASE
                WHEN Category = 'Compensation'
                THEN Rating
            END
        ), 2
    ) AS Compensation,

    ROUND(
        AVG(
            CASE
                WHEN Category = 'Work_Environment'
                THEN Rating
            END
        ), 2
    ) AS Work_Environment

FROM Survey_Long
GROUP BY Department;

```

-- Query 7: Customer Rating Summary by Region --

```

SELECT
    Region,
    COUNT(
        CASE
            WHEN Customer_Rating = '5'
            THEN 1
        END
    ) AS Rating_5,

    COUNT(
        CASE
            WHEN Customer_Rating = '4'
            THEN 1
        END
    ) AS Rating_4,

    COUNT(
        CASE
            WHEN Customer_Rating = '3'
            THEN 1
        END
    ) AS Rating_3,

    COUNT(
        CASE

```

```

        WHEN Customer_Rating IS NULL
        THEN 1
    END
) AS Not_Rated

FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 8: Company-Wide Weakest Survey Category --

WITH Survey_Long AS (

    SELECT
        Employee_ID,
        Department,
        'Work_Life_Balance' AS Category,
        Work_Life_Balance AS Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Management',
        Management_Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Growth_Opportunity',
        Growth_Opportunity
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Compensation',
        Compensation_Rating
    FROM Employee_Survey_Wide

    UNION ALL

    SELECT
        Employee_ID,
        Department,
        'Work_Environment',
        Work_Environment
    FROM Employee_Survey_Wide

```

)

```
SELECT
    Category,
    ROUND(AVG(Rating), 2) AS Avg_Rating
FROM Survey_Long
GROUP BY Category
ORDER BY Avg_Rating ASC;
```

D4.1.1 What is a View – Stored Query as Virtual Table

-- Query 1: Create a Simple View with Basic Filtering --

```
DROP VIEW IF EXISTS Valid_Orders;
```

```
CREATE VIEW Valid_Orders AS
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0
    AND Quantity > 0;
```

-- Query 2: Access the View --

```
SELECT *
FROM Valid_Orders;
```

-- Query 3: Create an Aggregate/Summary View --

```
DROP VIEW IF EXISTS Region_Sales_Summary;
```

```
CREATE VIEW Region_Sales_Summary AS
SELECT
    Region,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;
```

-- Query 4: Access Summary View with Sorting --

```
SELECT *
FROM Region_Sales_Summary
ORDER BY Total_Sales DESC;
```

-- Query 5: Create a View using Window Function --

```
DROP VIEW IF EXISTS Top_Customers_Per_Region;
```

```

CREATE VIEW Top_Customers_Per_Region AS
SELECT *
FROM (
    SELECT
        Order_ID,
        Region,
        Customer_ID,
        Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Region
            ORDER BY Sales DESC
        ) AS Row_Num
    FROM Dirty_Sales_Dataset
) AS Ranked
WHERE Row_Num <= 5;

```

```
-- Query 6: Access Window Function View --
```

```

SELECT *
FROM Top_Customers_Per_Region;

```

```
-- Query 7: Apply Further Filtering on a View --
```

```

SELECT *
FROM Region_Sales_Summary
WHERE Total_Sales > 500000
ORDER BY Total_Sales DESC;

```

```
-- Query 8: Create a View Referencing Another View --
```

```

DROP VIEW IF EXISTS High_Value_Regions;

CREATE VIEW High_Value_Regions AS
SELECT
    Region,
    Total_Sales
FROM Region_Sales_Summary
WHERE Total_Sales > 500000;

```

```
-- Query 9: Access View of Another View --
```

```

SELECT *
FROM High_Value_Regions
ORDER BY Total_Sales DESC;

```

```
-- Query 10: View Reflects Newly Inserted Data --
```

```

INSERT INTO Dirty_Sales_Dataset
VALUES (...);

```

```
SELECT *  
FROM Valid_Orders;
```

-- Query 11: Delete a View --

```
DROP VIEW IF EXISTS Valid_Orders;
```

-- Query 12: Update a View by Dropping and Recreating It --

```
DROP VIEW IF EXISTS Region_Sales_Summary;
```

```
CREATE VIEW Region_Sales_Summary AS  
SELECT  
    Region,  
    COUNT(*) AS Total_Orders,  
    SUM(Sales) AS Total_Sales,  
    ROUND(AVG(Sales), 2) AS Avg_Sales  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Region;
```

-- Query 13: Create Customer Spending Summary View --

```
DROP VIEW IF EXISTS Customer_Spending_Summary;
```

```
CREATE VIEW Customer_Spending_Summary AS  
SELECT  
    Customer_ID,  
    Customer_Name,  
    COUNT(*) AS Total_Orders,  
    SUM(Sales) AS Total_Spending,  
    AVG(Sales) AS Avg_Order_Value  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Customer_ID, Customer_Name;
```

-- Query 14: Filter and Sort Customer Spending View --

```
SELECT *  
FROM Customer_Spending_Summary  
WHERE Total_Spending > 20000  
ORDER BY Total_Spending DESC  
LIMIT 20;
```

-- Query 15: Inspect All Views in SQLite --

```
SELECT  
    name,  
    sql
```

```
FROM sqlite_master
WHERE type = 'view';
```

```
-- Query 16: Create a View with Selected Columns --
```

```
DROP VIEW IF EXISTS Clean_Order_Details;
```

```
CREATE VIEW Clean_Order_Details AS
SELECT
```

```
    Order_ID,
    Order_Date,
    Customer_ID,
    Customer_Name,
    Region,
    Category,
    Sales,
    Quantity
```

```
FROM Dirty_Sales_Dataset
```

```
WHERE Sales > 0
    AND Quantity > 0;
```

```
-- Query 17: Access Selected Columns View --
```

```
SELECT
```

```
    Order_ID,
    Customer_Name,
    Region,
    Sales
```

```
FROM Clean_Order_Details
```

```
ORDER BY Sales DESC
```

```
LIMIT 20;
```

```
-- Query 18: Create a View for Security/Column Abstraction --
```

```
DROP VIEW IF EXISTS Customer_Public_Data;
```

```
CREATE VIEW Customer_Public_Data AS
```

```
SELECT
```

```
    Customer_ID,
    Customer_Name,
    Region,
    Segment
```

```
FROM Dirty_Sales_Dataset;
```

```
-- Query 19: Access the Restricted-Column View --
```

```
SELECT *
```

```
FROM Customer_Public_Data;
```

```
-- Query 20: Drop Multiple Views --
```

```
DROP VIEW IF EXISTS Customer_Spending_Summary;
```

```
DROP VIEW IF EXISTS High_Value_Regions;
```

```
DROP VIEW IF EXISTS Clean_Order_Details;
```

D4.1.2 Creating and Querying a View

```
-- Query 1: CREATE VIEW – Complete Syntax Breakdown --
```

```
CREATE VIEW view_name AS
```

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition
```

```
GROUP BY column
```

```
HAVING condition
```

```
ORDER BY column;
```

```
-- Query 2: Pattern 1 – Simple Filtering View --
```

```
CREATE VIEW Cancelled_Orders AS
```

```
SELECT Order_ID, Customer_Name, Sales, Order_Date
```

```
FROM Dirty_Sales_Dataset
```

```
WHERE Order_Status = 'Cancelled';
```

```
-- Query 3: Query Cancelled_Orders View --
```

```
SELECT *
```

```
FROM Cancelled_Orders;
```

```
-- Query 4: Count Records from View --
```

```
SELECT COUNT(*) AS Total_Cancelled_Orders
```

```
FROM Cancelled_Orders;
```

```
-- Query 5: Filter Records from View --
```

```
SELECT *
```

```
FROM Cancelled_Orders
```

```
WHERE Sales > 5000;
```

```
-- Query 6: Pattern 2 – Column-Restricted View --
```

```
CREATE VIEW Customer_Public_Info AS
```

```
SELECT
```

```
    Customer_ID,
```

```
    Customer_Name,
```

```
    Region,
```

```
    City
```

```
FROM Dirty_Sales_Dataset;
```

```
-- Query 7: Query Column-Restricted View --
```

```
SELECT *
```

```
FROM Customer_Public_Info;
```

-- Query 8: Pattern 3 – JOIN Wala View --

```
CREATE VIEW Sales_With_Rep_Info AS
SELECT
    d.Order_ID,
    d.Salesperson,
    d.Sales,
    d.Region,
    s.Region_Assigned,
    s.Years_Experience
FROM Dirty_Sales_Dataset d
INNER JOIN Salesperson_Info s
    ON d.Salesperson = s.Salesperson;
```

-- Query 9: Filter JOIN View --

```
SELECT *
FROM Sales_With_Rep_Info
WHERE Years_Experience > 5;
```

-- Query 10: Pattern 4 – Window Function Wala View --

```
CREATE VIEW Customer_Order_Ranking AS
SELECT
    Order_ID,
    Customer_ID,
    Order_Date,
    Sales,
    ROW_NUMBER() OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Order_Sequence
FROM Dirty_Sales_Dataset;
```

-- Query 11: Find First Order of Every Customer --

```
SELECT *
FROM Customer_Order_Ranking
WHERE Order_Sequence = 1;
```

-- Query 12: Pattern 5 – CTE Wala View --

```
CREATE VIEW Monthly_Sales_Trend AS
WITH Monthly_Data AS (
    SELECT
        strftime('%Y-%m', Order_Date) AS Sales_Month,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    WHERE Sales > 0
    GROUP BY Sales_Month
)
SELECT *
FROM Monthly_Data;
```

-- Query 13: Query Monthly Sales Trend --

```
SELECT *
FROM Monthly_Sales_Trend
ORDER BY Sales_Month;
```

-- Query 14: Pattern 6 – Multiple Aggregations Wala View --

```
CREATE VIEW Dashboard_KPIs AS
SELECT
    Region,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Revenue,
    SUM(Profit) AS Total_Profit,
    ROUND(AVG(Sales), 2) AS Avg_Order_Value,
    COUNT(
        CASE
            WHEN Order_Status = 'Cancelled' THEN 1
        END
    ) AS Cancelled_Count,
    ROUND(
        COUNT(
            CASE
                WHEN Order_Status = 'Cancelled' THEN 1
            END
        ) * 100.0 / COUNT(*),
        2
    ) AS Cancellation_Rate
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

-- Query 15: Simple SELECT on View --

```
SELECT *
FROM Dashboard_KPIs;
```

-- Query 16: WHERE Filter on View --

```
SELECT *
FROM Dashboard_KPIs
WHERE Total_Revenue > 500000;
```

-- Query 17: ORDER BY on View --

```
SELECT *
FROM Dashboard_KPIs
ORDER BY Cancellation_Rate DESC;
```

-- Query 18: Select Specific Columns from View --

```
SELECT
    Region,
    Total_Revenue
FROM Dashboard_KPIs;
```

-- Query 19: JOIN View with Another View --

```
SELECT
    dk.Region,
    dk.Total_Revenue,
    v.Order_ID
FROM Dashboard_KPIs dk
JOIN Cancelled_Orders v
    ON dk.Region = (
        SELECT Region
        FROM Dirty_Sales_Dataset
        WHERE Order_ID = v.Order_ID
    )
LIMIT 10;
```

-- Query 20: Aggregate from View --

```
SELECT
    SUM(Total_Revenue) AS Grand_Total_Revenue
FROM Dashboard_KPIs;
```

-- Query 21: View ke Andar ORDER BY --

```
CREATE VIEW Sorted_Sales AS
SELECT *
FROM Dirty_Sales_Dataset
ORDER BY Sales DESC;
```

-- Query 22: Better Practice – ORDER BY While Querying View --

```
SELECT *
FROM Sorted_Sales
ORDER BY Sales DESC;
```

-- Query 23: Comprehensive View – Salesperson Performance --

```
CREATE VIEW Salesperson_Performance AS
SELECT
    Salesperson,
    Region,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales,
    ROUND(AVG(Sales), 2) AS Avg_Sale_Value,
    SUM(Profit) AS Total_Profit
FROM Dirty_Sales_Dataset
WHERE Order_Status != 'Cancelled'
GROUP BY
    Salesperson,
    Region;
```

-- Query 24: Top 10 Salespeople by Total Sales --

```
SELECT *
FROM Salesperson_Performance
ORDER BY Total_Sales DESC
```

```
LIMIT 10;
```

```
-- Query 25: Find Loss-Making Salespeople --
```

```
SELECT
    Salesperson,
    Total_Profit
FROM Salesperson_Performance
WHERE Total_Profit < 0;
```

```
-- Query 26: Region-Wise Sales Rollup --
```

```
SELECT
    Region,
    SUM(Total_Sales) AS Region_Grand_Total
FROM Salesperson_Performance
GROUP BY Region;
```

D4.1.3 Updatable Views vs Read-Only Views

```
-- Query 1: Create a Simple Updatable View --
```

```
CREATE VIEW Simple_North_Orders AS
SELECT
    Order_ID,
    Customer_Name,
    Sales,
    Region
FROM Dirty_Sales_Dataset
WHERE Region = 'North';
```

```
-- Query 2: UPDATE Through View (SQLite Mein Fail Hoga) --
```

```
UPDATE Simple_North_Orders
SET Sales = 5500
WHERE Order_ID = 'ORD001234';
```

```
-- Query 3: Verify Data in Original Table --
```

```
SELECT
    Order_ID,
    Sales
FROM Dirty_Sales_Dataset
WHERE Order_ID = 'ORD001234';
```

```
-- Query 4: Create a Read-Only View Using GROUP BY --
```

```
CREATE VIEW Region_Sales_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 5: UPDATE Read-Only Aggregate View (Error Expected) --  
UPDATE Region_Sales_Summary  
SET Total_Sales = 600000  
WHERE Region = 'North';
```

```
-- Query 6: Simple View with WHERE Clause --  
CREATE VIEW Editable_Pending_Orders AS  
SELECT  
    Order_ID,  
    Customer_Name,  
    Order_Status,  
    Sales  
FROM Dirty_Sales_Dataset  
WHERE Order_Status = 'Pending';
```

```
-- Query 7: UPDATE Through Pending Orders View --  
UPDATE Editable_Pending_Orders  
SET Order_Status = 'Delivered'  
WHERE Order_ID = (  
    SELECT Order_ID  
    FROM Editable_Pending_Orders  
    LIMIT 1  
);
```

```
-- Query 8: Verify UPDATE in Original Table --  
SELECT  
    Order_ID,  
    Order_Status  
FROM Dirty_Sales_Dataset  
WHERE Order_ID = (  
    SELECT Order_ID  
    FROM Editable_Pending_Orders  
    LIMIT 1  
);
```

```
-- Query 9: Create Read-Only Category Summary View --  
CREATE VIEW Category_Totals AS  
SELECT  
    Category,  
    SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Category;
```

```
-- Query 10: UPDATE Aggregate View (FAIL Expected) --  
UPDATE Category_Totals  
SET Total_Sales = 1000000  
WHERE Category = 'Electronics';
```

```
-- Query 11: INSERT Through Simple View --
```

```

INSERT INTO Editable_Pending_Orders (
    Order_ID,
    Customer_Name,
    Order_Status,
    Sales
)
VALUES (
    'ORD999999',
    'Test Customer',
    'Pending',
    5000
);

```

```

-- Query 12: INSERT Through Complex View (Error Expected) --
INSERT INTO Category_Totals (
    Category,
    Total_Sales
)
VALUES (
    'NewCategory',
    10000
);

```

```

-- Query 13: SQLite View UPDATE Test --
UPDATE Editable_Pending_Orders
SET Order_Status = 'Delivered'
WHERE Order_ID = 'ORD001';

```

```

-- Query 14: Create View with WITH CHECK OPTION (Database-Specific) --
CREATE VIEW North_Orders_Only AS
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
WITH CHECK OPTION;

```

```

-- Query 15: Attempt to Violate View Condition --
UPDATE North_Orders_Only
SET Region = 'South'
WHERE Order_ID = 'ORD001';

```

```

-- Query 16: Create Test View for SQLite --
CREATE VIEW Test_Updatable_View AS
SELECT
    Order_ID,
    Sales
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

```

```

-- Query 17: UPDATE SQLite View (Error Expected) --

```

```
UPDATE Test_Updatable_View
SET Sales = 9999
WHERE Order_ID = 'ORD000001';
```

```
-- Query 18: Check View Definition in SQLite --
```

```
SELECT
    name,
    sql
FROM sqlite_master
WHERE type = 'view';
```

```
-- Query 19: Check All Views in the Database --
```

```
SELECT
    name
FROM sqlite_master
WHERE type = 'view'
ORDER BY name;
```

```
-- Query 20: Create a Simple View for Read Operations --
```

```
CREATE VIEW North_Sales_ReadOnly AS
SELECT
    Order_ID,
    Customer_Name,
    Sales,
    Region
FROM Dirty_Sales_Dataset
WHERE Region = 'North';
```

```
-- Query 21: Read Data from View --
```

```
SELECT *
FROM North_Sales_ReadOnly;
```

```
-- Query 22: Delete Through View (SQLite Error Expected) --
```

```
DELETE FROM North_Sales_ReadOnly
WHERE Order_ID = 'ORD000001';
```

```
-- Query 23: INSERT Through View (SQLite Error Expected) --
```

```
INSERT INTO North_Sales_ReadOnly (
    Order_ID,
    Customer_Name,
    Sales,
    Region
)
VALUES (
    'ORD888888',
    'New Customer',
    7500,
    'North'
);
```

D4.1.4 When to Use Views in Data Analyst Workflow

-- Query 1: Scenario 1 – Weekly Sales Report View --

```
CREATE VIEW Weekly_Sales_Report AS
SELECT
    strftime('%Y-%W', Order_Date) AS Week,
    Region,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY
    Week,
    Region;
```

-- Query 2: Query Weekly Sales Report --

```
SELECT *
FROM Weekly_Sales_Report
WHERE Week = '2025-24';
```

-- Query 3: Scenario 2 – Clean Data View --

```
CREATE VIEW Clean_Sales_Data AS
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0
    AND Quantity > 0
    AND Discount BETWEEN 0 AND 1;
```

-- Query 4: Query Clean Sales Data --

```
SELECT *
FROM Clean_Sales_Data;
```

-- Query 5: Scenario 3 – BI Tool Dashboard Source View --

```
CREATE VIEW Dashboard_Source AS
SELECT
    Order_Date,
    Region,
    Category,
    Sales,
    Profit,
    Customer_ID
FROM Dirty_Sales_Dataset
WHERE Order_Status != 'Cancelled';
```

-- Query 6: Scenario 4 – One-Time Analysis (Direct Query) --

```
SELECT
    Category,
    SUM(Sales) AS Total
FROM Dirty_Sales_Dataset
```

```
WHERE Order_Date >= '2025-01-01'  
GROUP BY Category;
```

```
-- Query 7: Scenario 5 – Exploratory Analysis --  
SELECT DISTINCT Region  
FROM Dirty_Sales_Dataset;
```

```
-- Query 8: Count Negative Sales --  
SELECT COUNT(*) AS Negative_Sales_Count  
FROM Dirty_Sales_Dataset  
WHERE Sales < 0;
```

```
-- Query 9: Preview Raw Data --  
SELECT *  
FROM Dirty_Sales_Dataset  
LIMIT 100;
```

```
-- Query 10: Layer 1 – Data Cleaning Foundation View --  
CREATE VIEW Clean_Orders AS  
SELECT *  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
      AND Quantity > 0  
      AND Discount BETWEEN 0 AND 1  
      AND Order_Status != 'Cancelled';
```

```
-- Query 11: Layer 2 – Daily Business Summary View --  
CREATE VIEW Daily_Business_Summary AS  
SELECT  
    Order_Date,  
    COUNT(*) AS Orders,  
    SUM(Sales) AS Revenue,  
    SUM(Profit) AS Profit,  
    ROUND(AVG(Sales), 2) AS Avg_Order_Value  
FROM Clean_Orders  
GROUP BY Order_Date;
```

```
-- Query 12: Layer 3 – Customer Segmentation View --  
CREATE VIEW Customer_Segments AS  
SELECT  
    Customer_ID,  
    Customer_Name,  
    SUM(Sales) AS Total_Spending,  
    COUNT(*) AS Total_Orders,  
    CASE  
        WHEN SUM(Sales) >= 50000 THEN 'VIP'  
        WHEN SUM(Sales) >= 20000 THEN 'Regular'  
        ELSE 'Occasional'  
    END AS Segment
```

```
FROM Clean_Orders
GROUP BY
    Customer_ID,
    Customer_Name;
```

```
-- Query 13: Count VIP Customers --
SELECT COUNT(*) AS VIP_Customer_Count
FROM Customer_Segments
WHERE Segment = 'VIP';
```

```
-- Query 14: Last 30 Days Daily Sales Trend --
SELECT *
FROM Daily_Business_Summary
WHERE Order_Date >= date('now', '-30 days')
ORDER BY Order_Date;
```

```
-- Query 15: Segment-Wise Revenue and Customer Count --
SELECT
    Segment,
    SUM(Total_Spending) AS Segment_Revenue,
    COUNT(*) AS Customer_Count
FROM Customer_Segments
GROUP BY Segment;
```

```
-- Query 16: Layer 1 – Clean Orders View --
CREATE VIEW Clean_Orders_Layer1 AS
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0
    AND Quantity > 0
    AND Order_Status != 'Cancelled';
```

```
-- Query 17: Layer 2 – Category Performance View --
CREATE VIEW Category_Performance AS
SELECT
    Category,
    COUNT(*) AS Orders,
    SUM(Sales) AS Revenue,
    SUM(Profit) AS Profit,
    ROUND(
        SUM(Profit) * 100.0 / SUM(Sales),
        2
    ) AS Profit_Margin_Percent
FROM Clean_Orders_Layer1
GROUP BY Category;
```

```
-- Query 18: Query Category Performance --
SELECT *
FROM Category_Performance
```

```
ORDER BY Profit_Margin_Percent DESC;
```

```
-- Query 19: Check All Views in SQLite --
```

```
SELECT
    name,
    sql
FROM sqlite_master
WHERE type = 'view'
ORDER BY name;
```

```
-- Query 20: Drop Unused View --
```

```
DROP VIEW IF EXISTS Weekly_Sales_Report;
```

```
-- Query 21: Drop Multiple Layered Views --
```

```
DROP VIEW IF EXISTS Category_Performance;
```

```
DROP VIEW IF EXISTS Clean_Orders_Layer1;
```

```
DROP VIEW IF EXISTS Customer_Segments;
```

```
DROP VIEW IF EXISTS Daily_Business_Summary;
```

```
DROP VIEW IF EXISTS Clean_Orders;
```

```
-- Query 22: View-Based Business Summary --
```

```
CREATE VIEW Region_Business_Summary AS
SELECT
    Region,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Revenue,
    SUM(Profit) AS Total_Profit,
    ROUND(AVG(Sales), 2) AS Average_Order_Value
FROM Clean_Sales_Data
GROUP BY Region;
```

```
-- Query 23: Query Region Business Summary --
```

```
SELECT *
FROM Region_Business_Summary
ORDER BY Total_Revenue DESC;
```

```
-- Query 24: Filter High-Revenue Regions --
```

```
SELECT *
FROM Region_Business_Summary
WHERE Total_Revenue > 500000;
```

```
-- Query 25: View Naming Convention Example --
```

```
CREATE VIEW vw_Clean_Orders AS
SELECT *
```

```

FROM Dirty_Sales_Dataset
WHERE Sales > 0
      AND Quantity > 0
      AND Order_Status != 'Cancelled';

-- Query 26: Layered View Architecture Example --
CREATE VIEW vw_Category_Performance AS
SELECT
    Category,
    COUNT(*) AS Orders,
    SUM(Sales) AS Revenue,
    SUM(Profit) AS Profit,
    ROUND(
        SUM(Profit) * 100.0 / SUM(Sales),
        2
    ) AS Profit_Margin_Percent
FROM vw_Clean_Orders
GROUP BY Category;

-- Query 27: Final Layered View Analysis --
SELECT *
FROM vw_Category_Performance
ORDER BY Profit_Margin_Percent DESC;

```

D4.2.1 Creating and Using Temp Tables

```

-- Query 1: SQL Server – Local Temp Table --
SELECT *
INTO #Region_Summary
FROM (
    SELECT
        Region,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
) AS temp_data;

-- Query 2: SQL Server – Query Local Temp Table --
SELECT *
FROM #Region_Summary;

-- Query 3: SQL Server – Global Temp Table --
SELECT *
INTO ##Global_Summary
FROM Dirty_Sales_Dataset;

-- Query 4: PostgreSQL – Create Temporary Table --
CREATE TEMP TABLE Region_Summary AS
SELECT
    Region,

```

```
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 5: PostgreSQL – Query Temporary Table --
SELECT *
FROM Region_Summary;
```

```
-- Query 6: MySQL – Create Temporary Table --
CREATE TEMPORARY TABLE Region_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 7: SQLite – Create Temporary Table --
CREATE TEMP TABLE Region_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 8: SQLite – Query Temporary Table --
SELECT *
FROM Region_Summary;
```

```
-- Query 9: Example 1 – Create High-Value Customers Temp Table --
CREATE TEMP TABLE High_Value_Customers AS
SELECT
    Customer_ID,
    Customer_Name,
    SUM(Sales) AS Total_Spending
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY
    Customer_ID,
    Customer_Name
HAVING SUM(Sales) > 30000;
```

```
-- Query 10: Top 10 High-Value Customers --
SELECT *
FROM High_Value_Customers
ORDER BY Total_Spending DESC
LIMIT 10;
```

```
-- Query 11: Count High-Value Customers --
```

```

SELECT
    COUNT(*) AS Total_High_Value_Customers
FROM High_Value_Customers;

-- Query 12: Average Spending Among High-Value Customers --
SELECT
    AVG(Total_Spending) AS Avg_Spending_Among_VIPs
FROM High_Value_Customers;

-- Query 13: Example 2 – Create Order Analysis Temp Table --
CREATE TEMP TABLE Order_Analysis AS
SELECT
    Order_ID,
    Customer_ID,
    Region,
    Sales,
    Order_Date
FROM Dirty_Sales_Dataset
WHERE Sales > 0;

-- Query 14: Create Index on Temp Table --
CREATE INDEX idx_temp_customer
ON Order_Analysis(Customer_ID);

-- Query 15: Fast Lookup Using Indexed Column --
SELECT *
FROM Order_Analysis
WHERE Customer_ID = 'CUST05177';

-- Query 16: Example 3 – Create Empty Sales Targets Table --
CREATE TEMP TABLE Sales_Targets (
    Region TEXT,
    Target_Amount REAL
);

-- Query 17: Insert Manual Target Data --
INSERT INTO Sales_Targets (
    Region,
    Target_Amount
)
VALUES
    ('North', 600000),
    ('South', 550000),
    ('East', 500000),
    ('West', 620000);

-- Query 18: Compare Actual Sales with Targets --
SELECT

```

```

    d.Region,
    SUM(d.Sales) AS Actual_Sales,
    st.Target_Amount,
    ROUND(
        SUM(d.Sales) * 100.0 / st.Target_Amount,
        2
    ) AS Achievement_Percent
FROM Dirty_Sales_Dataset d
JOIN Sales_Targets st
    ON d.Region = st.Region
GROUP BY
    d.Region,
    st.Target_Amount;

```

-- Query 19: Example 4 – Step 1: Valid Orders --

```

CREATE TEMP TABLE Step1_Valid_Orders AS
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0
    AND Order_Status != 'Cancelled';

```

-- Query 20: Step 2 – Customer-Wise Summary --

```

CREATE TEMP TABLE Step2_Customer_Summary AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Sales
FROM Step1_Valid_Orders
GROUP BY Customer_ID;

```

-- Query 21: Step 3 – High-Value Customers --

```

CREATE TEMP TABLE Step3_High_Value AS
SELECT *
FROM Step2_Customer_Summary
WHERE Total_Sales > (
    SELECT AVG(Total_Sales)
    FROM Step2_Customer_Summary
);

```

-- Query 22: Final Result – Above-Average Customers --

```

SELECT
    COUNT(*) AS Above_Average_Customers
FROM Step3_High_Value;

```

-- Query 23: Debug Step 1 Temp Table --

```

SELECT *
FROM Step1_Valid_Orders
LIMIT 10;

```

-- Query 24: Debug Step 2 Temp Table --

```
SELECT *
FROM Step2_Customer_Summary
LIMIT 10;
```

```
-- Query 25: Example 5 – Explicitly Drop Temp Tables --
DROP TABLE IF EXISTS High_Value_Customers;
```

```
DROP TABLE IF EXISTS Order_Analysis;
```

```
DROP TABLE IF EXISTS Sales_Targets;
```

```
-- Query 26: Avoid Naming Conflicts with tmp_ Prefix --
CREATE TEMP TABLE tmp_Region_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 27: Practical DBeaver Test – Create Analysis Table --
CREATE TEMP TABLE Region_Category_Analysis AS
SELECT
    Region,
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY
    Region,
    Category;
```

```
-- Query 28: Create Index for Performance --
CREATE INDEX idx_region
ON Region_Category_Analysis(Region);
```

```
-- Query 29: Region-Wise Category Analysis --
SELECT *
FROM Region_Category_Analysis
WHERE Region = 'North'
ORDER BY Total_Sales DESC;
```

```
-- Query 30: Category-Wise Grand Total --
SELECT
    Category,
    SUM(Total_Sales) AS Grand_Total
FROM Region_Category_Analysis
GROUP BY Category;
```

```
-- Query 31: Cleanup Temp Table --  
DROP TABLE IF EXISTS Region_Category_Analysis;
```

D4.2.2 Temp Table vs CTE – Performance Comparison

```
-- Query 1: CTE Approach – Heavy Aggregation --  
WITH Customer_Totals AS (  
    SELECT  
        Customer_ID,  
        SUM(Sales) AS Total_Sales,  
        COUNT(*) AS Order_Count,  
        AVG(Sales) AS Avg_Sales  
    FROM Dirty_Sales_Dataset  
    GROUP BY Customer_ID  
)  
SELECT  
    ct1.Customer_ID,  
    ct1.Total_Sales,  
    (  
        SELECT AVG(Total_Sales)  
        FROM Customer_Totals  
    ) AS Overall_Avg,  
    ct1.Total_Sales - (  
        SELECT AVG(Total_Sales)  
        FROM Customer_Totals  
    ) AS Diff_From_Avg  
FROM Customer_Totals ct1  
WHERE ct1.Total_Sales > (  
    SELECT AVG(Total_Sales)  
    FROM Customer_Totals  
)  
);  
  
-- Query 2: Temp Table Approach – Create Aggregation Table --  
CREATE TEMP TABLE Customer_Totals AS  
SELECT  
    Customer_ID,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Order_Count,  
    AVG(Sales) AS Avg_Sales  
FROM Dirty_Sales_Dataset  
GROUP BY Customer_ID;  
  
-- Query 3: Temp Table Approach – Reuse Aggregated Data --  
SELECT  
    ct1.Customer_ID,  
    ct1.Total_Sales,  
    (  
        SELECT AVG(Total_Sales)  
        FROM Customer_Totals  
    ) AS Overall_Avg,  
    ct1.Total_Sales - (  
        SELECT AVG(Total_Sales)
```

```

        FROM Customer_Totals
    ) AS Diff_From_Avg
FROM Customer_Totals ct1
WHERE ct1.Total_Sales > (
    SELECT AVG(Total_Sales)
    FROM Customer_Totals
);

```

```

-- Query 4: Cleanup Customer Totals Temp Table --
DROP TABLE IF EXISTS Customer_Totals;

```

```

-- Query 5: CTE Version – Heavy Aggregation --

```

```

WITH Heavy_Aggregation AS (
    SELECT
        Customer_ID,
        Region,
        Category,
        SUM(Sales) AS Total_Sales,
        AVG(Discount) AS Avg_Discount,
        COUNT(*) AS Order_Count
    FROM Dirty_Sales_Dataset
    GROUP BY
        Customer_ID,
        Region,
        Category
)
SELECT *
FROM Heavy_Aggregation
WHERE Total_Sales > 10000

UNION ALL

SELECT *
FROM Heavy_Aggregation
WHERE Order_Count > 5;

```

```

-- Query 6: Temp Table Version – Heavy Aggregation --

```

```

CREATE TEMP TABLE Heavy_Aggregation AS
SELECT
    Customer_ID,
    Region,
    Category,
    SUM(Sales) AS Total_Sales,
    AVG(Discount) AS Avg_Discount,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY
    Customer_ID,
    Region,
    Category;

```

-- Query 7: Temp Table Version – Multiple References --

```
SELECT *
FROM Heavy_Aggregation
WHERE Total_Sales > 10000
```

UNION ALL

```
SELECT *
FROM Heavy_Aggregation
WHERE Order_Count > 5;
```

-- Query 8: Cleanup Heavy Aggregation Temp Table --

```
DROP TABLE IF EXISTS Heavy_Aggregation;
```

-- Query 9: SQLite – Enable Query Timer --

```
.timer ON
```

-- Query 10: CTE Performance Test --

```
WITH Customer_Agg AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total,
        COUNT(*) AS Cnt
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
)
SELECT
    (
        SELECT COUNT(*)
        FROM Customer_Agg
        WHERE Total > 20000
    ) AS High_Spenders,
    (
        SELECT COUNT(*)
        FROM Customer_Agg
        WHERE Cnt > 5
    ) AS Frequent_Buyers,
    (
        SELECT AVG(Total)
        FROM Customer_Agg
    ) AS Overall_Avg;
```

-- Query 11: Create Temp Table for Performance Test --

```
CREATE TEMP TABLE Customer_Agg AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total,
    COUNT(*) AS Cnt
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 12: Temp Table Performance Test --

```
SELECT
(
    SELECT COUNT(*)
    FROM Customer_Agg
    WHERE Total > 20000
) AS High_Spenders,
(
    SELECT COUNT(*)
    FROM Customer_Agg
    WHERE Cnt > 5
) AS Frequent_Buyers,
(
    SELECT AVG(Total)
    FROM Customer_Agg
) AS Overall_Avg;
```

-- Query 13: Cleanup Performance Test Table --

```
DROP TABLE IF EXISTS Customer_Agg;
```

-- Query 14: CTE with Single Reference --

```
WITH Customer_Summary AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
)
SELECT *
FROM Customer_Summary
WHERE Total_Sales > 20000;
```

-- Query 15: Temp Table with Single Reference --

```
CREATE TEMP TABLE Customer_Summary AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 16: Query Single-Reference Temp Table --

```
SELECT *
FROM Customer_Summary
WHERE Total_Sales > 20000;
```

-- Query 17: Cleanup Single-Reference Temp Table --

```
DROP TABLE IF EXISTS Customer_Summary;
```

-- Query 18: Create Temp Table for Index Comparison --

```
CREATE TEMP TABLE Indexed_Customer_Summary AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 19: Create Index on Temp Table --

```
CREATE INDEX idx_customer_summary
ON Indexed_Customer_Summary(Customer_ID);
```

-- Query 20: Indexed Lookup on Temp Table --

```
SELECT *
FROM Indexed_Customer_Summary
WHERE Customer_ID = 'CUST05177';
```

-- Query 21: SQLite – Explain Query Plan for CTE --

```
EXPLAIN QUERY PLAN
WITH Heavy_Aggregation AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
)
SELECT *
FROM Heavy_Aggregation
WHERE Total_Sales > 10000;
```

-- Query 22: SQLite – Explain Query Plan for Temp Table --

```
EXPLAIN QUERY PLAN
SELECT *
FROM Indexed_Customer_Summary
WHERE Total_Sales > 10000;
```

-- Query 23: Cleanup Indexed Temp Table --

```
DROP TABLE IF EXISTS Indexed_Customer_Summary;
```

-- Query 24: CTE – Customer Aggregation for Comparison --

```
WITH Customer_Agg AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total,
        COUNT(*) AS Cnt
    FROM Dirty_Sales_Dataset
    GROUP BY Customer_ID
)
```

```
SELECT
    COUNT(*) AS Total_Customers
FROM Customer_Agg;
```

-- Query 25: Temp Table – Customer Aggregation for Comparison --

```
CREATE TEMP TABLE Comparison_Customer_Agg AS
SELECT
    Customer_ID,
    SUM(Sales) AS Total,
    COUNT(*) AS Cnt
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

-- Query 26: Query Comparison Temp Table --

```
SELECT
    COUNT(*) AS Total_Customers
FROM Comparison_Customer_Agg;
```

-- Query 27: Cleanup Comparison Table --

```
DROP TABLE IF EXISTS Comparison_Customer_Agg;
```

D4.2.3 GLOBAL vs LOCAL TEMP TABLES – Scope and Visibility

-- Query 1: Create a Local Temp Table (SQL Server) --

```
SELECT *
INTO #My_Local_Temp
FROM (
    SELECT
        Region,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
) AS temp_data;
```

-- Query 2: Query the Local Temp Table --

```
SELECT *
FROM #My_Local_Temp;
```

-- Query 3: Create a Global Temp Table (SQL Server) --

```
SELECT *
INTO ##My_Global_Temp
FROM (
    SELECT
        Region,
        SUM(Sales) AS Total_Sales
    FROM Dirty_Sales_Dataset
    GROUP BY Region
) AS temp_data;
```

-- Query 4: Query the Global Temp Table --

```
SELECT *
FROM ##My_Global_Temp;
```

```

-- Query 5: Local Temp Table – Analyst-Specific Analysis --
SELECT *
INTO #My_Analysis
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
      AND Sales > 5000;

-- Query 6: Count Records in Local Temp Table --
SELECT COUNT(*) AS Total_Records
FROM #My_Analysis;

-- Query 7: Global Temp Table – Shared Batch Data --
SELECT *
INTO ##Shared_Batch_Data
FROM Dirty_Sales_Dataset
WHERE Order_Date = CURRENT_DATE;

-- Query 8: Access Global Temp Table from Another Session --
SELECT *
FROM ##Shared_Batch_Data;

-- Query 9: SQLite Temporary Table – Connection-Specific --
CREATE TEMP TABLE My_Temp_Data AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 10: Query SQLite Temporary Table --
SELECT *
FROM My_Temp_Data;

-- Query 11: Create a Permanent Shared Table --
CREATE TABLE Shared_Analysis_Data AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 12: Access Permanent Table from Any Session --
SELECT *
FROM Shared_Analysis_Data;

-- Query 13: Clean Up Permanent Shared Table --
DROP TABLE Shared_Analysis_Data;

-- Query 14: SQLite Session Test – Create Temporary Table --
CREATE TEMP TABLE Session_Test AS
SELECT
    Region,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset

```

```

GROUP BY Region;

-- Query 15: Query Temporary Table in the Same Session --
SELECT *
FROM Session_Test;

-- Query 16: Test Temporary Table Access from Another Connection --
SELECT *
FROM Session_Test;
-- Expected Result in Another Connection:
-- Error: no such table: Session_Test

-- Query 17: Check Local Temp Table Record Count (SQL Server) --
SELECT COUNT(*) AS Total_Records
FROM #My_Local_Temp;

-- Query 18: Check Global Temp Table Record Count (SQL Server) --
SELECT COUNT(*) AS Total_Records
FROM ##My_Global_Temp;

-- Query 19: Drop Local Temp Table Explicitly --
DROP TABLE #My_Local_Temp;

-- Query 20: Drop Global Temp Table Explicitly --
DROP TABLE ##My_Global_Temp;

-- Query 21: SQLite Temporary Table Cleanup --
DROP TABLE My_Temp_Data;

-- Query 22: SQLite Session Test Cleanup --
DROP TABLE Session_Test;

-- Query 23: Verify Local Temp Table Isolation (SQL Server) --
-- Session A:
CREATE TABLE #Sales_Data (
    Region VARCHAR(50),
    Total_Sales DECIMAL(18, 2)
);

-- Session B:
CREATE TABLE #Sales_Data (
    Region VARCHAR(50),
    Total_Sales DECIMAL(18, 2)
);

-- Both sessions can use the same local temp table name independently.

-- Query 24: Global Temp Table Naming Collision Example --
-- Session A:
CREATE TABLE ##Shared_Sales_Data (
    Region VARCHAR(50),
    Total_Sales DECIMAL(18, 2)
);

-- Session B:

```

```

-- Attempting to create the same global temp table
-- may result in a naming conflict.

-- Query 25: Check Whether a Global Temp Table Exists (SQL Server) --
IF OBJECT_ID('tempdb..##Shared_Sales_Data') IS NOT NULL
BEGIN
    SELECT *
    FROM ##Shared_Sales_Data;
END;

-- Query 26: Drop Global Temp Table if It Exists --
IF OBJECT_ID('tempdb..##Shared_Sales_Data') IS NOT NULL
BEGIN
    DROP TABLE ##Shared_Sales_Data;
END;

-- Query 27: SQLite Permanent Table for Cross-Session Sharing --
CREATE TABLE Shared_Sales_Data AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 28: Query Shared SQLite Table --
SELECT *
FROM Shared_Sales_Data;

-- Query 29: Clean Up Shared SQLite Table --
DROP TABLE Shared_Sales_Data;

-- Query 30: Compare Temporary and Permanent Tables in SQLite --
CREATE TEMP TABLE Local_Sales_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

CREATE TABLE Global_Sales_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;

-- Query 31: Query Both Tables --
SELECT *
FROM Local_Sales_Summary;

SELECT *
FROM Global_Sales_Summary;

-- Query 32: Clean Up Both Tables --
DROP TABLE Local_Sales_Summary;

```

```
DROP TABLE Global_Sales_Summary;
```

D4.2.4 USING TEMP TABLES FOR MULTI-STEP DATA TRANSFORMATIONS

```
-- =====  
-- PIPELINE 1: TOP-3 CUSTOMERS BY REGION  
-- =====
```

```
-- Query 1: Step 1 – Clean Raw Data  
DROP TABLE IF EXISTS Step1_Clean_Data;
```

```
CREATE TEMP TABLE Step1_Clean_Data AS  
SELECT
```

```
    Order_ID,  
    Customer_ID,  
    Customer_Name,  
    Region,  
    Sales,  
    Profit,  
    Order_Date  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
    AND Order_Status != 'Cancelled'  
    AND Customer_ID IS NOT NULL;
```

```
-- Query 2: Verify Clean Data  
SELECT COUNT(*) AS Clean_Row_Count  
FROM Step1_Clean_Data;
```

```
-- Query 3: Step 2 – Customer-Wise Aggregation  
DROP TABLE IF EXISTS Step2_Customer_Aggregates;
```

```
CREATE TEMP TABLE Step2_Customer_Aggregates AS  
SELECT
```

```
    Customer_ID,  
    Customer_Name,  
    Region,  
    SUM(Sales) AS Total_Spending,  
    SUM(Profit) AS Total_Profit,  
    COUNT(*) AS Total_Orders,  
    ROUND(  
        SUM(Profit) * 100.0 / SUM(Sales),  
        2  
    ) AS Profit_Margin_Percent  
FROM Step1_Clean_Data  
GROUP BY  
    Customer_ID,  
    Customer_Name,  
    Region;
```

```
-- Query 4: Verify Customer Aggregates  
SELECT COUNT(*) AS Unique_Customers  
FROM Step2_Customer_Aggregates;
```

```

-- Query 5: Preview Customer Aggregates
SELECT *
FROM Step2_Customer_Aggregates
LIMIT 5;

-- Query 6: Step 3 – Rank Customers Within Each Region
DROP TABLE IF EXISTS Step3_Ranked_Customers;

CREATE TEMP TABLE Step3_Ranked_Customers AS
SELECT
    *,
    ROW_NUMBER() OVER (
        PARTITION BY Region
        ORDER BY Total_Spending DESC
    ) AS Spending_Rank
FROM Step2_Customer_Aggregates;

-- Query 7: Verify Top-3 Customers Per Region
SELECT *
FROM Step3_Ranked_Customers
WHERE Spending_Rank <= 3
ORDER BY
    Region,
    Spending_Rank;

-- Query 8: Step 4 – Filter Top-3 and Add Customer Label
DROP TABLE IF EXISTS Step4_Final_VIP_Report;

CREATE TEMP TABLE Step4_Final_VIP_Report AS
SELECT
    Region,
    Spending_Rank,
    Customer_Name,
    Total_Spending,
    Total_Profit,
    Profit_Margin_Percent,
    CASE
        WHEN Profit_Margin_Percent > 20
            THEN 'Highly Profitable VIP'
        WHEN Profit_Margin_Percent > 0
            THEN 'Profitable VIP'
        ELSE 'VIP But Loss-Making – Review!'
    END AS Customer_Health_Status
FROM Step3_Ranked_Customers
WHERE Spending_Rank <= 3;

-- Query 9: Final VIP Report
SELECT *
FROM Step4_Final_VIP_Report
ORDER BY
    Region,
    Spending_Rank;

```

```

-- Query 10: Step 5 – Cleanup Intermediate Tables
DROP TABLE IF EXISTS Step1_Clean_Data;

DROP TABLE IF EXISTS Step2_Customer_Aggregates;

DROP TABLE IF EXISTS Step3_Ranked_Customers;

-- Step4_Final_VIP_Report ko report ke liye retain kar sakte ho.


-- =====
-- PIPELINE 2: INDEPENDENT DEBUGGING
-- =====

-- Query 11: Check Clean Data for a Specific Customer
SELECT *
FROM Step1_Clean_Data
WHERE Customer_ID = 'CUST05177';

-- Query 12: Check Customer Aggregation
SELECT *
FROM Step2_Customer_Aggregates
WHERE Customer_ID = 'CUST05177';

-- Query 13: Check Customer Ranking
SELECT *
FROM Step3_Ranked_Customers
WHERE Customer_ID = 'CUST05177';


-- =====
-- PIPELINE 3: MODIFY CLEANING LOGIC
-- =====

-- Query 14: Recreate Step 1 with Discount Filter
DROP TABLE IF EXISTS Step1_Clean_Data;

CREATE TEMP TABLE Step1_Clean_Data AS
SELECT
    Order_ID,
    Customer_ID,
    Customer_Name,
    Region,
    Sales,
    Profit,
    Order_Date
FROM Dirty_Sales_Dataset
WHERE Sales > 0
    AND Order_Status != 'Cancelled'
    AND Customer_ID IS NOT NULL
    AND Discount BETWEEN 0 AND 1;

-- Query 15: Verify Updated Clean Data

```

```

SELECT COUNT(*) AS Updated_Clean_Row_Count
FROM Step1_Clean_Data;

-- Query 16: Re-run Customer Aggregation
DROP TABLE IF EXISTS Step2_Customer_Aggregates;

CREATE TEMP TABLE Step2_Customer_Aggregates AS
SELECT
    Customer_ID,
    Customer_Name,
    Region,
    SUM(Sales) AS Total_Spending,
    SUM(Profit) AS Total_Profit,
    COUNT(*) AS Total_Orders,
    ROUND(
        SUM(Profit) * 100.0 / SUM(Sales),
        2
    ) AS Profit_Margin_Percent
FROM Step1_Clean_Data
GROUP BY
    Customer_ID,
    Customer_Name,
    Region;

```

```

-- =====
-- PIPELINE 4: DATA QUALITY REPORT
-- =====

```

```

-- Query 17: Step 1 – Flag Data Quality Issues
DROP TABLE IF EXISTS DQ_Step1_Flagged_Rows;

CREATE TEMP TABLE DQ_Step1_Flagged_Rows AS
SELECT
    Order_ID,
    CASE
        WHEN Sales < 0 THEN 1
        ELSE 0
    END AS Has_Negative_Sales,
    CASE
        WHEN Quantity <= 0 THEN 1
        ELSE 0
    END AS Has_Invalid_Quantity,
    CASE
        WHEN Discount < 0 OR Discount > 1 THEN 1
        ELSE 0
    END AS Has_Invalid_Discount,
    CASE
        WHEN Customer_Rating IS NULL THEN 1
        ELSE 0
    END AS Has_Missing_Rating
FROM Dirty_Sales_Dataset;

```

```

-- Query 18: Verify Flagged Rows

```

```
SELECT *
FROM DQ_Step1_Flagged_Rows
LIMIT 10;
```

```
-- Query 19: Step 2 – Create Data Quality Summary
DROP TABLE IF EXISTS DQ_Step2_Summary;
```

```
CREATE TEMP TABLE DQ_Step2_Summary AS
SELECT
    SUM(Has_Negative_Sales) AS Total_Negative_Sales,
    SUM(Has_Invalid_Quantity) AS Total_Invalid_Quantity,
    SUM(Has_Invalid_Discount) AS Total_Invalid_Discount,
    SUM(Has_Missing_Rating) AS Total_Missing_Rating,
    COUNT(*) AS Total_Rows
FROM DQ_Step1_Flagged_Rows;
```

```
-- Query 20: Verify Data Quality Summary
SELECT *
FROM DQ_Step2_Summary;
```

```
-- Query 21: Step 3 – Negative Sales Percentage
SELECT
    'Negative Sales' AS Issue_Type,
    Total_Negative_Sales AS Issue_Count,
    ROUND(
        Total_Negative_Sales * 100.0 / Total_Rows,
        2
    ) AS Issue_Percentage
FROM DQ_Step2_Summary;
```

```
-- Query 22: Invalid Quantity Percentage
SELECT
    'Invalid Quantity' AS Issue_Type,
    Total_Invalid_Quantity AS Issue_Count,
    ROUND(
        Total_Invalid_Quantity * 100.0 / Total_Rows,
        2
    ) AS Issue_Percentage
FROM DQ_Step2_Summary;
```

```
-- Query 23: Invalid Discount Percentage
SELECT
    'Invalid Discount' AS Issue_Type,
    Total_Invalid_Discount AS Issue_Count,
    ROUND(
        Total_Invalid_Discount * 100.0 / Total_Rows,
        2
    ) AS Issue_Percentage
FROM DQ_Step2_Summary;
```

-- Query 24: Missing Rating Percentage

```
SELECT
  'Missing Rating' AS Issue_Type,
  Total_Missing_Rating AS Issue_Count,
  ROUND(
    Total_Missing_Rating * 100.0 / Total_Rows,
    2
  ) AS Issue_Percentage
FROM DQ_Step2_Summary;
```

-- Query 25: Complete Data Quality Percentage Report

```
SELECT
  'Negative Sales' AS Issue_Type,
  Total_Negative_Sales AS Issue_Count,
  ROUND(
    Total_Negative_Sales * 100.0 / Total_Rows,
    2
  ) AS Issue_Percentage
FROM DQ_Step2_Summary

UNION ALL

SELECT
  'Invalid Quantity',
  Total_Invalid_Quantity,
  ROUND(
    Total_Invalid_Quantity * 100.0 / Total_Rows,
    2
  )
FROM DQ_Step2_Summary

UNION ALL

SELECT
  'Invalid Discount',
  Total_Invalid_Discount,
  ROUND(
    Total_Invalid_Discount * 100.0 / Total_Rows,
    2
  )
FROM DQ_Step2_Summary

UNION ALL

SELECT
  'Missing Rating',
  Total_Missing_Rating,
  ROUND(
    Total_Missing_Rating * 100.0 / Total_Rows,
    2
  )
FROM DQ_Step2_Summary
```

ORDER BY Issue_Percentage DESC;

```
-- =====  
-- PIPELINE 5: NAMING CONVENTION  
-- =====
```

```
-- Query 26: Recommended Step Naming  
-- Step1_Clean_Data  
-- Step2_Customer_Aggregates  
-- Step3_Ranked_Customers  
-- Step4_Final_Report
```

```
-- Query 27: Avoid Unclear Table Names  
-- temp1  
-- data_final  
-- xyz_table
```

```
-- =====  
-- PIPELINE 6: COMPLETE DATA QUALITY PIPELINE  
-- =====
```

```
-- Query 28: Step 1 – Flag Data Quality Issues
```

DROP TABLE IF EXISTS DQ_Step1_Flagged_Rows;

**CREATE TEMP TABLE DQ_Step1_Flagged_Rows AS
SELECT**

```
    Order_ID,  
    CASE  
        WHEN Sales < 0 THEN 1  
        ELSE 0  
    END AS Has_Negative_Sales,  
    CASE  
        WHEN Quantity <= 0 THEN 1  
        ELSE 0  
    END AS Has_Invalid_Quantity,  
    CASE  
        WHEN Discount < 0 OR Discount > 1 THEN 1  
        ELSE 0  
    END AS Has_Invalid_Discount  
FROM Dirty_Sales_Dataset;
```

```
-- Query 29: Step 2 – Create Summary Table
```

DROP TABLE IF EXISTS DQ_Step2_Summary;

**CREATE TEMP TABLE DQ_Step2_Summary AS
SELECT**

```
    SUM(Has_Negative_Sales) AS Total_Negative_Sales,  
    SUM(Has_Invalid_Quantity) AS Total_Invalid_Quantity,  
    SUM(Has_Invalid_Discount) AS Total_Invalid_Discount,  
    COUNT(*) AS Total_Rows  
FROM DQ_Step1_Flagged_Rows;
```

-- Query 30: Final Data Quality Report

```
SELECT
    Total_Negative_Sales,
    Total_Invalid_Quantity,
    Total_Invalid_Discount,
    Total_Rows,
    ROUND(
        Total_Negative_Sales * 100.0 / Total_Rows,
        2
    ) AS Pct_Negative_Sales,
    ROUND(
        Total_Invalid_Quantity * 100.0 / Total_Rows,
        2
    ) AS Pct_Invalid_Quantity,
    ROUND(
        Total_Invalid_Discount * 100.0 / Total_Rows,
        2
    ) AS Pct_Invalid_Discount
FROM DQ_Step2_Summary;
```

```
-- =====
-- PIPELINE 7: FINAL CLEANUP
-- =====
```

-- Query 31: Drop All Temporary Tables

```
DROP TABLE IF EXISTS Step1_Clean_Data;

DROP TABLE IF EXISTS Step2_Customer_Aggregates;

DROP TABLE IF EXISTS Step3_Ranked_Customers;

DROP TABLE IF EXISTS Step4_Final_VIP_Report;

DROP TABLE IF EXISTS DQ_Step1_Flagged_Rows;

DROP TABLE IF EXISTS DQ_Step2_Summary;
```

D4.3.1 MATERIALIZED VIEW – CONCEPT AND IMPLEMENTATION

```
-- =====
-- SECTION 1: BASIC MATERIALIZED VIEW SYNTAX
-- PostgreSQL
-- =====
```

-- Query 1: Create a Materialized View

```
CREATE MATERIALIZED VIEW Region_Sales_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    AVG(Sales) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0
```

```
GROUP BY Region;
```

```
-- Query 2: Query the Materialized View
```

```
SELECT *  
FROM Region_Sales_Summary;
```

```
-- Query 3: Refresh the Materialized View
```

```
REFRESH MATERIALIZED VIEW Region_Sales_Summary;
```

```
-- Query 4: Query Updated Materialized View
```

```
SELECT *  
FROM Region_Sales_Summary;
```

```
-- Query 5: Drop the Materialized View
```

```
DROP MATERIALIZED VIEW IF EXISTS Region_Sales_Summary;
```

```
-- =====  
-- SECTION 2: SQLITE – MATERIALIZED VIEW SIMULATION  
-- =====
```

```
-- Query 6: Create Simulated Materialized View
```

```
CREATE TABLE Simulated_Materialized_View AS  
SELECT  
    Region,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Region;
```

```
-- Query 7: Query Simulated Materialized View
```

```
SELECT *  
FROM Simulated_Materialized_View;
```

```
-- Query 8: Refresh Simulation – Drop Existing Table
```

```
DROP TABLE IF EXISTS Simulated_Materialized_View;
```

```
-- Query 9: Refresh Simulation – Recreate Table
```

```
CREATE TABLE Simulated_Materialized_View AS  
SELECT  
    Region,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Region;
```

```
-- Query 10: Query Refreshed Simulated Materialized View
```

```

SELECT *
FROM Simulated_Materialized_View;

-- Query 11: Cleanup Simulated Materialized View
DROP TABLE IF EXISTS Simulated_Materialized_View;

-- =====
-- SECTION 3: REGION SALES SUMMARY
-- POSTGRESQL MATERIALIZED VIEW
-- =====

-- Query 12: Create Region Sales Materialized View
CREATE MATERIALIZED VIEW Region_Sales_Summary AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    AVG(Sales) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;

-- Query 13: Read Stored Region Sales Data
SELECT *
FROM Region_Sales_Summary;

-- Query 14: Refresh Region Sales Data
REFRESH MATERIALIZED VIEW Region_Sales_Summary;

-- Query 15: Read Updated Region Sales Data
SELECT *
FROM Region_Sales_Summary;

-- Query 16: Drop Region Sales Materialized View
DROP MATERIALIZED VIEW IF EXISTS Region_Sales_Summary;

-- =====
-- SECTION 4: POSTGRESQL REFRESH OPTIONS
-- =====

-- Query 17: Standard Refresh
REFRESH MATERIALIZED VIEW Region_Sales_Summary;

-- Query 18: Concurrent Refresh
-- Requires a suitable UNIQUE index on the materialized view.
REFRESH MATERIALIZED VIEW CONCURRENTLY Region_Sales_Summary;

```

```

-- Query 19: Create Unique Index for Concurrent Refresh
CREATE UNIQUE INDEX idx_region_sales_summary
ON Region_Sales_Summary (Region);

-- Query 20: Concurrent Refresh After Unique Index
REFRESH MATERIALIZED VIEW CONCURRENTLY Region_Sales_Summary;

-- =====
-- SECTION 5: SQLITE – CATEGORY SUMMARY SIMULATION
-- =====

-- Query 21: Create Category Summary
DROP TABLE IF EXISTS MV_Category_Summary;

CREATE TABLE MV_Category_Summary AS
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    ROUND(AVG(Sales), 2) AS Avg_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Category;

-- Query 22: Query Category Summary
SELECT *
FROM MV_Category_Summary
ORDER BY Total_Sales DESC;

-- Query 23: Refresh Category Summary
DROP TABLE IF EXISTS MV_Category_Summary;

CREATE TABLE MV_Category_Summary AS
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    ROUND(AVG(Sales), 2) AS Avg_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Category;

-- Query 24: Query Refreshed Category Summary
SELECT *
FROM MV_Category_Summary
ORDER BY Total_Sales DESC;

-- Query 25: Cleanup Category Summary

```

```
DROP TABLE IF EXISTS MV_Category_Summary;
```

```
-- =====  
-- SECTION 6: MATERIALIZED VIEW REFRESH WORKFLOW  
-- =====
```

```
-- Query 26: Create Stored Summary Table
```

```
DROP TABLE IF EXISTS MV_Daily_Sales_Summary;
```

```
CREATE TABLE MV_Daily_Sales_Summary AS  
SELECT
```

```
    Order_Date,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Total_Orders,  
    ROUND(AVG(Sales), 2) AS Avg_Order_Value  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Order_Date;
```

```
-- Query 27: Read Stored Summary
```

```
SELECT *  
FROM MV_Daily_Sales_Summary  
ORDER BY Order_Date;
```

```
-- Query 28: Refresh Stored Summary
```

```
DROP TABLE IF EXISTS MV_Daily_Sales_Summary;
```

```
CREATE TABLE MV_Daily_Sales_Summary AS  
SELECT
```

```
    Order_Date,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Total_Orders,  
    ROUND(AVG(Sales), 2) AS Avg_Order_Value  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Order_Date;
```

```
-- Query 29: Verify Refreshed Summary
```

```
SELECT *  
FROM MV_Daily_Sales_Summary  
ORDER BY Order_Date;
```

```
-- Query 30: Cleanup Stored Summary
```

```
DROP TABLE IF EXISTS MV_Daily_Sales_Summary;
```

```
-- =====  
-- SECTION 7: COMPLETE SQLITE PRACTICAL CYCLE  
-- =====
```

```
-- Query 31: Step 1 – Create Simulated Materialized View
DROP TABLE IF EXISTS MV_Category_Summary;
```

```
CREATE TABLE MV_Category_Summary AS
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    ROUND(AVG(Sales), 2) AS Avg_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Category;
```

```
-- Query 32: Step 2 – Query Stored Data
SELECT *
FROM MV_Category_Summary
ORDER BY Total_Sales DESC;
```

```
-- Query 33: Step 3 – Refresh by Dropping Old Data
DROP TABLE IF EXISTS MV_Category_Summary;
```

```
-- Query 34: Step 4 – Recreate with Latest Data
CREATE TABLE MV_Category_Summary AS
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count,
    ROUND(AVG(Sales), 2) AS Avg_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Category;
```

```
-- Query 35: Step 5 – Verify Refreshed Data
SELECT *
FROM MV_Category_Summary
ORDER BY Total_Sales DESC;
```

```
-- Query 36: Step 6 – Final Cleanup
DROP TABLE IF EXISTS MV_Category_Summary;
```

D4.3.2 REFRESH MATERIALIZED VIEW – REFRESH STRATEGIES

```
-- =====
-- SECTION 1: POSTGRESQL REFRESH
-- =====
```

```
-- Query 1: Simple / Complete Refresh
REFRESH MATERIALIZED VIEW Region_Sales_Summary;
```

```
-- Query 2: Concurrent Refresh
-- Requires a UNIQUE INDEX on the Materialized View.
REFRESH MATERIALIZED VIEW CONCURRENTLY Region_Sales_Summary;
```

```
-- Query 3: Create Unique Index for Concurrent Refresh
CREATE UNIQUE INDEX idx_region_summary
ON Region_Sales_Summary (Region);
```

```
-- Query 4: Concurrent Refresh After Creating Index
REFRESH MATERIALIZED VIEW CONCURRENTLY Region_Sales_Summary;
```

```
-- =====
-- SECTION 2: SQLITE – DROP + CREATE REFRESH
-- =====
```

```
-- Query 5: Drop Existing Simulated Materialized View
DROP TABLE IF EXISTS MV_Category_Summary;
```

```
-- Query 6: Recreate Materialized View Simulation
CREATE TABLE MV_Category_Summary AS
SELECT
    Category,
    SUM(Sales) AS Total_Sales,
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Category;
```

```
-- Query 7: Query Refreshed Category Summary
SELECT *
FROM MV_Category_Summary
ORDER BY Total_Sales DESC;
```

```
-- =====
-- SECTION 3: SQLITE – DELETE + INSERT REFRESH
-- =====
```

```
-- Query 8: Create Table Structure
CREATE TABLE IF NOT EXISTS MV_Category_Summary (
    Category TEXT,
    Total_Sales REAL,
    Order_Count INTEGER
);
```

```
-- Query 9: Delete Existing Data
DELETE FROM MV_Category_Summary;
```

-- Query 10: Insert Updated Aggregated Data

```
INSERT INTO MV_Category_Summary (  
    Category,  
    Total_Sales,  
    Order_Count  
)
```

```
SELECT  
    Category,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Category;
```

-- Query 11: Query Updated Summary

```
SELECT *  
FROM MV_Category_Summary  
ORDER BY Total_Sales DESC;
```

```
-- =====  
-- SECTION 4: SQLITE – STAGING TABLE + ATOMIC RENAME  
-- =====
```

-- Query 12: Create Staging Table

```
DROP TABLE IF EXISTS MV_Category_Summary_Staging;
```

```
CREATE TABLE MV_Category_Summary_Staging AS  
SELECT  
    Category,  
    SUM(Sales) AS Total_Sales,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Category;
```

-- Query 13: Rename Existing Table as Backup

```
DROP TABLE IF EXISTS MV_Category_Summary_Old;
```

```
ALTER TABLE MV_Category_Summary  
RENAME TO MV_Category_Summary_Old;
```

-- Query 14: Rename Staging Table as Live Table

```
ALTER TABLE MV_Category_Summary_Staging  
RENAME TO MV_Category_Summary;
```

-- Query 15: Delete Old Backup Table

```
DROP TABLE IF EXISTS MV_Category_Summary_Old;
```

-- Query 16: Query Refreshed Live Table

```
SELECT *
FROM MV_Category_Summary
ORDER BY Total_Sales DESC;
```

```
-- =====
-- SECTION 5: REGION PERFORMANCE REFRESH WORKFLOW
-- =====
```

```
-- Query 17: Initial Setup
DROP TABLE IF EXISTS MV_Region_Performance;
```

```
CREATE TABLE MV_Region_Performance AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales,
    SUM(Profit) AS Total_Profit,
    COUNT(*) AS Order_Count,
    ROUND(AVG(Sales), 2) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;
```

```
-- Query 18: Query Region Performance
SELECT *
FROM MV_Region_Performance
ORDER BY Total_Sales DESC;
```

```
-- Query 19: Find Regions with Negative Profit
SELECT *
FROM MV_Region_Performance
WHERE Total_Profit < 0;
```

```
-- Query 20: Create Region Performance Staging Table
DROP TABLE IF EXISTS MV_Region_Performance_Staging;
```

```
CREATE TABLE MV_Region_Performance_Staging AS
SELECT
    Region,
    SUM(Sales) AS Total_Sales,
    SUM(Profit) AS Total_Profit,
    COUNT(*) AS Order_Count,
    ROUND(AVG(Sales), 2) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region;
```

```
-- Query 21: Rename Existing Region Table
DROP TABLE IF EXISTS MV_Region_Performance_Old;
```

```
ALTER TABLE MV_Region_Performance
```

```
RENAME TO MV_Region_Performance_Old;
```

```
-- Query 22: Rename Staging Table to Live Table
```

```
ALTER TABLE MV_Region_Performance_Staging  
RENAME TO MV_Region_Performance;
```

```
-- Query 23: Remove Old Backup
```

```
DROP TABLE IF EXISTS MV_Region_Performance_Old;
```

```
-- Query 24: Query Refreshed Region Performance
```

```
SELECT *  
FROM MV_Region_Performance  
ORDER BY Total_Sales DESC;
```

```
-- =====  
-- SECTION 6: LAST REFRESHED TIMESTAMP  
-- =====
```

```
-- Query 25: Create Summary with Refresh Timestamp
```

```
DROP TABLE IF EXISTS MV_Region_Performance;
```

```
CREATE TABLE MV_Region_Performance AS  
SELECT  
    Region,  
    SUM(Sales) AS Total_Sales,  
    datetime('now') AS Last_Refreshed_At  
FROM Dirty_Sales_Dataset  
GROUP BY Region;
```

```
-- Query 26: Display Last Refreshed Timestamp
```

```
SELECT DISTINCT  
    Last_Refreshed_At  
FROM MV_Region_Performance;
```

```
-- Query 27: Query Summary with Timestamp
```

```
SELECT *  
FROM MV_Region_Performance  
ORDER BY Total_Sales DESC;
```

```
-- =====  
-- SECTION 7: POSTGRESQL AUTOMATIC REFRESH  
-- =====
```

```
-- Query 28: Schedule Daily Refresh Using pg_cron
```

```
SELECT cron.schedule(  
    'refresh-region-summary',  
    '0 2 * * *',  
    'REFRESH MATERIALIZED VIEW Region_Sales_Summary'
```

```
);
```

```
-- Query 29: Check Scheduled Jobs
```

```
SELECT *  
FROM cron.job;
```

```
-- Query 30: Remove Scheduled Refresh Job
```

```
SELECT cron.unschedule('refresh-region-summary');
```

```
-- =====  
-- SECTION 8: COMPLETE SQLITE REFRESH PRACTICE  
-- =====
```

```
-- Query 31: Step 1 – Initial Materialized View Simulation
```

```
DROP TABLE IF EXISTS MV_Test;
```

```
CREATE TABLE MV_Test AS
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count,  
    datetime('now') AS Refreshed_At  
FROM Dirty_Sales_Dataset  
GROUP BY Region;
```

```
-- Query 32: Step 2 – Query Initial Data
```

```
SELECT *  
FROM MV_Test;
```

```
-- Query 33: Step 3 – Create Updated Staging Table
```

```
DROP TABLE IF EXISTS MV_Test_Staging;
```

```
CREATE TABLE MV_Test_Staging AS
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count,  
    datetime('now') AS Refreshed_At  
FROM Dirty_Sales_Dataset  
GROUP BY Region;
```

```
-- Query 34: Step 4 – Rename Existing Table
```

```
DROP TABLE IF EXISTS MV_Test_Old;
```

```
ALTER TABLE MV_Test  
RENAME TO MV_Test_Old;
```

```
-- Query 35: Step 5 – Rename Staging Table to Live Table
```

```
ALTER TABLE MV_Test_Staging  
RENAME TO MV_Test;
```

```

-- Query 36: Step 6 – Delete Old Backup
DROP TABLE IF EXISTS MV_Test_Old;

-- Query 37: Step 7 – Verify Updated Timestamp
SELECT *
FROM MV_Test;

-- =====
-- SECTION 9: FINAL CLEANUP
-- =====

-- Query 38: Cleanup Category Summary
DROP TABLE IF EXISTS MV_Category_Summary;

-- Query 39: Cleanup Region Performance Summary
DROP TABLE IF EXISTS MV_Region_Performance;

-- Query 40: Cleanup Test Summary
DROP TABLE IF EXISTS MV_Test;

```

D4.3.3 WHEN TO USE MATERIALIZED VIEWS IN DATA SCIENCE PIPELINES

```

-- Query 1 --
-- Feature Engineering: Pre-Computed Customer-Level Features

CREATE TABLE MV_Customer_Features AS
SELECT
    Customer_ID,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Lifetime_Value,
    AVG(Sales) AS Avg_Order_Value,
    MAX(Order_Date) AS Last_Order_Date,
    julianday('now') - julianday(MAX(Order_Date)) AS Days_Since_Last_Order,
    COUNT(DISTINCT Category) AS Category_Diversity
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Customer_ID;

-- Query 2 --
-- Daily Sales Dashboard Aggregation

CREATE TABLE MV_Daily_Sales_Dashboard AS
SELECT
    Order_Date,
    Region,
    Category,
    SUM(Sales) AS Total_Sales,
    SUM(Profit) AS Total_Profit,

```

```
    COUNT(*) AS Order_Count
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Order_Date, Region, Category;
```

```
-- Query 3 --
-- Training Dataset Snapshot for Reproducibility
```

```
CREATE TABLE MV_Training_Dataset_Snapshot AS
SELECT
    Customer_ID,
    Total_Orders,
    Lifetime_Value,
    Avg_Order_Value,
    Days_Since_Last_Order,
    CASE
        WHEN Days_Since_Last_Order > 90 THEN 1
        ELSE 0
    END AS Churn_Label
FROM MV_Customer_Features;
```

```
-- Query 4 --
-- Real-Time Fraud Detection
-- Materialized View should NOT be used here
```

```
-- Fraud detection requires fresh and real-time transaction data.
-- Use a direct query, normal view, or real-time streaming system.
```

```
-- Query 5 --
-- Exploratory Data Analysis (EDA)
```

```
SELECT DISTINCT
    Region
FROM Dirty_Sales_Dataset;
```

```
SELECT
    COUNT(*)
FROM Dirty_Sales_Dataset
WHERE Sales < 0;
```

```
-- Query 6 --
-- Feature Store Pattern: Transactional Features
```

```
CREATE TABLE MV_Customer_Transactional_Features AS
SELECT
    Customer_ID,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Spend
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

```
-- Query 7 --  
-- Feature Store Pattern: Behavioral Features
```

```
CREATE TABLE MV_Customer_Behavioral_Features AS  
SELECT  
    Customer_ID,  
    AVG(  
        julianday(Ship_Date) - julianday(Order_Date)  
    ) AS Avg_Shipping_Wait,  
    COUNT(  
        CASE  
            WHEN Return_Status = 'Yes' THEN 1  
        END  
    ) AS Return_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Customer_ID;
```

```
-- Query 8 --  
-- Combine Feature Groups into a Feature Matrix
```

```
SELECT  
    t.Customer_ID,  
    t.Order_Count,  
    t.Total_Spend,  
    b.Avg_Shipping_Wait,  
    b.Return_Count  
FROM MV_Customer_Transactional_Features t  
JOIN MV_Customer_Behavioral_Features b  
    ON t.Customer_ID = b.Customer_ID;
```

```
-- Query 9 --  
-- A/B Test Results Snapshot
```

```
CREATE TABLE MV_AB_Test_Snapshot_2025_06_19 AS  
SELECT  
    Customer_ID,  
    CASE  
        WHEN Customer_ID LIKE '%1' THEN 'Group_A'  
        ELSE 'Group_B'  
    END AS Test_Group,  
    SUM(Sales) AS Total_Sales_During_Test  
FROM Dirty_Sales_Dataset  
WHERE Order_Date BETWEEN '2025-06-01' AND '2025-06-19'  
GROUP BY Customer_ID;
```

```
-- Query 10 --  
-- Versioning Materialized Views
```

```
CREATE TABLE MV_Customer_Features_20250619 AS  
SELECT ...;
```

```
CREATE TABLE MV_Customer_Features_20250620 AS
SELECT ...;
```

```
-- Query 11 --
-- Feature Store Practice: Transactional Features
```

```
CREATE TABLE MV_Cust_Features_Transactional AS
SELECT
    Customer_ID,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Lifetime_Value,
    ROUND(AVG(Sales), 2) AS Avg_Order_Value
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Customer_ID;
```

```
-- Query 12 --
-- Feature Store Practice: Recency Features
```

```
CREATE TABLE MV_Cust_Features_Recency AS
SELECT
    Customer_ID,
    MAX(Order_Date) AS Last_Order_Date,
    CAST(
        julianday('2025-06-19') - julianday(MAX(Order_Date))
        AS INTEGER
    ) AS Days_Since_Last_Order
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;
```

```
-- Query 13 --
-- Combine Feature Tables into a Model-Ready Dataset
```

```
SELECT
    t.Customer_ID,
    t.Total_Orders,
    t.Lifetime_Value,
    t.Avg_Order_Value,
    r.Days_Since_Last_Order,
    CASE
        WHEN r.Days_Since_Last_Order > 90 THEN 1
        ELSE 0
    END AS Likely_Churned
FROM MV_Cust_Features_Transactional t
JOIN MV_Cust_Features_Recency r
    ON t.Customer_ID = r.Customer_ID
LIMIT 20;
```

D4.3.4 MATERIALIZED VIEW VS REGULAR VIEW – PERFORMANCE DIFFERENCE

```
-- Query 1 --
-- Regular View: Simple Filter
```

```

CREATE VIEW Simple_North_View AS
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

-- Query the Regular View
SELECT *
FROM Simple_North_View
WHERE Sales > 5000;

-- Query 2 --
-- Regular View: Heavy Customer Aggregation

```

```

CREATE VIEW Heavy_Customer_Analysis AS
SELECT
    Customer_ID,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales,
    AVG(Discount) AS Avg_Discount,
    SUM(
        CASE
            WHEN Return_Status = 'Yes' THEN 1
            ELSE 0
        END
    ) AS Return_Count,
    MAX(Order_Date) AS Last_Order_Date
FROM Dirty_Sales_Dataset
GROUP BY Customer_ID;

```

```

-- Query 3 --
-- Regular View: JOIN + Window Function

```

```

CREATE VIEW Complex_Sales_Report AS
SELECT
    d.Order_ID,
    d.Sales,
    s.Region_Assigned,
    s.Years_Experience,
    d.Sales - AVG(d.Sales) OVER (
        PARTITION BY d.Region
    ) AS Diff_From_Region_Avg
FROM Dirty_Sales_Dataset d
JOIN Salesperson_Info s
    ON d.Salesperson = s.Salesperson;

```

```

-- Query 4 --
-- Create Regular View for Performance Testing

```

```

CREATE VIEW RV_Customer_Analysis AS
SELECT
    Customer_ID,

```

```

    Customer_Name,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales,
    MAX(Order_Date) AS Last_Order,
    MIN(Order_Date) AS First_Order
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Customer_ID, Customer_Name;

```

```

-- Query the Regular View
SELECT *
FROM RV_Customer_Analysis
ORDER BY Total_Sales DESC
LIMIT 20;

```

```

-- Query 5 --
-- Simulated Materialized View Using a Table

```

```

CREATE TABLE MV_Customer_Analysis AS
SELECT
    Customer_ID,
    Customer_Name,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS Avg_Sales,
    MAX(Order_Date) AS Last_Order,
    MIN(Order_Date) AS First_Order
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Customer_ID, Customer_Name;

```

```

-- Query the Materialized View Table
SELECT *
FROM MV_Customer_Analysis
ORDER BY Total_Sales DESC
LIMIT 20;

```

```

-- Query 6 --
-- Repeated Queries on Regular View

```

```

SELECT COUNT(*)
FROM RV_Customer_Analysis
WHERE Total_Sales > 20000;

```

```

SELECT COUNT(*)
FROM RV_Customer_Analysis
WHERE Order_Count > 5;

```

```

SELECT AVG(Total_Sales)
FROM RV_Customer_Analysis;

```

```
-- Query 7 --  
-- Repeated Queries on Materialized View Table
```

```
SELECT COUNT(*)  
FROM MV_Customer_Analysis  
WHERE Total_Sales > 20000;
```

```
SELECT COUNT(*)  
FROM MV_Customer_Analysis  
WHERE Order_Count > 5;
```

```
SELECT AVG(Total_Sales)  
FROM MV_Customer_Analysis;
```

```
-- Query 8 --  
-- Create Index on Materialized View Table
```

```
CREATE INDEX idx_mv_customer  
ON MV_Customer_Analysis(Customer_ID);
```

```
-- Filter Using Indexed Column  
SELECT *  
FROM MV_Customer_Analysis  
WHERE Customer_ID = 'CUST05177';
```

```
-- Query 9 --  
-- Final Regular View Performance Test
```

```
CREATE VIEW RV_Final_Test AS  
SELECT  
    Region,  
    Category,  
    SUM(Sales) AS Total,  
    COUNT(*) AS Cnt,  
    AVG(Profit_Margin) AS Avg_Margin  
FROM Dirty_Sales_Dataset  
GROUP BY Region, Category;
```

```
-- Run Multiple Queries on Regular View
```

```
SELECT *  
FROM RV_Final_Test  
ORDER BY Total DESC;
```

```
SELECT *  
FROM RV_Final_Test  
WHERE Cnt > 1000;
```

```
SELECT SUM(Total)  
FROM RV_Final_Test;
```

```
-- Query 10 --  
-- Final Materialized View Performance Test
```

```

CREATE TABLE MV_Final_Test AS
SELECT
    Region,
    Category,
    SUM(Sales) AS Total,
    COUNT(*) AS Cnt,
    AVG(Profit_Margin) AS Avg_Margin
FROM Dirty_Sales_Dataset
GROUP BY Region, Category;

-- Run the Same Queries on Materialized Table
SELECT *
FROM MV_Final_Test
ORDER BY Total DESC;

SELECT *
FROM MV_Final_Test
WHERE Cnt > 1000;

SELECT SUM(Total)
FROM MV_Final_Test;

```

```

-- Query 11 --
-- Cleanup: Drop Test Views and Tables

```

```

DROP VIEW IF EXISTS Simple_North_View;

DROP VIEW IF EXISTS Heavy_Customer_Analysis;

DROP VIEW IF EXISTS Complex_Sales_Report;

DROP VIEW IF EXISTS RV_Customer_Analysis;

DROP VIEW IF EXISTS RV_Final_Test;

DROP TABLE IF EXISTS MV_Customer_Analysis;

DROP TABLE IF EXISTS MV_Final_Test;

```

D5.1.1 HOW SQL ENGINE PROCESSES A QUERY – LOGICAL ORDER VS WRITTEN ORDER

```

-- Query 1 --
-- Written Order of SQL Query

```

```

SELECT
    column1,
    column2
FROM table_name
WHERE condition
GROUP BY column
HAVING condition
ORDER BY column
LIMIT 10;

```

```
-- Query 2 --  
-- Logical Order Demonstration
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Region  
HAVING COUNT(*) > 5000  
ORDER BY Order_Count DESC  
LIMIT 3;
```

```
-- Query 3 --  
-- Column Alias in WHERE: Error Example
```

```
SELECT  
    Sales,  
    (Sales * 1.1) AS Adjusted_Sales  
FROM Dirty_Sales_Dataset  
WHERE Adjusted_Sales > 5000;
```

```
-- Query 4 --  
-- Column Alias in WHERE: Correct Approach
```

```
SELECT  
    Sales,  
    (Sales * 1.1) AS Adjusted_Sales  
FROM Dirty_Sales_Dataset  
WHERE (Sales * 1.1) > 5000;
```

```
-- Query 5 --  
-- Alias in HAVING: SQLite Example
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Region  
HAVING Order_Count > 5000;
```

```
-- Query 6 --  
-- Portable HAVING Syntax
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Region
```

```
HAVING COUNT(*) > 5000;
```

```
-- Query 7 --  
-- Aggregate Function in WHERE: Error Example
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
WHERE COUNT(*) > 100  
GROUP BY Region;
```

```
-- Query 8 --  
-- Aggregate Filter Using HAVING
```

```
SELECT  
    Region,  
    COUNT(*) AS Order_Count  
FROM Dirty_Sales_Dataset  
GROUP BY Region  
HAVING COUNT(*) > 100;
```

```
-- Query 9 --  
-- FROM + JOIN Before WHERE
```

```
SELECT  
    d.Order_ID,  
    d.Sales,  
    s.Region_Assigned  
FROM Dirty_Sales_Dataset d  
JOIN Salesperson_Info s  
    ON d.Salesperson = s.Salesperson  
WHERE d.Sales > 5000;
```

```
-- Query 10 --  
-- SELECT DISTINCT Logical Order
```

```
SELECT DISTINCT  
    Region  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
ORDER BY Region;
```

```
-- Query 11 --  
-- WHERE Filter Before GROUP BY
```

```
SELECT  
    Category,  
    SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset
```

```
WHERE Sales > 0
GROUP BY Category
HAVING SUM(Sales) > 1000000
ORDER BY Total_Sales DESC;
```

```
-- Query 12 --
-- WHERE with Aggregate: Error Test
```

```
SELECT
    Category,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE SUM(Sales) > 1000000
GROUP BY Category;
```

```
-- Query 13 --
-- WHERE with SELECT Alias: Error Test
```

```
SELECT
    Sales,
    Sales * 2 AS Double_Sales
FROM Dirty_Sales_Dataset
WHERE Double_Sales > 10000;
```

```
-- Query 14 --
-- Logical Order Practice: Multiple Clauses
```

```
SELECT
    Region,
    Category,
    COUNT(*) AS Order_Count,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region, Category
HAVING SUM(Sales) > 100000
ORDER BY Total_Sales DESC
LIMIT 10;
```

```
-- Query 15 --
-- Logical Order Practice: DISTINCT + Filter + Sorting
```

```
SELECT DISTINCT
    Category
FROM Dirty_Sales_Dataset
WHERE Sales > 0
ORDER BY Category;
```

```
-- Query 16 --
-- Logical Order Practice: Region-Level Sales
```

```

SELECT
    Region,
    COUNT(*) AS Total_Orders,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
WHERE Sales > 0
GROUP BY Region
HAVING SUM(Sales) > 500000
ORDER BY Total_Sales DESC
LIMIT 3;

```

D5.1.2 EXPLAIN / EXPLAIN ANALYZE – READING QUERY EXECUTION PLAN

```

-- Query 1 --
-- Basic EXPLAIN QUERY PLAN in SQLite

```

```

EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

```

```

-- Query 2 --
-- Simple Filter: Plan Before Index

```

```

EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

```

```

-- Query 3 --
-- Create Index on Region

```

```

CREATE INDEX idx_region
ON Dirty_Sales_Dataset(Region);

```

```

-- Query 4 --
-- Simple Filter: Plan After Index

```

```

EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

```

```

-- Query 5 --
-- JOIN Query Execution Plan

```

```

EXPLAIN QUERY PLAN
SELECT
    d.Order_ID,
    d.Sales,

```

```
        s.Region_Assigned
FROM Dirty_Sales_Dataset d
JOIN Salesperson_Info s
    ON d.Salesperson = s.Salesperson;
```

```
-- Query 6 --
-- GROUP BY Query Execution Plan
```

```
EXPLAIN QUERY PLAN
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 7 --
-- ORDER BY Query Execution Plan
```

```
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
ORDER BY Sales DESC
LIMIT 10;
```

```
-- Query 8 --
-- PostgreSQL EXPLAIN ANALYZE Example
-- This query is for PostgreSQL reference only
```

```
EXPLAIN ANALYZE
SELECT
    Region,
    SUM(Sales) AS Total_Sales
FROM Dirty_Sales_Dataset
GROUP BY Region;
```

```
-- Query 9 --
-- Before Index: Customer_ID Execution Plan
```

```
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Customer_ID = 'CUST05177';
```

```
-- Query 10 --
-- Create Index on Customer_ID
```

```
CREATE INDEX idx_customer_id
ON Dirty_Sales_Dataset(Customer_ID);
```

```
-- Query 11 --  
-- After Index: Customer_ID Execution Plan
```

```
EXPLAIN QUERY PLAN  
SELECT *  
FROM Dirty_Sales_Dataset  
WHERE Customer_ID = 'CUST05177';
```

```
-- Query 12 --  
-- Compare Query Plan with Aggregation and Filter
```

```
EXPLAIN QUERY PLAN  
SELECT  
    Region,  
    COUNT(*) AS Order_Count,  
    SUM(Sales) AS Total_Sales  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0  
GROUP BY Region  
HAVING SUM(Sales) > 100000;
```

```
-- Query 13 --  
-- Query Plan for Multiple Filters
```

```
EXPLAIN QUERY PLAN  
SELECT *  
FROM Dirty_Sales_Dataset  
WHERE Region = 'North'  
    AND Sales > 5000;
```

```
-- Query 14 --  
-- Query Plan for Sorting and Filtering
```

```
EXPLAIN QUERY PLAN  
SELECT  
    Order_ID,  
    Customer_ID,  
    Sales  
FROM Dirty_Sales_Dataset  
WHERE Sales > 1000  
ORDER BY Sales DESC  
LIMIT 20;
```

```
-- Query 15 --  
-- Query Plan for DISTINCT
```

```
EXPLAIN QUERY PLAN  
SELECT DISTINCT  
    Region  
FROM Dirty_Sales_Dataset  
WHERE Sales > 0
```

```
ORDER BY Region;
```

```
-- Query 16 --  
-- Query Plan for JOIN + Filter
```

```
EXPLAIN QUERY PLAN  
SELECT  
    d.Order_ID,  
    d.Customer_ID,  
    d.Sales,  
    s.Region_Assigned  
FROM Dirty_Sales_Dataset d  
JOIN Salesperson_Info s  
    ON d.Salesperson = s.Salesperson  
WHERE d.Sales > 5000;
```

```
-- Query 17 --  
-- Cleanup: Drop Created Indexes
```

```
DROP INDEX IF EXISTS idx_region;
```

```
DROP INDEX IF EXISTS idx_customer_id;
```

D5.1.3 Seq Scan vs **Index** Scan – What They Mean

```
-- Query 1: Check Sequential Scan Without Index --  
EXPLAIN QUERY PLAN  
SELECT *  
FROM Dirty_Sales_Dataset  
WHERE Product_ID = 'P00927';
```

```
-- Query 2: Create Index and Check Index Scan --  
CREATE INDEX idx_product_id  
ON Dirty_Sales_Dataset(Product_ID);
```

```
EXPLAIN QUERY PLAN  
SELECT *  
FROM Dirty_Sales_Dataset  
WHERE Product_ID = 'P00927';
```

```
-- Query 3: Index Scan with Range Query --  
CREATE INDEX idx_sales  
ON Dirty_Sales_Dataset(Sales);
```

```
EXPLAIN QUERY PLAN  
SELECT *  
FROM Dirty_Sales_Dataset  
WHERE Sales > 50000;
```

```
-- Query 4: Function on Column May Cause Sequential Scan --
```

```
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE UPPER(Product_ID) = 'P00927';
```

```
-- Query 5: LIKE with Leading Wildcard --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Product_ID LIKE '%927';
```

```
-- Query 6: LIKE with Trailing Wildcard --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Product_ID LIKE 'P009%';
```

```
-- Query 7: Broad Filter – Sequential Scan May Be Better --
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0;
```

```
-- Query 8: High Selectivity Query --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Customer_ID = 'CUST05177';
```

```
-- Query 9: Low Selectivity Query --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0;
```

```
-- Query 10: Check Query Plan for Product_ID --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Product_ID = 'P00927';
```

```
-- Query 11: Check Query Plan for Customer_ID --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Customer_ID = 'CUST05177';
```

```
-- Query 12: Check Query Plan for Broad Sales Filter --
```

```
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0;
```

```
-- Query 13: Create Customer_ID Index --
CREATE INDEX idx_customer_id
ON Dirty_Sales_Dataset(Customer_ID);
```

```
-- Query 14: Compare Selective Customer Query --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Customer_ID = 'CUST05177';
```

```
-- Query 15: Compare Non-Selective Sales Query --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0;
```

```
-- Query 16: Remove Indexes After Testing --
DROP INDEX IF EXISTS idx_product_id;

DROP INDEX IF EXISTS idx_sales;

DROP INDEX IF EXISTS idx_customer_id;
```

D5.1.4 Cost Estimation in Query Planner

```
-- Query 1: PostgreSQL EXPLAIN – View Estimated Cost --
-- PostgreSQL mein (SQLite mein directly nahi chalega)
EXPLAIN
SELECT *
FROM dirty_sales_dataset
WHERE region = 'North';
```

```
-- Query 2: Create Region Index --
CREATE INDEX idx_region
ON Dirty_Sales_Dataset(Region);
```

```
-- High Selectivity Query
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';
```

```
-- Query 3: Create Customer_ID Index --
CREATE INDEX idx_customer
```

```

ON Dirty_Sales_Dataset(Customer_ID);

-- Highly Selective Query
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Customer_ID = 'CUST05177';

-- Query 4: Update Statistics Using ANALYZE --
ANALYZE Dirty_Sales_Dataset;

-- Check Query Plan After ANALYZE
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

-- Query 5: Create Multiple Indexes --
CREATE INDEX idx_region
ON Dirty_Sales_Dataset(Region);

CREATE INDEX idx_category
ON Dirty_Sales_Dataset(Category);

-- Compare Multiple Indexes
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
AND Category = 'Electronics';

-- Query 6: Check Unique Values and Total Rows --
SELECT
    COUNT(DISTINCT Region) AS Unique_Regions,
    COUNT(DISTINCT Customer_ID) AS Unique_Customers,
    COUNT(DISTINCT Product_ID) AS Unique_Products,
    COUNT(*) AS Total_Rows
FROM Dirty_Sales_Dataset;

-- Query 7: Update Table Statistics --
ANALYZE Dirty_Sales_Dataset;

-- Query 8: Create Index for Region Testing --
CREATE INDEX idx_region_test
ON Dirty_Sales_Dataset(Region);

-- Query 9: Create Index for Customer Testing --
CREATE INDEX idx_customer_test
ON Dirty_Sales_Dataset(Customer_ID);

```

```

-- Query 10: Check Query Plan for Region --
-- Low Selectivity
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North';

-- Query 11: Check Query Plan for Customer_ID --
-- High Selectivity
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Customer_ID = 'CUST05177';

-- Query 12: Calculate Selectivity for Region --
SELECT
    COUNT(DISTINCT Region) * 1.0 / COUNT(*) AS Region_Selectivity
FROM Dirty_Sales_Dataset;

-- Query 13: Calculate Selectivity for Customer_ID --
SELECT
    COUNT(DISTINCT Customer_ID) * 1.0 / COUNT(*) AS Customer_Selectivity
FROM Dirty_Sales_Dataset;

-- Query 14: Compare Region and Customer_ID Selectivity --
SELECT
    COUNT(DISTINCT Region) AS Unique_Regions,
    COUNT(DISTINCT Customer_ID) AS Unique_Customers,
    COUNT(*) AS Total_Rows,
    COUNT(DISTINCT Region) * 1.0 / COUNT(*) AS Region_Selectivity,
    COUNT(DISTINCT Customer_ID) * 1.0 / COUNT(*) AS Customer_Selectivity
FROM Dirty_Sales_Dataset;

-- Query 15: Check Query Plan for Multiple Filters --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Region = 'North'
    AND Category = 'Electronics';

-- Query 16: Check Query Plan for Broad Filter --
EXPLAIN QUERY PLAN
SELECT *
FROM Dirty_Sales_Dataset
WHERE Sales > 0;

```

```
-- Query 17: Remove Test Indexes --  
DROP INDEX IF EXISTS idx_region;  
  
DROP INDEX IF EXISTS idx_category;  
  
DROP INDEX IF EXISTS idx_customer;  
  
DROP INDEX IF EXISTS idx_region_test;  
  
DROP INDEX IF EXISTS idx_customer_test;
```